TurfBooking Project Code Reference

This PDF was generated automatically from the project code files.
Order: serial number, file path, then source code.

Included folders: src, backend
Included code types: .vue, .js, .css, .php

==========
1. backend/add_turf.php
==========

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

include 'db_connection.php';

function send_response($success, $status, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'status' => $status
    ], $extra));
    exit();
}

function save_turf_image_file($uploadedFile, $turfId) {
    if (!is_array($uploadedFile)) {
        return ['ok' => false, 'error' => 'Turf image is required.'];
    }

    if (($uploadedFile['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        return ['ok' => false, 'error' => 'Image upload failed. Please select a valid image.'];
    }

    $maxSize = 5 * 1024 * 1024; // 5MB
    if (($uploadedFile['size'] ?? 0) > $maxSize) {
        return ['ok' => false, 'error' => 'Image size must be 5MB or less.'];
    }

    $originalName = (string)($uploadedFile['name'] ?? '');
    $ext = strtolower(pathinfo($originalName, PATHINFO_EXTENSION));
    $allowedExt = ['jpg', 'jpeg', 'png', 'webp'];
    if (!in_array($ext, $allowedExt, true)) {
        return ['ok' => false, 'error' => 'Only JPG, PNG, and WEBP images are allowed.'];
    }

    $tmpPath = (string)($uploadedFile['tmp_name'] ?? '');
    if ($tmpPath === '' || !is_uploaded_file($tmpPath)) {
        return ['ok' => false, 'error' => 'Invalid uploaded file.'];
    }

    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mime = $finfo ? finfo_file($finfo, $tmpPath) : '';
    if ($finfo) finfo_close($finfo);

    $allowedMime = ['image/jpeg', 'image/png', 'image/webp'];
    if (!in_array($mime, $allowedMime, true)) {
        return ['ok' => false, 'error' => 'Uploaded file is not a valid image.'];
    }

    $uploadDir = __DIR__ . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'turf_images';
    if (!is_dir($uploadDir) && !mkdir($uploadDir, 0755, true)) {
        return ['ok' => false, 'error' => 'Server cannot create image upload directory.'];
```

```
    }

    $safeRandom = bin2hex(random_bytes(6));
    $fileName = 'turf_' . (int)$turfId . '_' . time() . '_' . $safeRandom . '.' . $ext;
    $absolutePath = $uploadDir . DIRECTORY_SEPARATOR . $fileName;
    $relativeUrl = 'uploads/turf_images/' . $fileName;

    if (!move_uploaded_file($tmpPath, $absolutePath)) {
        return ['ok' => false, 'error' => 'Failed to save uploaded image.'];
    }

    return [
        'ok' => true,
        'absolute_path' => $absolutePath,
        'relative_url' => $relativeUrl
    ];
}

function get_owner_id_by_email($conn, $ownerEmail) {
    if ($ownerEmail === '') return 0;

    $stmt = $conn->prepare("SELECT owner_id FROM turf_owners WHERE LOWER(email) = LOWER(?) LIMIT 1");
    if (!$stmt) return 0;

    $stmt->bind_param('s', $ownerEmail);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();

    return $row && isset($row['owner_id']) ? (int)$row['owner_id'] : 0;
}

function find_user_for_owner_seed($conn, $ownerId, $ownerEmail) {
    if ($ownerEmail !== '') {
        $stmt = $conn->prepare("SELECT full_name, email, password_hash FROM users WHERE LOWER(email) =
LOWER(?) LIMIT 1");
        if ($stmt) {
            $stmt->bind_param('s', $ownerEmail);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row) return $row;
        }
    }

    if ($ownerId > 0) {
        $stmt = $conn->prepare("SELECT full_name, email, password_hash FROM users WHERE user_id = ? LIMIT 1");
        if ($stmt) {
            $stmt->bind_param('i', $ownerId);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row) return $row;
        }
    }

    return null;
}

function create_owner_from_user_seed($conn, $userSeed) {
    if (!$userSeed || !is_array($userSeed)) return 0;

    $ownerEmail = strtolower(trim((string)($userSeed['email'] ?? '')));
    if ($ownerEmail === '') return 0;

    $existingOwnerId = get_owner_id_by_email($conn, $ownerEmail);
    if ($existingOwnerId > 0) return $existingOwnerId;

    $ownerName = trim((string)($userSeed['full_name'] ?? 'Owner'));
```

```
    if ($ownerName === '') $ownerName = 'Owner';

    $passwordHash = trim((string)($userSeed['password_hash'] ?? ''));
    if ($passwordHash === '') {
        $passwordHash = password_hash(uniqid('owner_seed_', true), PASSWORD_DEFAULT);
    }

    $stmt = $conn->prepare("INSERT INTO turf_owners (owner_name, email, password_hash, status) VALUES (?, ?,
?, 'verified')");
    if (!$stmt) return 0;

    $stmt->bind_param('sss', $ownerName, $ownerEmail, $passwordHash);
    $ok = $stmt->execute();
    $insertedId = $ok ? (int)$stmt->insert_id : 0;
    $stmt->close();

    if ($insertedId > 0) return $insertedId;

    return get_owner_id_by_email($conn, $ownerEmail);
}

function find_owner_id($conn, $ownerId, $ownerEmail) {
    if ($ownerId > 0) {
        $stmt = $conn->prepare("SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1");
        if ($stmt) {
            $stmt->bind_param('i', $ownerId);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row && isset($row['owner_id'])) {
                return (int)$row['owner_id'];
            }
        }
    }

    $ownerIdFromEmail = get_owner_id_by_email($conn, $ownerEmail);
    if ($ownerIdFromEmail > 0) {
        return $ownerIdFromEmail;
    }

    $userSeed = find_user_for_owner_seed($conn, $ownerId, $ownerEmail);
    return create_owner_from_user_seed($conn, $userSeed);
}

$contentType = $_SERVER['CONTENT_TYPE'] ?? '';
if (stripos($contentType, 'application/json') !== false) {
    $data = json_decode(file_get_contents('php://input'), true);
} else {
    $data = $_POST;
    if (empty($data)) {
        parse_str(file_get_contents('php://input'), $data);
    }
}

if (!is_array($data) || empty($data)) {
    send_response(false, 'No data received.', 400);
}

$owner_id = intval($data['owner_id'] ?? 0);
$owner_email = strtolower(trim((string)($data['owner_email'] ?? '')));
$turf_name = trim((string)($data['turf_name'] ?? ''));
$sport_type = strtolower(trim((string)($data['sport_type'] ?? 'football')));
$address = trim((string)($data['address'] ?? ''));
$area = trim((string)($data['area'] ?? ''));
$city = trim((string)($data['city'] ?? ''));
$location = trim((string)($data['location'] ?? ''));
$latitude = trim((string)($data['latitude'] ?? ''));
$longitude = trim((string)($data['longitude'] ?? ''));
$price_per_hour = $data['price_per_hour'] ?? null;
$description = trim((string)($data['description'] ?? ''));
$is_featured = intval($data['is_featured'] ?? 0) === 1 ? 1 : 0;
```

```
$status = 'pending';
$cancel_before_hours = intval($data['cancel_before_hours'] ?? 24);
$refund_percent = intval($data['refund_percent'] ?? 80);

if ($turf_name === '') {
    send_response(false, 'Turf name is required.', 400);
}

if ($address === '') {
    send_response(false, 'Address is required.', 400);
}

if (!is_numeric($price_per_hour) || floatval($price_per_hour) <= 0) {
    send_response(false, 'Price per hour must be a positive number.', 400);
}

$price_per_hour = floatval($price_per_hour);

$allowedSports = ['football', 'cricket', 'badminton', 'basketball', 'tennis', 'other'];
if (!in_array($sport_type, $allowedSports, true)) {
    $sport_type = 'other';
}

// Owner turf always goes to admin verification queue first.
$status = 'pending';

if ($cancel_before_hours < 0) {
    $cancel_before_hours = 24;
}

if ($refund_percent < 0 || $refund_percent > 100) {
    $refund_percent = 80;
}

if ($latitude !== '' && !is_numeric($latitude)) {
    send_response(false, 'Latitude must be numeric.', 400);
}

if ($longitude !== '' && !is_numeric($longitude)) {
    send_response(false, 'Longitude must be numeric.', 400);
}

if ($latitude !== '') {
    $latValue = floatval($latitude);
    if ($latValue < -90 || $latValue > 90) {
        send_response(false, 'Latitude must be between -90 and 90.', 400);
    }
}

if ($longitude !== '') {
    $lngValue = floatval($longitude);
    if ($lngValue < -180 || $lngValue > 180) {
        send_response(false, 'Longitude must be between -180 and 180.', 400);
    }
}

$resolved_owner_id = find_owner_id($conn, $owner_id, $owner_email);
if ($resolved_owner_id <= 0) {
    send_response(false, 'Owner account not found in turf_owners table.', 400);
}

if (!isset($_FILES['turf_image'])) {
    send_response(false, 'Turf image is required.', 400);
}

$sql = "INSERT INTO turfs (
    owner_id,
    turf_name,
    sport_type,
    address,
    area,
    city,
```

```
    location,
    latitude,
    longitude,
    price_per_hour,
    description,
    is_featured,
    status,
    cancel_before_hours,
    refund_percent
) VALUES (
    ?, ?, ?, ?, NULLIF(?, ''), NULLIF(?, ''), NULLIF(?, ''), NULLIF(?, ''), NULLIF(?, ''), ?, NULLIF(?, ''),
?, ?, ?, ?
)";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while preparing turf insert.', 500);
}

$stmt->bind_param(
    'isssssssdsisii',
    $resolved_owner_id,
    $turf_name,
    $sport_type,
    $address,
    $area,
    $city,
    $location,
    $latitude,
    $longitude,
    $price_per_hour,
    $description,
    $is_featured,
    $status,
    $cancel_before_hours,
    $refund_percent
);

$conn->begin_transaction();

if (!$stmt->execute()) {
    $errorText = $stmt->error;
    $stmt->close();
    $conn->rollback();
    send_response(false, 'Failed to add turf. ' . $errorText, 500);
}

$turfId = (int)$stmt->insert_id;
$stmt->close();

$imageResult = save_turf_image_file($_FILES['turf_image'], $turfId);
if (!$imageResult['ok']) {
    $conn->rollback();
    send_response(false, $imageResult['error'], 400);
}

$imageUrl = (string)$imageResult['relative_url'];
$insertImageSql = "INSERT INTO turf_images (turf_id, image_url, is_primary) VALUES (?, ?, 1)";
$imageStmt = $conn->prepare($insertImageSql);

if (!$imageStmt) {
    if (file_exists($imageResult['absolute_path'])) {
        @unlink($imageResult['absolute_path']);
    }
    $conn->rollback();
    send_response(false, 'Server error while saving turf image metadata.', 500);
}

$imageStmt->bind_param('is', $turfId, $imageUrl);
if (!$imageStmt->execute()) {
    $imgError = $imageStmt->error;
    $imageStmt->close();
```

```
    if (file_exists($imageResult['absolute_path'])) {
        @unlink($imageResult['absolute_path']);
    }
    $conn->rollback();
    send_response(false, 'Failed to save turf image. ' . $imgError, 500);
}

$imageStmt->close();
$conn->commit();

send_response(true, 'Turf submitted for admin verification.', 200, [
    'turf_id' => $turfId,
    'image_url' => $imageUrl
]);
?>
```

==========
2. backend/admin_booking_action.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'admin_common.php';

$data = read_request_data();
if (empty($data)) {
    send_json_response(false, 'No data received.', 400);
}

$adminId = intval($data['admin_id'] ?? 0);
$bookingId = intval($data['booking_id'] ?? 0);
$action = strtolower(trim((string)($data['action'] ?? '')));

if (!admin_exists($conn, $adminId)) {
    send_json_response(false, 'Admin access denied.', 403);
}

if ($bookingId <= 0) {
    send_json_response(false, 'Invalid booking id.', 400);
}

if ($action === 'approve') $action = 'confirm';
if ($action === 'decline') $action = 'reject';

if ($action !== 'confirm' && $action !== 'reject') {
    send_json_response(false, 'Invalid action.', 400);
}

$targetStatus = $action === 'confirm' ? 'confirmed' : 'cancelled';

$checkStmt = $conn->prepare('SELECT booking_id, booking_status FROM bookings WHERE booking_id = ? LIMIT 1');
if (!$checkStmt) {
    send_json_response(false, 'Server error while loading booking.', 500);
}
$checkStmt->bind_param('i', $bookingId);
$checkStmt->execute();
$checkRes = $checkStmt->get_result();
$booking = $checkRes ? $checkRes->fetch_assoc() : null;
```

```
$checkStmt->close();

if (!$booking) {
    send_json_response(false, 'Booking not found.', 404);
}

$currentStatus = strtolower((string)($booking['booking_status'] ?? ''));

if ($currentStatus === $targetStatus) {
    $pointsAwarded = 0;
    if ($targetStatus === 'confirmed') {
        $pointResult = award_confirm_points($conn, $bookingId);
        if (!$pointResult['ok']) {
            send_json_response(false, 'Booking already confirmed, but points sync failed: ' .
$pointResult['reason'], 500);
        }
        $pointsAwarded = (int)($pointResult['points'] ?? 0);
    }

    send_json_response(true, 'Booking already in requested status.', 200, [
        'booking_id' => $bookingId,
        'status' => $targetStatus,
        'points_awarded' => $pointsAwarded
    ]);
}

if ($currentStatus !== 'pending') {
    send_json_response(false, 'Only pending bookings can be processed.', 400, [
        'current_status' => $currentStatus
    ]);
}

$conn->begin_transaction();

$updateStmt = $conn->prepare('UPDATE bookings SET booking_status = ? WHERE booking_id = ? AND booking_status =
 ? LIMIT 1');
if (!$updateStmt) {
    $conn->rollback();
    send_json_response(false, 'Server error while updating booking.', 500);
}
$pending = 'pending';
$updateStmt->bind_param('sis', $targetStatus, $bookingId, $pending);
$ok = $updateStmt->execute();
$affected = $updateStmt->affected_rows;
$updateStmt->close();

if (!$ok || $affected <= 0) {
    $conn->rollback();
    send_json_response(false, 'Booking status update failed.', 500);
}

$pointsAwarded = 0;
if ($targetStatus === 'confirmed') {
    $pointResult = award_confirm_points($conn, $bookingId);
    if (!$pointResult['ok']) {
        $conn->rollback();
        send_json_response(false, 'Booking updated but points failed: ' . $pointResult['reason'], 500);
    }
    $pointsAwarded = (int)($pointResult['points'] ?? 0);
}

$conn->commit();

send_json_response(true, 'Booking request processed successfully.', 200, [
    'booking_id' => $bookingId,
    'status' => $targetStatus,
    'points_awarded' => $pointsAwarded
]);
?>
```

==========
3. backend/admin_common.php
==========

```php
<?php
require_once 'db_connection.php';

function send_json_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

function column_exists($conn, $table, $column) {
    $safeTable = preg_replace('/[^a-zA-Z0-9_]/', '', $table);
    if ($safeTable === '') {
        return false;
    }

    $safeColumn = $conn->real_escape_string($column);
    $res = $conn->query("SHOW COLUMNS FROM `$safeTable` LIKE '$safeColumn'");
    return $res && $res->num_rows > 0;
}

function read_request_data() {
    $contentType = $_SERVER['CONTENT_TYPE'] ?? '';
    if (stripos($contentType, 'application/json') !== false) {
        $data = json_decode(file_get_contents('php://input'), true);
        return is_array($data) ? $data : [];
    }

    $data = $_POST;
    if (!is_array($data) || empty($data)) {
        parse_str(file_get_contents('php://input'), $data);
    }

    return is_array($data) ? $data : [];
}

function admin_exists($conn, $adminId) {
    if ($adminId <= 0) {
        return false;
    }

    if (table_exists($conn, 'admins')) {
        $stmt = $conn->prepare('SELECT admin_id FROM admins WHERE admin_id = ? LIMIT 1');
        if ($stmt) {
            $stmt->bind_param('i', $adminId);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row && isset($row['admin_id'])) {
                return true;
            }
        }
    }

    if (table_exists($conn, 'users') && column_exists($conn, 'users', 'role')) {
        $stmt = $conn->prepare("SELECT user_id FROM users WHERE user_id = ? AND role = 'admin' LIMIT 1");
        if ($stmt) {
            $stmt->bind_param('i', $adminId);
            $stmt->execute();
```

```
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            return $row && isset($row['user_id']);
        }
    }

    return false;
}

function award_confirm_points($conn, $bookingId) {
    $bookingId = (int)$bookingId;
    if ($bookingId <= 0) {
        return ['ok' => false, 'points' => 0, 'reason' => 'Invalid booking id'];
    }

    $stmt = $conn->prepare('SELECT user_id, booked_price, booking_status FROM bookings WHERE booking_id = ?
LIMIT 1');
    if (!$stmt) {
        return ['ok' => false, 'points' => 0, 'reason' => 'Failed to load booking'];
    }

    $stmt->bind_param('i', $bookingId);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();

    if (!$row) {
        return ['ok' => false, 'points' => 0, 'reason' => 'Booking not found'];
    }

    $status = strtolower((string)($row['booking_status'] ?? ''));
    if ($status !== 'confirmed' && $status !== 'completed') {
        return ['ok' => false, 'points' => 0, 'reason' => 'Booking is not confirmed'];
    }

    $userId = (int)($row['user_id'] ?? 0);
    $bookedPrice = (float)($row['booked_price'] ?? 0);
    $points = (int)floor($bookedPrice / 100);

    if ($userId <= 0 || $points <= 0) {
        return ['ok' => true, 'points' => 0, 'reason' => 'No points applicable'];
    }

    $note = 'booking_confirm:' . $bookingId;
    $alreadyLogged = false;

    if (table_exists($conn, 'user_point_logs') && column_exists($conn, 'user_point_logs', 'user_id') &&
column_exists($conn, 'user_point_logs', 'note')) {
        $checkStmt = $conn->prepare('SELECT log_id FROM user_point_logs WHERE user_id = ? AND note = ? LIMIT
1');
        if ($checkStmt) {
            $checkStmt->bind_param('is', $userId, $note);
            $checkStmt->execute();
            $checkRes = $checkStmt->get_result();
            $alreadyLogged = $checkRes && $checkRes->num_rows > 0;
            $checkStmt->close();
        }
    }

    if ($alreadyLogged) {
        return ['ok' => true, 'points' => 0, 'reason' => 'Points already awarded'];
    }

    if (table_exists($conn, 'user_points') && column_exists($conn, 'user_points', 'user_id')) {
        if (column_exists($conn, 'user_points', 'total_points') && column_exists($conn, 'user_points',
'booking_points')) {
            $upsert = "INSERT INTO user_points (user_id, total_points, booking_points)
                       VALUES (?, ?, ?)
                       ON DUPLICATE KEY UPDATE
                         total_points = total_points + VALUES(total_points),
```

```
                        booking_points = booking_points + VALUES(booking_points)";
            $upsertStmt = $conn->prepare($upsert);
            if ($upsertStmt) {
                $upsertStmt->bind_param('iii', $userId, $points, $points);
                $upsertStmt->execute();
                $upsertStmt->close();
            }
        } elseif (column_exists($conn, 'user_points', 'points')) {
            $upsert = "INSERT INTO user_points (user_id, points)
                       VALUES (?, ?)
                       ON DUPLICATE KEY UPDATE points = points + VALUES(points)";
            $upsertStmt = $conn->prepare($upsert);
            if ($upsertStmt) {
                $upsertStmt->bind_param('ii', $userId, $points);
                $upsertStmt->execute();
                $upsertStmt->close();
            }
        }
    }

    if (table_exists($conn, 'user_point_logs') && column_exists($conn, 'user_point_logs', 'user_id')) {
        $inserted = false;

        if (column_exists($conn, 'user_point_logs', 'source') && column_exists($conn, 'user_point_logs',
'points')) {
            $source = 'booking';
            if (column_exists($conn, 'user_point_logs', 'note')) {
                $logSql = 'INSERT INTO user_point_logs (user_id, source, points, note) VALUES (?, ?, ?, ?)';
                $logStmt = $conn->prepare($logSql);
                if ($logStmt) {
                    $logStmt->bind_param('isis', $userId, $source, $points, $note);
                    $inserted = $logStmt->execute();
                    $logStmt->close();
                }
            } else {
                $logSql = 'INSERT INTO user_point_logs (user_id, source, points) VALUES (?, ?, ?)';
                $logStmt = $conn->prepare($logSql);
                if ($logStmt) {
                    $logStmt->bind_param('isi', $userId, $source, $points);
                    $inserted = $logStmt->execute();
                    $logStmt->close();
                }
            }
        } elseif (column_exists($conn, 'user_point_logs', 'action') && column_exists($conn, 'user_point_logs',
 'points_change')) {
            $action = 'booking';
            if (column_exists($conn, 'user_point_logs', 'note')) {
                $logSql = 'INSERT INTO user_point_logs (user_id, action, points_change, note) VALUES (?, ?, ?,
 ?)';
                $logStmt = $conn->prepare($logSql);
                if ($logStmt) {
                    $logStmt->bind_param('isis', $userId, $action, $points, $note);
                    $inserted = $logStmt->execute();
                    $logStmt->close();
                }
            } else {
                $logSql = 'INSERT INTO user_point_logs (user_id, action, points_change) VALUES (?, ?, ?)';
                $logStmt = $conn->prepare($logSql);
                if ($logStmt) {
                    $logStmt->bind_param('isi', $userId, $action, $points);
                    $inserted = $logStmt->execute();
                    $logStmt->close();
                }
            }
        }

        if (!$inserted && !column_exists($conn, 'user_point_logs', 'note')) {
            return ['ok' => true, 'points' => 0, 'reason' => 'Could not insert point log'];
        }
    }

    return ['ok' => true, 'points' => $points, 'reason' => 'Points awarded'];
```

```
}
?>
```

==========
4. backend/admin_dashboard.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'admin_common.php';

$adminId = intval($_GET['admin_id'] ?? 0);
if (!admin_exists($conn, $adminId)) {
    send_json_response(false, 'Admin access denied.', 403);
}

$stats = [
    'users_active' => 0,
    'users_banned' => 0,
    'owners_verified' => 0,
    'owners_suspended' => 0,
    'owners_pending' => 0,
    'pending_turfs' => 0,
    'pending_bookings' => 0,
    'pending_refunds' => 0,
    'today_revenue' => 0
];

$monitorUsers = [];
$monitorOwners = [];
$pendingTurfs = [];
$pendingBookings = [];
$pendingRefunds = [];
$analytics = [
    'top_rated_turfs' => [],
    'most_active_users' => [],
    'revenue_trend' => [],
    'booking_growth' => []
];

if (table_exists($conn, 'users')) {
    $sql = "SELECT
              SUM(CASE WHEN status = 'active' THEN 1 ELSE 0 END) AS users_active,
              SUM(CASE WHEN status = 'banned' THEN 1 ELSE 0 END) AS users_banned
            FROM users";
    $res = $conn->query($sql);
    $row = $res ? $res->fetch_assoc() : null;
    if ($row) {
        $stats['users_active'] = (int)($row['users_active'] ?? 0);
        $stats['users_banned'] = (int)($row['users_banned'] ?? 0);
    }
}

if (table_exists($conn, 'turf_owners')) {
    $sql = "SELECT
              SUM(CASE WHEN status = 'verified' THEN 1 ELSE 0 END) AS owners_verified,
              SUM(CASE WHEN status = 'suspended' THEN 1 ELSE 0 END) AS owners_suspended,
              SUM(CASE WHEN status = 'pending' THEN 1 ELSE 0 END) AS owners_pending
            FROM turf_owners";
    $res = $conn->query($sql);
```

```
    $row = $res ? $res->fetch_assoc() : null;
    if ($row) {
        $stats['owners_verified'] = (int)($row['owners_verified'] ?? 0);
        $stats['owners_suspended'] = (int)($row['owners_suspended'] ?? 0);
        $stats['owners_pending'] = (int)($row['owners_pending'] ?? 0);
    }
}

if (table_exists($conn, 'turfs')) {
    $res = $conn->query("SELECT COUNT(*) AS c FROM turfs WHERE status = 'pending'");
    $row = $res ? $res->fetch_assoc() : null;
    $stats['pending_turfs'] = (int)($row['c'] ?? 0);
}

if (table_exists($conn, 'bookings')) {
    $res = $conn->query("SELECT COUNT(*) AS c FROM bookings WHERE booking_status = 'pending'");
    $row = $res ? $res->fetch_assoc() : null;
    $stats['pending_bookings'] = (int)($row['c'] ?? 0);

    $revRes = $conn->query("SELECT COALESCE(SUM(booked_price), 0) AS amount
                            FROM bookings
                            WHERE booking_status IN ('confirmed', 'completed')
                              AND DATE(created_at) = CURDATE()");
    $revRow = $revRes ? $revRes->fetch_assoc() : null;
    $stats['today_revenue'] = (float)($revRow['amount'] ?? 0);
}

if (table_exists($conn, 'refund_requests')) {
    $res = $conn->query("SELECT COUNT(*) AS c FROM refund_requests WHERE status = 'pending'");
    $row = $res ? $res->fetch_assoc() : null;
    $stats['pending_refunds'] = (int)($row['c'] ?? 0);
}

if (table_exists($conn, 'users') && table_exists($conn, 'bookings')) {
    $userSql = "SELECT
                  u.user_id,
                  u.full_name,
                  u.email,
                  u.status,
                  COUNT(b.booking_id) AS total_bookings,
                  SUM(CASE WHEN b.booking_status = 'confirmed' THEN 1 ELSE 0 END) AS confirmed_bookings,
                  SUM(CASE WHEN b.booking_status = 'completed' THEN 1 ELSE 0 END) AS completed_bookings,
                  MAX(b.created_at) AS last_booking_at
                FROM users u
                LEFT JOIN bookings b ON b.user_id = u.user_id
                GROUP BY u.user_id, u.full_name, u.email, u.status
                ORDER BY total_bookings DESC, u.user_id DESC
                LIMIT 12";
    $userRes = $conn->query($userSql);
    while ($row = $userRes ? $userRes->fetch_assoc() : null) {
        if (!$row) break;
        $monitorUsers[] = [
            'user_id' => (int)$row['user_id'],
            'full_name' => $row['full_name'],
            'email' => $row['email'],
            'status' => $row['status'],
            'total_bookings' => (int)($row['total_bookings'] ?? 0),
            'confirmed_bookings' => (int)($row['confirmed_bookings'] ?? 0),
            'completed_bookings' => (int)($row['completed_bookings'] ?? 0),
            'last_booking_at' => $row['last_booking_at']
        ];
    }
}

if (table_exists($conn, 'turf_owners') && table_exists($conn, 'turfs')) {
    $ownerSql = "SELECT
                  o.owner_id,
                  o.owner_name,
                  o.email,
                  o.status,
                  COUNT(t.turf_id) AS total_turfs,
                  SUM(CASE WHEN t.status = 'active' THEN 1 ELSE 0 END) AS active_turfs,
```

```
                  SUM(CASE WHEN t.status = 'pending' THEN 1 ELSE 0 END) AS pending_turfs
                FROM turf_owners o
                LEFT JOIN turfs t ON t.owner_id = o.owner_id
                GROUP BY o.owner_id, o.owner_name, o.email, o.status
                ORDER BY total_turfs DESC, o.owner_id DESC
                LIMIT 12";
    $ownerRes = $conn->query($ownerSql);
    while ($row = $ownerRes ? $ownerRes->fetch_assoc() : null) {
        if (!$row) break;
        $monitorOwners[] = [
            'owner_id' => (int)$row['owner_id'],
            'owner_name' => $row['owner_name'],
            'email' => $row['email'],
            'status' => $row['status'],
            'total_turfs' => (int)($row['total_turfs'] ?? 0),
            'active_turfs' => (int)($row['active_turfs'] ?? 0),
            'pending_turfs' => (int)($row['pending_turfs'] ?? 0)
        ];
    }
}

if (table_exists($conn, 'turfs') && table_exists($conn, 'turf_owners')) {
    $turfSql = "SELECT
                  t.turf_id,
                  t.turf_name,
                  t.sport_type,
                  t.city,
                  t.area,
                  t.address,
                  t.price_per_hour,
                  t.status,
                  t.created_at,
                  o.owner_id,
                  o.owner_name,
                  o.email AS owner_email
                FROM turfs t
                JOIN turf_owners o ON o.owner_id = t.owner_id
                WHERE t.status = 'pending'
                ORDER BY t.created_at ASC, t.turf_id ASC
                LIMIT 50";
    $turfRes = $conn->query($turfSql);
    while ($row = $turfRes ? $turfRes->fetch_assoc() : null) {
        if (!$row) break;
        $pendingTurfs[] = [
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name'],
            'sport_type' => $row['sport_type'],
            'city' => $row['city'],
            'area' => $row['area'],
            'address' => $row['address'],
            'price_per_hour' => (float)($row['price_per_hour'] ?? 0),
            'status' => $row['status'],
            'created_at' => $row['created_at'],
            'owner_id' => (int)$row['owner_id'],
            'owner_name' => $row['owner_name'],
            'owner_email' => $row['owner_email']
        ];
    }
}

if (table_exists($conn, 'bookings') && table_exists($conn, 'users') && table_exists($conn, 'slots') &&
table_exists($conn, 'turfs')) {
    $bookSql = "SELECT
                  b.booking_id,
                  b.user_id,
                  b.slot_id,
                  b.booking_status,
                  b.booked_price,
                  b.created_at,
                  u.full_name,
                  u.email,
                  s.slot_date,
```

```
                  s.start_time,
                  s.end_time,
                  t.turf_id,
                  t.turf_name
                FROM bookings b
                JOIN users u ON u.user_id = b.user_id
                JOIN slots s ON s.slot_id = b.slot_id
                JOIN turfs t ON t.turf_id = s.turf_id
                WHERE b.booking_status = 'pending'
                ORDER BY b.created_at ASC, b.booking_id ASC
                LIMIT 60";
    $bookRes = $conn->query($bookSql);
    while ($row = $bookRes ? $bookRes->fetch_assoc() : null) {
        if (!$row) break;
        $pendingBookings[] = [
            'booking_id' => (int)$row['booking_id'],
            'user_id' => (int)$row['user_id'],
            'slot_id' => (int)$row['slot_id'],
            'booking_status' => $row['booking_status'],
            'booked_price' => (float)($row['booked_price'] ?? 0),
            'created_at' => $row['created_at'],
            'user_name' => $row['full_name'],
            'user_email' => $row['email'],
            'slot_date' => $row['slot_date'],
            'start_time' => $row['start_time'],
            'end_time' => $row['end_time'],
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name']
        ];
    }
}

if (table_exists($conn, 'refund_requests') && table_exists($conn, 'bookings') && table_exists($conn, 'users'))
 {
    $refundSql = "SELECT
                    r.refund_id,
                    r.booking_id,
                    r.requested_by,
                    r.requested_amount,
                    r.status,
                    r.admin_note,
                    r.created_at,
                    b.user_id,
                    b.booking_status,
                    b.booked_price,
                    u.full_name,
                    u.email
                  FROM refund_requests r
                  JOIN bookings b ON b.booking_id = r.booking_id
                  JOIN users u ON u.user_id = b.user_id
                  WHERE r.status = 'pending'
                  ORDER BY r.created_at ASC, r.refund_id ASC
                  LIMIT 60";
    $refundRes = $conn->query($refundSql);
    while ($row = $refundRes ? $refundRes->fetch_assoc() : null) {
        if (!$row) break;
        $pendingRefunds[] = [
            'refund_id' => (int)$row['refund_id'],
            'booking_id' => (int)$row['booking_id'],
            'requested_by' => $row['requested_by'],
            'requested_amount' => (float)($row['requested_amount'] ?? 0),
            'status' => $row['status'],
            'admin_note' => $row['admin_note'],
            'created_at' => $row['created_at'],
            'user_id' => (int)$row['user_id'],
            'user_name' => $row['full_name'],
            'user_email' => $row['email'],
            'booking_status' => $row['booking_status'],
            'booked_price' => (float)($row['booked_price'] ?? 0)
        ];
    }
}
```

```
if (table_exists($conn, 'turfs')) {
    $topTurfSql = "SELECT turf_id, turf_name, rating_avg, city, price_per_hour
                   FROM turfs
                   WHERE status = 'active'
                   ORDER BY rating_avg DESC, turf_id DESC
                   LIMIT 6";
    $topRes = $conn->query($topTurfSql);
    while ($row = $topRes ? $topRes->fetch_assoc() : null) {
        if (!$row) break;
        $analytics['top_rated_turfs'][] = [
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name'],
            'rating_avg' => (float)($row['rating_avg'] ?? 0),
            'city' => $row['city'],
            'price_per_hour' => (float)($row['price_per_hour'] ?? 0)
        ];
    }
}

if (table_exists($conn, 'bookings') && table_exists($conn, 'users')) {
    $activeUserSql = "SELECT u.user_id, u.full_name, u.email, COUNT(b.booking_id) AS booking_count
                      FROM users u
                      JOIN bookings b ON b.user_id = u.user_id
                      GROUP BY u.user_id, u.full_name, u.email
                      ORDER BY booking_count DESC, u.user_id DESC
                      LIMIT 6";
    $activeRes = $conn->query($activeUserSql);
    while ($row = $activeRes ? $activeRes->fetch_assoc() : null) {
        if (!$row) break;
        $analytics['most_active_users'][] = [
            'user_id' => (int)$row['user_id'],
            'full_name' => $row['full_name'],
            'email' => $row['email'],
            'booking_count' => (int)($row['booking_count'] ?? 0)
        ];
    }

    $trendSql = "SELECT DATE_FORMAT(created_at, '%Y-%m') AS ym,
                        COUNT(*) AS booking_count,
                        COALESCE(SUM(CASE WHEN booking_status IN ('confirmed', 'completed') THEN booked_price
ELSE 0 END), 0) AS revenue
                 FROM bookings
                 WHERE created_at >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
                 GROUP BY DATE_FORMAT(created_at, '%Y-%m')
                 ORDER BY ym ASC";
    $trendRes = $conn->query($trendSql);
    while ($row = $trendRes ? $trendRes->fetch_assoc() : null) {
        if (!$row) break;
        $analytics['revenue_trend'][] = [
            'month' => $row['ym'],
            'revenue' => (float)($row['revenue'] ?? 0)
        ];
        $analytics['booking_growth'][] = [
            'month' => $row['ym'],
            'bookings' => (int)($row['booking_count'] ?? 0)
        ];
    }
}

send_json_response(true, 'Admin dashboard loaded.', 200, [
    'stats' => $stats,
    'monitor_users' => $monitorUsers,
    'monitor_owners' => $monitorOwners,
    'pending_turfs' => $pendingTurfs,
    'pending_bookings' => $pendingBookings,
    'pending_refunds' => $pendingRefunds,
    'analytics' => $analytics
]);
?>
```

==========
5. backend/admin_refund_action.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'admin_common.php';

function ensure_owner_wallet_tables($conn) {
    if (!table_exists($conn, 'turf_owners')) {
        return;
    }

    $conn->query("CREATE TABLE IF NOT EXISTS owner_wallets (
        owner_wallet_id INT AUTO_INCREMENT PRIMARY KEY,
        owner_id INT NOT NULL UNIQUE,
        balance DECIMAL(10,2) NOT NULL DEFAULT 0.00,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        CONSTRAINT fk_owner_wallet_owner FOREIGN KEY (owner_id) REFERENCES turf_owners(owner_id)
          ON DELETE CASCADE ON UPDATE CASCADE
    ) ENGINE=InnoDB");

    $conn->query("CREATE TABLE IF NOT EXISTS owner_wallet_transactions (
        owner_wallet_txn_id INT AUTO_INCREMENT PRIMARY KEY,
        owner_id INT NOT NULL,
        txn_type ENUM('cancellation_share','booking_income','withdrawal','adjustment') NOT NULL DEFAULT
'cancellation_share',
        amount DECIMAL(10,2) NOT NULL,
        reference_note VARCHAR(255),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_owner_wallet_txn_owner FOREIGN KEY (owner_id) REFERENCES turf_owners(owner_id)
          ON DELETE CASCADE ON UPDATE CASCADE
    ) ENGINE=InnoDB");
}

function resolve_admin_id_for_fk($conn, $adminId) {
    if ($adminId <= 0 || !table_exists($conn, 'admins')) {
        return 0;
    }

    $stmt = $conn->prepare('SELECT admin_id FROM admins WHERE admin_id = ? LIMIT 1');
    if (!$stmt) {
        return 0;
    }

    $stmt->bind_param('i', $adminId);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();

    return (int)($row['admin_id'] ?? 0);
}

$data = read_request_data();
if (empty($data)) {
    send_json_response(false, 'No data received.', 400);
}

$adminId = intval($data['admin_id'] ?? 0);
$refundId = intval($data['refund_id'] ?? 0);
$action = strtolower(trim((string)($data['action'] ?? '')));
```

```
$adminNote = trim((string)($data['admin_note'] ?? ''));

if (!admin_exists($conn, $adminId)) {
    send_json_response(false, 'Admin access denied.', 403);
}

if ($refundId <= 0) {
    send_json_response(false, 'Invalid refund id.', 400);
}

if ($action === 'process') $action = 'paid';
if ($action === 'approve') $action = 'approved';
if ($action === 'reject') $action = 'rejected';

$allowed = ['approved', 'rejected', 'paid'];
if (!in_array($action, $allowed, true)) {
    send_json_response(false, 'Invalid refund action.', 400);
}

$persistAdminId = resolve_admin_id_for_fk($conn, $adminId);

$sql = "SELECT
          r.refund_id,
          r.booking_id,
          r.status,
          r.requested_amount,
          b.user_id,
          b.booked_price,
          b.booking_status,
          t.owner_id,
          t.turf_name
        FROM refund_requests r
        JOIN bookings b ON b.booking_id = r.booking_id
        JOIN slots s ON s.slot_id = b.slot_id
        JOIN turfs t ON t.turf_id = s.turf_id
        WHERE r.refund_id = ?
        LIMIT 1";
$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_json_response(false, 'Server error while loading refund request.', 500);
}
$stmt->bind_param('i', $refundId);
$stmt->execute();
$res = $stmt->get_result();
$row = $res ? $res->fetch_assoc() : null;
$stmt->close();

if (!$row) {
    send_json_response(false, 'Refund request not found.', 404);
}

$currentStatus = strtolower((string)($row['status'] ?? 'pending'));
$bookingId = (int)$row['booking_id'];
$userId = (int)$row['user_id'];
$ownerId = (int)$row['owner_id'];
$turfName = (string)($row['turf_name'] ?? 'Turf');
$bookedPrice = (float)($row['booked_price'] ?? 0);

if ($bookedPrice <= 0) {
    send_json_response(false, 'Invalid booking amount for refund processing.', 400);
}

$ownerShare = round($bookedPrice * 0.20, 2);
$userRefund = round($bookedPrice - $ownerShare, 2);

if ($action === 'rejected') {
    if ($currentStatus === 'rejected') {
        send_json_response(true, 'Refund request already rejected.', 200, [
            'refund_id' => $refundId,
            'status' => 'rejected'
        ]);
    }
```

```
    if ($persistAdminId > 0) {
        $updateSql = 'UPDATE refund_requests SET status = ?, admin_id = ?, admin_note = ?, updated_at = NOW()
WHERE refund_id = ? LIMIT 1';
        $updateStmt = $conn->prepare($updateSql);
        if (!$updateStmt) {
            send_json_response(false, 'Server error while rejecting refund.', 500);
        }
        $rejected = 'rejected';
        $updateStmt->bind_param('sisi', $rejected, $persistAdminId, $adminNote, $refundId);
    } else {
        $updateSql = 'UPDATE refund_requests SET status = ?, admin_id = NULL, admin_note = ?, updated_at =
NOW() WHERE refund_id = ? LIMIT 1';
        $updateStmt = $conn->prepare($updateSql);
        if (!$updateStmt) {
            send_json_response(false, 'Server error while rejecting refund.', 500);
        }
        $rejected = 'rejected';
        $updateStmt->bind_param('ssi', $rejected, $adminNote, $refundId);
    }

    $ok = $updateStmt->execute();
    $updateStmt->close();

    if (!$ok) {
        send_json_response(false, 'Failed to reject refund request.', 500);
    }

    $extra = [
        'refund_id' => $refundId,
        'status' => 'rejected'
    ];
    if ($persistAdminId <= 0) {
        $extra['warning'] = 'Admin id is not in admins table, so admin_id stored as NULL.';
    }

    send_json_response(true, 'Refund request rejected.', 200, $extra);
}

if ($currentStatus === 'approved' || $currentStatus === 'paid') {
    send_json_response(true, 'Refund already processed.', 200, [
        'refund_id' => $refundId,
        'status' => $currentStatus,
        'booking_id' => $bookingId,
        'refund_amount' => $userRefund,
        'owner_share' => $ownerShare
    ]);
}

$inTransaction = false;

try {
    $conn->begin_transaction();
    $inTransaction = true;

    $finalStatus = 'paid';
    $finalNote = $adminNote !== '' ? $adminNote : 'Admin approved cancellation refund payout';

    if ($persistAdminId > 0) {
        $refundUpdateSql = 'UPDATE refund_requests SET status = ?, requested_amount = ?, admin_id = ?,
admin_note = ?, updated_at = NOW() WHERE refund_id = ? LIMIT 1';
        $refundUpdateStmt = $conn->prepare($refundUpdateSql);
        if (!$refundUpdateStmt) {
            throw new RuntimeException('Server error while updating refund status.');
        }
        $refundUpdateStmt->bind_param('sdisi', $finalStatus, $userRefund, $persistAdminId, $finalNote,
$refundId);
    } else {
        $refundUpdateSql = 'UPDATE refund_requests SET status = ?, requested_amount = ?, admin_id = NULL,
admin_note = ?, updated_at = NOW() WHERE refund_id = ? LIMIT 1';
        $refundUpdateStmt = $conn->prepare($refundUpdateSql);
        if (!$refundUpdateStmt) {
```

```
            throw new RuntimeException('Server error while updating refund status.');
        }
        $refundUpdateStmt->bind_param('sdsi', $finalStatus, $userRefund, $finalNote, $refundId);
    }

    $ok = $refundUpdateStmt->execute();
    $refundUpdateStmt->close();
    if (!$ok) {
        throw new RuntimeException('Failed to update refund request.');
    }

    $bookingUpdateSql = "UPDATE bookings
                         SET booking_status = 'cancelled', refund_amount = ?
                         WHERE booking_id = ?
                         LIMIT 1";
    $bookingStmt = $conn->prepare($bookingUpdateSql);
    if ($bookingStmt) {
        $bookingStmt->bind_param('di', $userRefund, $bookingId);
        $bookingStmt->execute();
        $bookingStmt->close();
    }

    if (table_exists($conn, 'wallets')) {
        $initWalletSql = 'INSERT INTO wallets (user_id, balance) VALUES (?, 0) ON DUPLICATE KEY UPDATE user_id
 = user_id';
        $initStmt = $conn->prepare($initWalletSql);
        if ($initStmt) {
            $initStmt->bind_param('i', $userId);
            $initStmt->execute();
            $initStmt->close();
        }

        $walletUpdateSql = 'UPDATE wallets SET balance = balance + ? WHERE user_id = ?';
        $walletStmt = $conn->prepare($walletUpdateSql);
        if ($walletStmt) {
            $walletStmt->bind_param('di', $userRefund, $userId);
            $walletStmt->execute();
            $walletStmt->close();
        }

        if (table_exists($conn, 'wallet_transactions')) {
            $txnType = 'refund';
            $note = 'Refund for booking #' . $bookingId . ' approved by admin';
            $txnSql = 'INSERT INTO wallet_transactions (user_id, txn_type, amount, reference_note) VALUES (?,
?, ?, ?)';
            $txnStmt = $conn->prepare($txnSql);
            if ($txnStmt) {
                $txnStmt->bind_param('isds', $userId, $txnType, $userRefund, $note);
                $txnStmt->execute();
                $txnStmt->close();
            }
        }
    }

    if ($ownerId > 0) {
        ensure_owner_wallet_tables($conn);

        if (table_exists($conn, 'owner_wallets')) {
            $ownerInitSql = 'INSERT INTO owner_wallets (owner_id, balance) VALUES (?, 0) ON DUPLICATE KEY
UPDATE owner_id = owner_id';
            $ownerInitStmt = $conn->prepare($ownerInitSql);
            if ($ownerInitStmt) {
                $ownerInitStmt->bind_param('i', $ownerId);
                $ownerInitStmt->execute();
                $ownerInitStmt->close();
            }

            $ownerUpdateSql = 'UPDATE owner_wallets SET balance = balance + ? WHERE owner_id = ?';
            $ownerWalletStmt = $conn->prepare($ownerUpdateSql);
            if ($ownerWalletStmt) {
                $ownerWalletStmt->bind_param('di', $ownerShare, $ownerId);
                $ownerWalletStmt->execute();
```

```
                $ownerWalletStmt->close();
            }
        }

        if (table_exists($conn, 'owner_wallet_transactions')) {
            $ownerTxnType = 'cancellation_share';
            $ownerNote = '20% cancellation share from booking #' . $bookingId . ' (' . $turfName . ')';
            $ownerTxnSql = 'INSERT INTO owner_wallet_transactions (owner_id, txn_type, amount, reference_note)
 VALUES (?, ?, ?, ?)';
            $ownerTxnStmt = $conn->prepare($ownerTxnSql);
            if ($ownerTxnStmt) {
                $ownerTxnStmt->bind_param('isds', $ownerId, $ownerTxnType, $ownerShare, $ownerNote);
                $ownerTxnStmt->execute();
                $ownerTxnStmt->close();
            }
        }
    }

    if (table_exists($conn, 'notifications')) {
        $userTitle = 'Refund approved';
        $userMessage = 'Booking #' . $bookingId . ' refund approved. Tk ' . number_format($userRefund, 2) . '
credited to your wallet.';
        if ($adminNote !== '') {
            $userMessage .= ' Admin note: ' . $adminNote;
        }
        $userNotifSql = 'INSERT INTO notifications (user_id, title, message) VALUES (?, ?, ?)';
        $userNotifStmt = $conn->prepare($userNotifSql);
        if ($userNotifStmt) {
            $userNotifStmt->bind_param('iss', $userId, $userTitle, $userMessage);
            $userNotifStmt->execute();
            $userNotifStmt->close();
        }

        if ($ownerId > 0) {
            $ownerTitle = 'Cancellation share credited';
            $ownerMessage = 'Tk ' . number_format($ownerShare, 2) . ' (20%) credited from booking #' .
$bookingId . '.';
            $ownerNotifSql = 'INSERT INTO notifications (owner_id, title, message) VALUES (?, ?, ?)';
            $ownerNotifStmt = $conn->prepare($ownerNotifSql);
            if ($ownerNotifStmt) {
                $ownerNotifStmt->bind_param('iss', $ownerId, $ownerTitle, $ownerMessage);
                $ownerNotifStmt->execute();
                $ownerNotifStmt->close();
            }
        }
    }

    $conn->commit();
    $inTransaction = false;

    $extra = [
        'refund_id' => $refundId,
        'status' => $finalStatus,
        'booking_id' => $bookingId,
        'refund_amount' => $userRefund,
        'owner_share' => $ownerShare
    ];
    if ($persistAdminId <= 0) {
        $extra['warning'] = 'Admin id is not in admins table, so admin_id stored as NULL.';
    }

    send_json_response(true, 'Refund approved and payout completed.', 200, $extra);
} catch (Throwable $e) {
    if ($inTransaction) {
        $conn->rollback();
    }
    send_json_response(false, 'Refund action failed: ' . $e->getMessage(), 500);
}
?>
```

==========
6. backend/admin_stats.php
==========

```php
<?php
header("Content-Type: application/json");
include "db_connection.php";

$stats = [];

$stats['users'] = $conn->query("SELECT COUNT(*) AS c FROM users")->fetch_assoc()['c'];
$stats['owners'] = $conn->query("SELECT COUNT(*) AS c FROM turf_owners")->fetch_assoc()['c'];
$stats['bookings'] = $conn->query("SELECT COUNT(*) AS c FROM bookings")->fetch_assoc()['c'];
$stats['refunds'] = $conn->query("SELECT COUNT(*) AS c FROM refund_requests WHERE
status='pending'")->fetch_assoc()['c'];

echo json_encode([
  "success" => true,
  "stats" => $stats
]);
```

==========
7. backend/admin_turf_action.php
==========

```php
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'admin_common.php';

function resolve_admin_id_for_verification($conn, $adminId) {
    if ($adminId <= 0 || !table_exists($conn, 'admins')) {
        return 0;
    }

    $stmt = $conn->prepare('SELECT admin_id FROM admins WHERE admin_id = ? LIMIT 1');
    if (!$stmt) {
        return 0;
    }

    $stmt->bind_param('i', $adminId);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();

    return (int)($row['admin_id'] ?? 0);
}

$data = read_request_data();
if (empty($data)) {
    send_json_response(false, 'No data received.', 400);
}

$adminId = intval($data['admin_id'] ?? 0);
$turfId = intval($data['turf_id'] ?? 0);
$action = strtolower(trim((string)($data['action'] ?? '')));
$note = trim((string)($data['note'] ?? ''));

if (!admin_exists($conn, $adminId)) {
    send_json_response(false, 'Admin access denied.', 403);
}
```

```
if ($turfId <= 0) {
    send_json_response(false, 'Invalid turf id.', 400);
}

$allowed = ['approved', 'rejected', 'requested_changes'];
if (!in_array($action, $allowed, true)) {
    send_json_response(false, 'Invalid action.', 400);
}

$statusMap = [
    'approved' => 'active',
    'rejected' => 'rejected',
    'requested_changes' => 'pending'
];
$newStatus = $statusMap[$action] ?? 'pending';

$checkStmt = $conn->prepare('SELECT turf_id, status FROM turfs WHERE turf_id = ? LIMIT 1');
if (!$checkStmt) {
    send_json_response(false, 'Server error while loading turf.', 500);
}
$checkStmt->bind_param('i', $turfId);
$checkStmt->execute();
$checkRes = $checkStmt->get_result();
$turf = $checkRes ? $checkRes->fetch_assoc() : null;
$checkStmt->close();

if (!$turf) {
    send_json_response(false, 'Turf not found.', 404);
}

$logAdminId = resolve_admin_id_for_verification($conn, $adminId);
$verificationLogged = false;

try {
    $conn->begin_transaction();

    $updateStmt = $conn->prepare('UPDATE turfs SET status = ? WHERE turf_id = ? LIMIT 1');
    if (!$updateStmt) {
        throw new RuntimeException('Server error while updating turf.');
    }

    $updateStmt->bind_param('si', $newStatus, $turfId);
    $ok = $updateStmt->execute();
    $updateStmt->close();

    if (!$ok) {
        throw new RuntimeException('Failed to update turf status.');
    }

    if (table_exists($conn, 'turf_verifications') && $logAdminId > 0) {
        $hasNote = column_exists($conn, 'turf_verifications', 'note');
        if ($hasNote) {
            $logSql = 'INSERT INTO turf_verifications (turf_id, admin_id, action, note) VALUES (?, ?, ?, ?)';
            $logStmt = $conn->prepare($logSql);
            if ($logStmt) {
                $logStmt->bind_param('iiss', $turfId, $logAdminId, $action, $note);
                $verificationLogged = $logStmt->execute();
                $logStmt->close();
            }
        } else {
            $logSql = 'INSERT INTO turf_verifications (turf_id, admin_id, action) VALUES (?, ?, ?)';
            $logStmt = $conn->prepare($logSql);
            if ($logStmt) {
                $logStmt->bind_param('iis', $turfId, $logAdminId, $action);
                $verificationLogged = $logStmt->execute();
                $logStmt->close();
            }
        }
    }

    $conn->commit();
```

```
} catch (Throwable $e) {
    if ($conn->errno || $conn->sqlstate) {
        $conn->rollback();
    }
    send_json_response(false, 'Turf action failed: ' . $e->getMessage(), 500);
}

$response = [
    'turf_id' => $turfId,
    'action' => $action,
    'status' => $newStatus,
    'verification_logged' => $verificationLogged
];

if ($logAdminId <= 0) {
    $response['warning'] = 'Verification log skipped because admin_id is not present in admins table.';
}

send_json_response(true, 'Turf verification updated successfully.', 200, $response);
?>
```

==========
8. backend/admin_user_action.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'admin_common.php';

$data = read_request_data();
if (empty($data)) {
    send_json_response(false, 'No data received.', 400);
}

$adminId = intval($data['admin_id'] ?? 0);
$targetType = strtolower(trim((string)($data['target_type'] ?? '')));
$targetId = intval($data['target_id'] ?? 0);
$action = strtolower(trim((string)($data['action'] ?? '')));

if (!admin_exists($conn, $adminId)) {
    send_json_response(false, 'Admin access denied.', 403);
}

if ($targetId <= 0) {
    send_json_response(false, 'Invalid target id.', 400);
}

if ($action !== 'ban' && $action !== 'unban') {
    send_json_response(false, 'Invalid action.', 400);
}

if ($targetType !== 'user' && $targetType !== 'owner') {
    send_json_response(false, 'Invalid target type.', 400);
}

if ($targetType === 'user') {
    $newStatus = $action === 'ban' ? 'banned' : 'active';

    $checkStmt = $conn->prepare('SELECT status FROM users WHERE user_id = ? LIMIT 1');
    if (!$checkStmt) {
      send_json_response(false, 'Server error while loading user.', 500);
```

```
    }
    $checkStmt->bind_param('i', $targetId);
    $checkStmt->execute();
    $checkRes = $checkStmt->get_result();
    $checkRow = $checkRes ? $checkRes->fetch_assoc() : null;
    $checkStmt->close();

    if (!$checkRow) {
      send_json_response(false, 'User not found.', 404);
    }

    $currentStatus = strtolower((string)($checkRow['status'] ?? 'active'));
    if ($currentStatus === $newStatus) {
      send_json_response(true, 'User already in requested status.', 200, [
        'target_type' => 'user',
        'target_id' => $targetId,
        'status' => $newStatus
      ]);
    }

    $conn->begin_transaction();

    $stmt = $conn->prepare('UPDATE users SET status = ? WHERE user_id = ? LIMIT 1');
    if (!$stmt) {
        $conn->rollback();
        send_json_response(false, 'Server error while updating user.', 500);
    }
    $stmt->bind_param('si', $newStatus, $targetId);
    $ok = $stmt->execute();
    $stmt->close();

    if (!$ok) {
        $conn->rollback();
        send_json_response(false, 'User status update failed.', 500);
    }

    if (column_exists($conn, 'users', 'role') && $action === 'unban') {
        $roleStmt = $conn->prepare("UPDATE users SET role = 'user' WHERE user_id = ? AND role = 'banned' LIMIT
 1");
        if ($roleStmt) {
            $roleStmt->bind_param('i', $targetId);
            $roleStmt->execute();
            $roleStmt->close();
        }
    }

    $conn->commit();
    send_json_response(true, 'User status updated successfully.', 200, [
        'target_type' => 'user',
        'target_id' => $targetId,
        'status' => $newStatus
    ]);
}

$newStatus = $action === 'ban' ? 'suspended' : 'verified';

$ownerCheckStmt = $conn->prepare('SELECT status FROM turf_owners WHERE owner_id = ? LIMIT 1');
if (!$ownerCheckStmt) {
    send_json_response(false, 'Server error while loading owner.', 500);
}
$ownerCheckStmt->bind_param('i', $targetId);
$ownerCheckStmt->execute();
$ownerCheckRes = $ownerCheckStmt->get_result();
$ownerCheckRow = $ownerCheckRes ? $ownerCheckRes->fetch_assoc() : null;
$ownerCheckStmt->close();

if (!$ownerCheckRow) {
    send_json_response(false, 'Owner not found.', 404);
}

$currentOwnerStatus = strtolower((string)($ownerCheckRow['status'] ?? 'pending'));
if ($currentOwnerStatus === $newStatus) {
```

```
    send_json_response(true, 'Owner already in requested status.', 200, [
        'target_type' => 'owner',
        'target_id' => $targetId,
        'status' => $newStatus
    ]);
}

$conn->begin_transaction();

$stmt = $conn->prepare('UPDATE turf_owners SET status = ? WHERE owner_id = ? LIMIT 1');
if (!$stmt) {
    $conn->rollback();
    send_json_response(false, 'Server error while updating owner.', 500);
}
$stmt->bind_param('si', $newStatus, $targetId);
$ok = $stmt->execute();
$stmt->close();

if (!$ok) {
    $conn->rollback();
    send_json_response(false, 'Owner status update failed.', 500);
}

if (table_exists($conn, 'turfs') && $action === 'ban') {
    $turfStmt = $conn->prepare("UPDATE turfs SET status = 'inactive' WHERE owner_id = ? AND status =
'active'");
    if ($turfStmt) {
        $turfStmt->bind_param('i', $targetId);
        $turfStmt->execute();
        $turfStmt->close();
    }
}

$conn->commit();

send_json_response(true, 'Owner status updated successfully.', 200, [
    'target_type' => 'owner',
    'target_id' => $targetId,
    'status' => $newStatus
]);
?>
```

==========
9. backend/book_slot.php
==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once "db_connection.php";
require_once "promo_code_helper.php";
date_default_timezone_set("Asia/Dhaka");

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        "success" => $success,
        "message" => $message
    ], $extra));
    exit();
}
```

```
$data = json_decode(file_get_contents("php://input"), true);
if (!is_array($data)) {
    send_response(false, "Invalid request body.", 400);
}

$user_id = intval($data['user_id'] ?? 0);
$slot_id = intval($data['slot_id'] ?? 0);
$price = $data['booked_price'] ?? null;
$promo_code = strtoupper(trim((string)($data['promo_code'] ?? '')));

if ($user_id <= 0 || $slot_id <= 0 || !is_numeric($price)) {
    send_response(false, "Invalid booking payload.", 400);
}

$client_booked_price = floatval($price);

// Validate slot exists
$slotStmt = $conn->prepare("
    SELECT s.slot_id, s.slot_date, s.start_time, s.end_time, s.base_price, t.owner_id, t.turf_name
    FROM slots s
    JOIN turfs t ON t.turf_id = s.turf_id
    WHERE s.slot_id = ? AND s.is_enabled = 1
    LIMIT 1
");
if (!$slotStmt) {
    send_response(false, "Server error while checking slot.", 500);
}
$slotStmt->bind_param("i", $slot_id);
$slotStmt->execute();
$slotResult = $slotStmt->get_result();
$slot = $slotResult ? $slotResult->fetch_assoc() : null;
$slotStmt->close();

if (!$slot) {
    send_response(false, "Slot not found or disabled.", 404);
}

$slotDate = (string)($slot['slot_date'] ?? '');
$slotStartTime = (string)($slot['start_time'] ?? '');
if ($slotDate === '' || $slotStartTime === '') {
    send_response(false, "Invalid slot date/time.", 400);
}

$now = new DateTime('now', new DateTimeZone('Asia/Dhaka'));
$slotStart = DateTime::createFromFormat('Y-m-d H:i:s', $slotDate . ' ' . substr($slotStartTime, 0, 8), new
DateTimeZone('Asia/Dhaka'));
if (!$slotStart) {
    send_response(false, "Invalid slot date/time.", 400);
}

if ($slotStart <= $now) {
    send_response(false, "Past date/time slot cannot be booked.", 400);
}

// Prevent duplicate active bookings for same slot
$checkSql = "SELECT booking_id
             FROM bookings
             WHERE slot_id = ?
               AND booking_status IN ('pending', 'confirmed', 'completed', 'resell_listed', 'resold')
             LIMIT 1";
$checkStmt = $conn->prepare($checkSql);
if (!$checkStmt) {
    send_response(false, "Server error while checking booking.", 500);
}
$checkStmt->bind_param("i", $slot_id);
$checkStmt->execute();
$checkStmt->store_result();
if ($checkStmt->num_rows > 0) {
    $checkStmt->close();
    send_response(false, "Slot already booked.", 200);
}
$checkStmt->close();
```

```
$subtotal = promo_calculate_slot_subtotal(
    floatval($slot['base_price'] ?? 0),
    (string)($slot['start_time'] ?? ''),
    (string)($slot['end_time'] ?? '')
);
if ($subtotal <= 0) {
    send_response(false, "Invalid slot price.", 400);
}

$discountAmount = 0;
$finalPrice = round($subtotal, 2);
if ($promo_code !== '') {
    $promoResult = promo_validate_for_owner_turf(
        $conn,
        intval($slot['owner_id'] ?? 0),
        $promo_code,
        $subtotal,
        $slotDate
    );
    if (!$promoResult['success']) {
        send_response(false, $promoResult['message'], 400);
    }
    $promo_code = (string)($promoResult['promo_code'] ?? $promo_code);
    $discountAmount = floatval($promoResult['discount_amount'] ?? 0);
    $finalPrice = floatval($promoResult['final_total'] ?? $subtotal);
}

if ($finalPrice <= 0) {
    send_response(false, "Invalid final booking amount.", 400);
}

if ($client_booked_price <= 0) {
    send_response(false, "Booked price must be positive.", 400);
}

$insertSql = "INSERT INTO bookings
              (user_id, slot_id, booking_status, booked_price, refund_amount, promo_code, created_at)
              VALUES (?, ?, 'pending', ?, 0, ?, NOW())";
$stmt = $conn->prepare($insertSql);
if (!$stmt) {
    send_response(false, "Server error while creating booking.", 500);
}

$stmt->bind_param("iids", $user_id, $slot_id, $finalPrice, $promo_code);
if (!$stmt->execute()) {
    $error = $stmt->error;
    $stmt->close();
    send_response(false, "Booking failed. " . $error, 500);
}

$bookingId = (int)$stmt->insert_id;
$stmt->close();

send_response(true, "Booking request sent successfully.", 200, [
    "booking_id" => $bookingId,
    "booked_price" => $finalPrice,
    "discount_amount" => $discountAmount,
    "promo_code" => $promo_code
]);
?>
```

==========
10. backend/cancel_booking.php
==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
```

```
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

if (!table_exists($conn, 'refund_requests')) {
    send_response(false, 'Refund request system is not available.', 500);
}

$contentType = $_SERVER['CONTENT_TYPE'] ?? '';
if (stripos($contentType, 'application/json') !== false) {
    $data = json_decode(file_get_contents('php://input'), true);
} else {
    $data = $_POST;
    if (empty($data)) {
        parse_str(file_get_contents('php://input'), $data);
    }
}

if (!is_array($data) || empty($data)) {
    send_response(false, 'No data received.', 400);
}

$userId = intval($data['user_id'] ?? 0);
$bookingId = intval($data['booking_id'] ?? 0);

if ($userId <= 0 || $bookingId <= 0) {
    send_response(false, 'Invalid cancellation request.', 400);
}

$sql = "
SELECT
  b.booking_id,
  b.booked_price,
  b.booking_status,
  s.slot_date,
  s.start_time,
  t.owner_id,
  t.turf_name
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
WHERE b.booking_id = ?
  AND b.user_id = ?
LIMIT 1
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while loading booking.', 500);
}
$stmt->bind_param('ii', $bookingId, $userId);
$stmt->execute();
```

```
$res = $stmt->get_result();
$row = $res ? $res->fetch_assoc() : null;
$stmt->close();

if (!$row) {
    send_response(false, 'Booking not found.', 404);
}

$status = strtolower((string)($row['booking_status'] ?? ''));
if ($status !== 'confirmed') {
    send_response(false, 'Only confirmed bookings can request cancellation.', 400);
}

$slotTs = strtotime((string)$row['slot_date'] . ' ' . (string)$row['start_time']);
$nowTs = time();
if (!$slotTs || $slotTs <= $nowTs) {
    send_response(false, 'Booking time has started or passed.', 400);
}

$bookedPrice = (float)($row['booked_price'] ?? 0);
if ($bookedPrice <= 0) {
    send_response(false, 'Invalid booking price.', 400);
}

$ownerShare = round($bookedPrice * 0.20, 2);
$userRefund = round($bookedPrice - $ownerShare, 2);

$existingStmt = $conn->prepare("SELECT refund_id, status FROM refund_requests WHERE booking_id = ? LIMIT 1");
if (!$existingStmt) {
    send_response(false, 'Server error while checking refund request.', 500);
}
$existingStmt->bind_param('i', $bookingId);
$existingStmt->execute();
$existingRes = $existingStmt->get_result();
$existing = $existingRes ? $existingRes->fetch_assoc() : null;
$existingStmt->close();

if ($existing) {
    $existingStatus = strtolower((string)($existing['status'] ?? 'pending'));
    if ($existingStatus === 'pending') {
        send_response(true, 'Cancellation request already pending for admin approval.', 200, [
            'booking_id' => $bookingId,
            'request_status' => 'pending',
            'refund_amount' => $userRefund,
            'owner_share' => $ownerShare
        ]);
    }

    if ($existingStatus === 'approved' || $existingStatus === 'paid') {
        send_response(false, 'Cancellation already processed for this booking.', 400);
    }
}

$conn->begin_transaction();

if ($existing) {
    $refundId = (int)($existing['refund_id'] ?? 0);
    $updateSql = "UPDATE refund_requests
                  SET requested_by = 'user', requested_amount = ?, status = 'pending', admin_id = NULL,
admin_note = ?, updated_at = NOW()
                  WHERE refund_id = ? LIMIT 1";
    $updateStmt = $conn->prepare($updateSql);
    if (!$updateStmt) {
        $conn->rollback();
        send_response(false, 'Server error while updating cancellation request.', 500);
    }
    $note = 'User cancellation request submitted';
    $updateStmt->bind_param('dsi', $userRefund, $note, $refundId);
    $ok = $updateStmt->execute();
    $affected = $updateStmt->affected_rows;
    $updateStmt->close();
```

```
    if (!$ok || $affected <= 0) {
        $conn->rollback();
        send_response(false, 'Could not submit cancellation request.', 500);
    }

    $requestId = $refundId;
} else {
    $insertSql = "INSERT INTO refund_requests (booking_id, requested_by, requested_amount, status, admin_note)
                  VALUES (?, 'user', ?, 'pending', ?)";
    $insertStmt = $conn->prepare($insertSql);
    if (!$insertStmt) {
        $conn->rollback();
        send_response(false, 'Server error while creating cancellation request.', 500);
    }
    $note = 'User cancellation request submitted';
    $insertStmt->bind_param('ids', $bookingId, $userRefund, $note);
    $ok = $insertStmt->execute();
    $requestId = $ok ? (int)$insertStmt->insert_id : 0;
    $insertStmt->close();

    if (!$ok || $requestId <= 0) {
        $conn->rollback();
        send_response(false, 'Could not create cancellation request.', 500);
    }
}

if (table_exists($conn, 'notifications')) {
    $title = 'Cancellation request submitted';
    $message = 'Booking #' . $bookingId . ' cancellation request is pending admin approval.';
    $notifySql = 'INSERT INTO notifications (user_id, title, message) VALUES (?, ?, ?)';
    $notifyStmt = $conn->prepare($notifySql);
    if ($notifyStmt) {
        $notifyStmt->bind_param('iss', $userId, $title, $message);
        $notifyStmt->execute();
        $notifyStmt->close();
    }
}

$conn->commit();

send_response(true, 'Cancellation request sent to admin.', 200, [
    'booking_id' => $bookingId,
    'refund_id' => $requestId,
    'request_status' => 'pending',
    'refund_amount' => $userRefund,
    'owner_share' => $ownerShare,
    'refund_percent' => 80
]);
?>
```

==========

11. backend/db_connection.php

==========

```
<?php
$host = "localhost";
$username = "root";
$password = "";
$database = "turf_booking_system";

$conn = new mysqli("localhost", "root", "", "turf_booking_system");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

==========

12. backend/get_available_slots.php
==========

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once "db_connection.php";
date_default_timezone_set("Asia/Dhaka");

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        "success" => $success,
        "status" => $message
    ], $extra));
    exit();
}

function seed_default_slots_if_missing($conn, $turfId, $slotDate, $basePrice) {
    $countSql = "SELECT COUNT(*) AS total FROM slots WHERE turf_id = ? AND slot_date = ?";
    $countStmt = $conn->prepare($countSql);
    if (!$countStmt) {
        return false;
    }

    $countStmt->bind_param("is", $turfId, $slotDate);
    $countStmt->execute();
    $countResult = $countStmt->get_result();
    $row = $countResult ? $countResult->fetch_assoc() : null;
    $countStmt->close();

    if (($row && intval($row["total"] ?? 0) > 0) || $basePrice <= 0) {
        return true;
    }

    $insertSql = "INSERT IGNORE INTO slots
        (turf_id, slot_date, start_time, end_time, base_price, is_enabled, created_at)
        VALUES (?, ?, ?, ?, ?, 1, NOW())";

    $insertStmt = $conn->prepare($insertSql);
    if (!$insertStmt) {
        return false;
    }

    for ($hour = 10; $hour <= 23; $hour++) {
        $start = sprintf("%02d:00:00", $hour);
        $end = ($hour === 23) ? "24:00:00" : sprintf("%02d:00:00", $hour + 1);

        $insertStmt->bind_param("isssd", $turfId, $slotDate, $start, $end, $basePrice);
        if (!$insertStmt->execute()) {
            $insertStmt->close();
            return false;
        }
    }

    $insertStmt->close();
    return true;
}

$turfId = intval($_GET['turf_id'] ?? 0);
$slotDate = trim((string)($_GET['slot_date'] ?? ''));

if ($turfId <= 0 || !preg_match('/^\d{4}-\d{2}-\d{2}$/', $slotDate)) {
    send_response(false, "Invalid turf_id or slot_date.", 400);
```

```
}

$todayDate = date("Y-m-d");
$nowTime = date("H:i:s");
if ($slotDate < $todayDate) {
    send_response(true, "Past date is not available for booking.", 200, [
        "slots" => [],
        "count" => 0
    ]);
}

$priceStmt = $conn->prepare("SELECT price_per_hour FROM turfs WHERE turf_id = ? LIMIT 1");
if (!$priceStmt) {
    send_response(false, "Server error while loading turf.", 500);
}
$priceStmt->bind_param("i", $turfId);
$priceStmt->execute();
$priceResult = $priceStmt->get_result();
$turf = $priceResult ? $priceResult->fetch_assoc() : null;
$priceStmt->close();

if (!$turf) {
    send_response(false, "Turf not found.", 404);
}

$basePrice = floatval($turf['price_per_hour'] ?? 0);
if (!seed_default_slots_if_missing($conn, $turfId, $slotDate, $basePrice)) {
    send_response(false, "Server error while creating ready slots.", 500);
}

$sql = "
SELECT
    s.slot_id,
    s.slot_date,
    s.start_time,
    s.end_time,
    s.base_price
FROM slots s
LEFT JOIN bookings b
    ON b.slot_id = s.slot_id
    AND b.booking_status IN ('pending', 'confirmed', 'completed', 'resell_listed', 'resold')
WHERE s.turf_id = ?
  AND s.slot_date = ?
  AND s.is_enabled = 1
  AND b.booking_id IS NULL
ORDER BY s.start_time ASC, s.end_time ASC
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, "Server error while loading slots.", 500);
}

$stmt->bind_param("is", $turfId, $slotDate);
$stmt->execute();
$result = $stmt->get_result();

$slots = [];
while ($row = $result->fetch_assoc()) {
    if ($slotDate === $todayDate && (string)$row["start_time"] <= $nowTime) {
        continue;
    }

    $slots[] = [
        "slot_id" => (int)$row["slot_id"],
        "slot_date" => (string)$row["slot_date"],
        "start_time" => (string)$row["start_time"],
        "end_time" => (string)$row["end_time"],
        "base_price" => (float)$row["base_price"]
    ];
}
```

```
$stmt->close();

send_response(true, "Available slots loaded.", 200, [
    "slots" => $slots,
    "count" => count($slots)
]);
?>
```

==========

13. backend/get_turfs.php

==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once "db_connection.php";

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        "success" => $success,
        "status" => $message
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

function column_exists($conn, $table, $column) {
    $safeTable = preg_replace('/[^a-zA-Z0-9_]/', '', $table);
    if ($safeTable === '') {
        return false;
    }

    $safeColumn = $conn->real_escape_string($column);
    $res = $conn->query("SHOW COLUMNS FROM `$safeTable` LIKE '$safeColumn'");
    return $res && $res->num_rows > 0;
}

$locationSelect = column_exists($conn, 'turfs', 'location')
    ? "t.location"
    : "'' AS location";

$imageSelect = table_exists($conn, 'turf_images')
    ? "(
        SELECT ti.image_url
        FROM turf_images ti
        WHERE ti.turf_id = t.turf_id
        ORDER BY ti.is_primary DESC, ti.image_id DESC
        LIMIT 1
      ) AS image_url"
    : "'' AS image_url";

$sql = "
SELECT
    t.turf_id,
    t.turf_name,
    t.sport_type,
```

```
    t.address,
    t.area,
    t.city,
    $locationSelect,
    t.price_per_hour,
    t.status,
    t.created_at,
    $imageSelect
FROM turfs t
WHERE t.status = 'active'
ORDER BY t.created_at DESC, t.turf_id DESC
";

$result = $conn->query($sql);
if (!$result) {
    send_response(false, "Failed to load turfs.", 500);
}

$turfs = [];
while ($row = $result->fetch_assoc()) {
    $createdAt = (string)($row['created_at'] ?? '');
    $createdTs = $createdAt !== '' ? strtotime($createdAt) : 0;
    $isNew = $createdTs > 0 ? ((time() - $createdTs) <= (7 * 24 * 3600)) : false;

    $turfs[] = [
        "turf_id" => (int)$row["turf_id"],
        "turf_name" => $row["turf_name"],
        "sport_type" => $row["sport_type"],
        "address" => $row["address"],
        "area" => $row["area"],
        "city" => $row["city"],
        "location" => $row["location"] ?? "",
        "price_per_hour" => (float)$row["price_per_hour"],
        "status" => $row["status"],
        "created_at" => $createdAt,
        "is_new" => $isNew,
        "image_url" => $row["image_url"] ?? ""
    ];
}

send_response(true, "Turfs loaded.", 200, [
    "turfs" => $turfs
]);
?>
```

==========

14. backend/get_user_bookings.php

==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}
```

```
function column_exists($conn, $table, $column) {
    $safeTable = preg_replace('/[^a-zA-Z0-9_]/', '', $table);
    if ($safeTable === '') {
        return false;
    }

    $safeColumn = $conn->real_escape_string($column);
    $res = $conn->query("SHOW COLUMNS FROM '$safeTable' LIKE '$safeColumn'");
    return $res && $res->num_rows > 0;
}

$userId = intval($_GET['user_id'] ?? 0);
if ($userId <= 0) {
    send_response(false, 'Invalid user id.', 400);
}

$locationSelect = column_exists($conn, 'turfs', 'location') ? 't.location,' : "'' AS location,";

$sql = "
SELECT
  b.booking_id,
  b.booking_status,
  b.booked_price,
  b.refund_amount,
  b.created_at,
  s.slot_id,
  s.slot_date,
  s.start_time,
  s.end_time,
  t.turf_id,
  t.turf_name,
  t.city,
  t.area,
  $locationSelect
  t.address,
  t.cancel_before_hours,
  t.refund_percent
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
WHERE b.user_id = ?
ORDER BY b.created_at DESC, b.booking_id DESC
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while loading bookings.', 500);
}
$stmt->bind_param('i', $userId);
$stmt->execute();
$res = $stmt->get_result();

$rows = [];
$nowTs = time();

while ($row = $res ? $res->fetch_assoc() : null) {
    if (!$row) break;

    $slotDate = (string)$row['slot_date'];
    $startTime = (string)$row['start_time'];
    $slotTs = strtotime($slotDate . ' ' . $startTime);
    $hoursLeft = $slotTs ? (($slotTs - $nowTs) / 3600) : -1;

    $canCancel = strtolower((string)$row['booking_status']) === 'confirmed' && $hoursLeft > 0;

    $rows[] = [
        'booking_id' => (int)$row['booking_id'],
        'booking_status' => $row['booking_status'],
        'booked_price' => (float)$row['booked_price'],
        'refund_amount' => (float)$row['refund_amount'],
        'created_at' => $row['created_at'],
```

```
        'slot_id' => (int)$row['slot_id'],
        'slot_date' => $row['slot_date'],
        'start_time' => $row['start_time'],
        'end_time' => $row['end_time'],
        'turf_id' => (int)$row['turf_id'],
        'turf_name' => $row['turf_name'],
        'location' => ($row['location'] ?? '') ?: ($row['area'] ?: ($row['city'] ?: $row['address'])),
        'cancel_before_hours' => (int)($row['cancel_before_hours'] ?? 24),
        'refund_percent' => (int)($row['refund_percent'] ?? 80),
        'can_cancel' => $canCancel
    ];
}

$stmt->close();

send_response(true, 'Bookings loaded.', 200, [
    'bookings' => $rows
]);
?>
```

==========
15. backend/get_user_transactions.php
==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

$userId = intval($_GET['user_id'] ?? 0);
if ($userId <= 0) {
    send_response(false, 'Invalid user id.', 400);
}

$rows = [];

$bookingSql = "
SELECT
  b.booking_id,
  b.booking_status,
  b.booked_price,
  b.refund_amount,
  b.created_at,
  s.slot_date,
  s.start_time,
  s.end_time,
  t.turf_name
```

```
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
WHERE b.user_id = ?
ORDER BY b.created_at DESC, b.booking_id DESC
";

$bookingStmt = $conn->prepare($bookingSql);
if (!$bookingStmt) {
    send_response(false, 'Server error while loading transactions.', 500);
}

$bookingStmt->bind_param('i', $userId);
$bookingStmt->execute();
$bookingRes = $bookingStmt->get_result();

while ($row = $bookingRes ? $bookingRes->fetch_assoc() : null) {
    if (!$row) break;

    $bookingId = (int)$row['booking_id'];
    $bookedPrice = (float)($row['booked_price'] ?? 0);
    $refundAmount = (float)($row['refund_amount'] ?? 0);
    $bookingStatus = strtolower((string)($row['booking_status'] ?? 'pending'));
    $slotSummary = trim(($row['slot_date'] ?? '') . ' ' . ($row['start_time'] ?? '') . ' - ' .
($row['end_time'] ?? ''));

    $rows[] = [
        'id' => 'payment-' . $bookingId,
        'type' => 'payment',
        'tone' => 'info',
        'reference' => 'Booking #' . $bookingId,
        'title' => 'Turf booking payment',
        'description' => trim(($row['turf_name'] ?? 'Turf booking') . ($slotSummary !== '' ? ' ? ' .
$slotSummary : '')),
        'method' => 'Booking payment',
        'status' => 'Paid',
        'amount' => -abs($bookedPrice),
        'time' => $row['created_at'],
        'sort_time' => $row['created_at']
    ];

    if ($bookingStatus === 'cancelled' && $refundAmount > 0) {
        $rows[] = [
            'id' => 'refund-' . $bookingId,
            'type' => 'refund',
            'tone' => 'success',
            'reference' => 'Booking #' . $bookingId,
            'title' => 'Cancellation refund',
            'description' => 'Refund approved for ' . ($row['turf_name'] ?? 'cancelled booking'),
            'method' => 'Wallet refund',
            'status' => 'Approved',
            'amount' => abs($refundAmount),
            'time' => $row['created_at'],
            'sort_time' => $row['created_at']
        ];
    }
}

$bookingStmt->close();

if (table_exists($conn, 'wallet_transactions')) {
    $walletSql = "
    SELECT wallet_txn_id, txn_type, amount, reference_note, created_at
    FROM wallet_transactions
    WHERE user_id = ?
    ORDER BY wallet_txn_id DESC
    ";

    $walletStmt = $conn->prepare($walletSql);
    if ($walletStmt) {
        $walletStmt->bind_param('i', $userId);
        $walletStmt->execute();
```

```
        $walletRes = $walletStmt->get_result();

        while ($txn = $walletRes ? $walletRes->fetch_assoc() : null) {
            if (!$txn) break;

            $txnType = strtolower((string)($txn['txn_type'] ?? 'adjustment'));
            $amount = (float)($txn['amount'] ?? 0);
            $rows[] = [
                'id' => 'wallet-' . (int)$txn['wallet_txn_id'],
                'type' => $txnType === 'refund' ? 'refund' : $txnType,
                'tone' => $txnType === 'refund' ? 'success' : 'warning',
                'reference' => 'Wallet Txn #' . (int)$txn['wallet_txn_id'],
                'title' => ucfirst(str_replace('_', ' ', $txnType)),
                'description' => (string)($txn['reference_note'] ?? 'Wallet transaction'),
                'method' => 'Wallet ledger',
                'status' => 'Recorded',
                'amount' => $amount,
                'time' => $txn['created_at'],
                'sort_time' => $txn['created_at']
            ];
        }

        $walletStmt->close();
    }
}

usort($rows, function ($a, $b) {
    return strtotime((string)($b['sort_time'] ?? '')) <=> strtotime((string)($a['sort_time'] ?? ''));
});

send_response(true, 'Transactions loaded.', 200, [
    'transactions' => $rows
]);
?>



==========================================================================================
16. backend/login.php
==========================================================================================

<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $status, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'status' => $status
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

function column_exists($conn, $table, $column) {
    $safeTable = preg_replace('/[^a-zA-Z0-9_]/', '', $table);
    if ($safeTable === '') return false;
```

```
    $col = $conn->real_escape_string($column);
    $res = $conn->query("SHOW COLUMNS FROM ‘$safeTable‘ LIKE ’$col’");
    return $res && $res->num_rows > 0;
}

function get_owner_id($conn, $userId, $email) {
    if (!table_exists($conn, ’turf_owners’)) return 0;

    if (column_exists($conn, ’turf_owners’, ’user_id’) && $userId > 0) {
        $stmt = $conn->prepare(’SELECT owner_id FROM turf_owners WHERE user_id = ? LIMIT 1’);
        if ($stmt) {
            $stmt->bind_param(’i’, $userId);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row && isset($row[’owner_id’])) {
                return (int)$row[’owner_id’];
            }
        }
    }

    if ($email !== ’’ && column_exists($conn, ’turf_owners’, ’email’)) {
        $stmt = $conn->prepare(’SELECT owner_id FROM turf_owners WHERE LOWER(email) = LOWER(?) LIMIT 1’);
        if ($stmt) {
            $stmt->bind_param(’s’, $email);
            $stmt->execute();
            $res = $stmt->get_result();
            $row = $res ? $res->fetch_assoc() : null;
            $stmt->close();
            if ($row && isset($row[’owner_id’])) {
                return (int)$row[’owner_id’];
            }
        }
    }

    return 0;
}

function verify_password_with_upgrade($conn, $table, $pkColumn, $pkValue, $storedHash, $plainPassword) {
    if ($storedHash === ’’) {
        return false;
    }

    $passwordOk = password_verify($plainPassword, $storedHash);
    if ($passwordOk) {
        return true;
    }

    if (!hash_equals($storedHash, $plainPassword)) {
        return false;
    }

    $newHash = password_hash($plainPassword, PASSWORD_DEFAULT);
    $sql = "UPDATE ‘$table‘ SET password_hash = ? WHERE ‘$pkColumn‘ = ? LIMIT 1";
    $stmt = $conn->prepare($sql);
    if ($stmt) {
        if (is_int($pkValue)) {
            $stmt->bind_param(’si’, $newHash, $pkValue);
        } else {
            $stmt->bind_param(’ss’, $newHash, $pkValue);
        }
        $stmt->execute();
        $stmt->close();
    }

    return true;
}

$contentType = $_SERVER[’CONTENT_TYPE’] ?? ’’;
if (stripos($contentType, ’application/json’) !== false) {
    $data = json_decode(file_get_contents(’php://input’), true);
```

```
} else {
    $data = $_POST;
    if (empty($data)) {
        parse_str(file_get_contents('php://input'), $data);
    }
}

if (!is_array($data) || empty($data)) {
    send_response(false, 'No data received.', 400);
}

$email = strtolower(trim((string)($data['email'] ?? '')));
$password = (string)($data['password'] ?? '');

if ($email === '' || $password === '') {
    send_response(false, 'Email and password are required.', 400);
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    send_response(false, 'Invalid email address.', 400);
}

$userFound = false;
$userPasswordMatched = false;
$userStatus = 'active';

if (table_exists($conn, 'users')) {
    $hasRoleColumn = column_exists($conn, 'users', 'role');
    $selectRole = $hasRoleColumn ? ', role' : '';

    $sql = "SELECT user_id, full_name, email, password_hash, status$selectRole
            FROM users
            WHERE LOWER(email) = LOWER(?)
            LIMIT 1";

    $stmt = $conn->prepare($sql);
    if (!$stmt) {
        send_response(false, 'Server error while preparing login.', 500);
    }

    $stmt->bind_param('s', $email);
    $stmt->execute();
    $result = $stmt->get_result();
    $user = $result ? $result->fetch_assoc() : null;
    $stmt->close();

    if ($user) {
        $userFound = true;
        $userStatus = strtolower(trim((string)($user['status'] ?? 'active')));

        if ($userStatus === 'active') {
            $storedHash = (string)($user['password_hash'] ?? '');
            $userPasswordMatched = verify_password_with_upgrade($conn, 'users', 'user_id',
(int)$user['user_id'], $storedHash, $password);
        }

        if ($userStatus === 'active' && $userPasswordMatched) {
            $roleValue = $hasRoleColumn ? strtolower(trim((string)($user['role'] ?? 'user'))) : 'user';
            $role = in_array($roleValue, ['user', 'owner', 'admin'], true) ? $roleValue : 'user';

            $ownerId = get_owner_id($conn, (int)$user['user_id'], (string)$user['email']);
            if ($role !== 'admin' && $ownerId > 0) {
                $role = 'owner';
            }

            $adminId = 0;
            if ($role === 'admin' && table_exists($conn, 'admins')) {
                $adminStmt = $conn->prepare('SELECT admin_id FROM admins WHERE LOWER(email) = LOWER(?) LIMIT
1');
                if ($adminStmt) {
                    $adminStmt->bind_param('s', $email);
                    $adminStmt->execute();
```

```
                    $adminRes = $adminStmt->get_result();
                    $adminRow = $adminRes ? $adminRes->fetch_assoc() : null;
                    $adminStmt->close();
                    $adminId = (int)($adminRow['admin_id'] ?? 0);
                }
            }

            // Compatibility: if admin role lives in users table only, keep admin access using that user id.
            if ($role === 'admin' && $adminId <= 0) {
                $adminId = (int)$user['user_id'];
            }

            send_response(true, 'Login successful.', 200, [
                'user' => [
                    'user_id' => (int)$user['user_id'],
                    'full_name' => $user['full_name'],
                    'email' => $user['email'],
                    'status' => $userStatus,
                    'role' => $role,
                    'owner_id' => $ownerId,
                    'admin_id' => $adminId
                ]
            ]);
        }
    }
}

if (table_exists($conn, 'admins')) {
    $adminSql = 'SELECT admin_id, full_name, email, password_hash FROM admins WHERE LOWER(email) = LOWER(?)
LIMIT 1';
    $adminStmt = $conn->prepare($adminSql);
    if (!$adminStmt) {
        send_response(false, 'Server error while preparing admin login.', 500);
    }

    $adminStmt->bind_param('s', $email);
    $adminStmt->execute();
    $adminRes = $adminStmt->get_result();
    $admin = $adminRes ? $adminRes->fetch_assoc() : null;
    $adminStmt->close();

    if ($admin) {
        $storedHash = (string)($admin['password_hash'] ?? '');
        if (!verify_password_with_upgrade($conn, 'admins', 'admin_id', (int)$admin['admin_id'], $storedHash,
$password)) {
            send_response(false, 'Invalid password.', 200);
        }

        send_response(true, 'Admin login successful.', 200, [
            'user' => [
                'user_id' => (int)$admin['admin_id'],
                'admin_id' => (int)$admin['admin_id'],
                'full_name' => $admin['full_name'],
                'email' => $admin['email'],
                'status' => 'active',
                'role' => 'admin',
                'owner_id' => 0
            ]
        ]);
    }
}

if ($userFound) {
    if ($userStatus !== 'active') {
        send_response(false, 'Account is not active.', 200);
    }

    if (!$userPasswordMatched) {
        send_response(false, 'Invalid password.', 200);
    }
}
```

```
send_response(false, 'User not found.', 200);
?>
```

==========
17. backend/owner_booking_clients.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

$ownerId = intval($_GET['owner_id'] ?? 0);
if ($ownerId <= 0) {
    send_response(false, 'Invalid owner id.', 400);
}

$ownerStmt = $conn->prepare('SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1');
if (!$ownerStmt) {
    send_response(false, 'Server error while checking owner.', 500);
}
$ownerStmt->bind_param('i', $ownerId);
$ownerStmt->execute();
$ownerRes = $ownerStmt->get_result();
$ownerRow = $ownerRes ? $ownerRes->fetch_assoc() : null;
$ownerStmt->close();

if (!$ownerRow) {
    send_response(false, 'Owner not found.', 404);
}

$sql = "
SELECT
  b.booking_id,
  b.booking_status,
  b.created_at,
  b.booked_price,
  s.slot_date,
  s.start_time,
  s.end_time,
  t.turf_id,
  t.turf_name,
  COALESCE(t.sport_type, '') AS sport_type,
  u.user_id,
  u.full_name AS customer_name,
  u.email AS customer_email,
  COALESCE(u.phone, '') AS customer_phone
FROM bookings b
JOIN slots s ON s.slot_id = b.slot_id
JOIN turfs t ON t.turf_id = s.turf_id
```

```
JOIN users u ON u.user_id = b.user_id
WHERE t.owner_id = ?
ORDER BY s.slot_date DESC, s.start_time DESC, b.booking_id DESC
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while loading booking clients.', 500);
}
$stmt->bind_param('i', $ownerId);
$stmt->execute();
$res = $stmt->get_result();

$rows = [];
while ($row = $res ? $res->fetch_assoc() : null) {
    if (!$row) {
        break;
    }

    $rows[] = [
        'booking_id' => (int)$row['booking_id'],
        'booking_status' => $row['booking_status'],
        'created_at' => $row['created_at'],
        'booked_price' => (float)($row['booked_price'] ?? 0),
        'slot_date' => $row['slot_date'],
        'start_time' => $row['start_time'],
        'end_time' => $row['end_time'],
        'turf_id' => (int)$row['turf_id'],
        'turf_name' => $row['turf_name'],
        'sport_type' => $row['sport_type'],
        'client' => [
            'user_id' => (int)$row['user_id'],
            'full_name' => $row['customer_name'],
            'email' => $row['customer_email'],
            'phone' => $row['customer_phone']
        ]
    ];
}

$stmt->close();

send_response(true, 'Booking client details loaded.', 200, [
    'bookings' => $rows
]);
?>
```

==========
18. backend/owner_finance.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
```

```
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

$ownerId = intval($_GET['owner_id'] ?? 0);
if ($ownerId <= 0) {
    send_response(false, 'Invalid owner id.', 400);
}

$ownerStmt = $conn->prepare('SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1');
if (!$ownerStmt) {
    send_response(false, 'Server error while checking owner.', 500);
}
$ownerStmt->bind_param('i', $ownerId);
$ownerStmt->execute();
$ownerRes = $ownerStmt->get_result();
$ownerRow = $ownerRes ? $ownerRes->fetch_assoc() : null;
$ownerStmt->close();

if (!$ownerRow) {
    send_response(false, 'Owner not found.', 404);
}

$summary = [
    'wallet_balance' => 0,
    'total_cancellation_earnings' => 0,
    'pending_refund_requests' => 0,
    'cancelled_bookings' => 0,
    'active_turfs' => 0
];

$transactions = [];

if (table_exists($conn, 'owner_wallets')) {
    $walletStmt = $conn->prepare('SELECT COALESCE(balance,0) AS balance FROM owner_wallets WHERE owner_id = ?
LIMIT 1');
    if ($walletStmt) {
        $walletStmt->bind_param('i', $ownerId);
        $walletStmt->execute();
        $walletRes = $walletStmt->get_result();
        $walletRow = $walletRes ? $walletRes->fetch_assoc() : null;
        $walletStmt->close();
        if ($walletRow) {
            $summary['wallet_balance'] = (float)($walletRow['balance'] ?? 0);
        }
    }
}

if (table_exists($conn, 'owner_wallet_transactions')) {
    $sumStmt = $conn->prepare("SELECT COALESCE(SUM(CASE WHEN txn_type = 'cancellation_share' THEN amount ELSE
0 END),0) AS total_amount FROM owner_wallet_transactions WHERE owner_id = ?");
    if ($sumStmt) {
        $sumStmt->bind_param('i', $ownerId);
        $sumStmt->execute();
        $sumRes = $sumStmt->get_result();
        $sumRow = $sumRes ? $sumRes->fetch_assoc() : null;
        $sumStmt->close();
        if ($sumRow) {
            $summary['total_cancellation_earnings'] = (float)($sumRow['total_amount'] ?? 0);
        }
    }

    $txnStmt = $conn->prepare('SELECT owner_wallet_txn_id, txn_type, amount, reference_note, created_at FROM
owner_wallet_transactions WHERE owner_id = ? ORDER BY owner_wallet_txn_id DESC LIMIT 8');
    if ($txnStmt) {
        $txnStmt->bind_param('i', $ownerId);
        $txnStmt->execute();
        $txnRes = $txnStmt->get_result();
```

```
        while ($row = $txnRes ? $txnRes->fetch_assoc() : null) {
            if (!$row) break;
            $transactions[] = [
                'txn_id' => (int)$row['owner_wallet_txn_id'],
                'txn_type' => $row['txn_type'],
                'amount' => (float)($row['amount'] ?? 0),
                'reference_note' => $row['reference_note'],
                'created_at' => $row['created_at']
            ];
        }
        $txnStmt->close();
    }
}

if (table_exists($conn, 'turfs')) {
    $activeStmt = $conn->prepare("SELECT COUNT(*) AS c FROM turfs WHERE owner_id = ? AND status = 'active'");
    if ($activeStmt) {
        $activeStmt->bind_param('i', $ownerId);
        $activeStmt->execute();
        $activeRes = $activeStmt->get_result();
        $activeRow = $activeRes ? $activeRes->fetch_assoc() : null;
        $activeStmt->close();
        $summary['active_turfs'] = (int)($activeRow['c'] ?? 0);
    }
}

if (table_exists($conn, 'refund_requests') && table_exists($conn, 'bookings') && table_exists($conn, 'slots')
&& table_exists($conn, 'turfs')) {
    $pendingSql = "SELECT COUNT(*) AS c
                   FROM refund_requests r
                   JOIN bookings b ON b.booking_id = r.booking_id
                   JOIN slots s ON s.slot_id = b.slot_id
                   JOIN turfs t ON t.turf_id = s.turf_id
                   WHERE t.owner_id = ? AND r.status = 'pending'";
    $pendingStmt = $conn->prepare($pendingSql);
    if ($pendingStmt) {
        $pendingStmt->bind_param('i', $ownerId);
        $pendingStmt->execute();
        $pendingRes = $pendingStmt->get_result();
        $pendingRow = $pendingRes ? $pendingRes->fetch_assoc() : null;
        $pendingStmt->close();
        $summary['pending_refund_requests'] = (int)($pendingRow['c'] ?? 0);
    }

    $cancelSql = "SELECT COUNT(*) AS c
                  FROM bookings b
                  JOIN slots s ON s.slot_id = b.slot_id
                  JOIN turfs t ON t.turf_id = s.turf_id
                  WHERE t.owner_id = ? AND b.booking_status = 'cancelled'";
    $cancelStmt = $conn->prepare($cancelSql);
    if ($cancelStmt) {
        $cancelStmt->bind_param('i', $ownerId);
        $cancelStmt->execute();
        $cancelRes = $cancelStmt->get_result();
        $cancelRow = $cancelRes ? $cancelRes->fetch_assoc() : null;
        $cancelStmt->close();
        $summary['cancelled_bookings'] = (int)($cancelRow['c'] ?? 0);
    }
}

send_response(true, 'Owner finance loaded.', 200, [
    'summary' => $summary,
    'transactions' => $transactions
]);
?>
```

==========
19. backend/owner_my_turfs.php
==========

```php
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function parse_request_body() {
    $payload = $_POST;
    if (!count($payload)) {
        $raw = file_get_contents('php://input');
        $decoded = json_decode($raw, true);
        if (is_array($decoded)) {
            $payload = $decoded;
        }
    }
    return $payload;
}

function ensure_owner_exists($conn, $ownerId) {
    $stmt = $conn->prepare('SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1');
    if (!$stmt) {
        return false;
    }
    $stmt->bind_param('i', $ownerId);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();
    return !!$row;
}

function load_owner_turfs($conn, $ownerId) {
    $rows = [];
    $stmt = $conn->prepare('
        SELECT turf_id, turf_name, sport_type, address, area, city, price_per_hour, status,
cancel_before_hours, refund_percent, created_at
        FROM turfs
        WHERE owner_id = ?
        ORDER BY turf_id DESC
    ');
    if (!$stmt) {
        return $rows;
    }
    $stmt->bind_param('i', $ownerId);
    $stmt->execute();
    $res = $stmt->get_result();
    while ($row = $res ? $res->fetch_assoc() : null) {
        if (!$row) break;
        $rows[] = [
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name'],
            'sport_type' => $row['sport_type'],
            'address' => $row['address'],
            'area' => $row['area'],
            'city' => $row['city'],
            'price_per_hour' => (float)($row['price_per_hour'] ?? 0),
```

```
            'status' => $row['status'],
            'cancel_before_hours' => (int)($row['cancel_before_hours'] ?? 0),
            'refund_percent' => (int)($row['refund_percent'] ?? 0),
            'created_at' => $row['created_at']
        ];
    }
    $stmt->close();
    return $rows;
}

$request = parse_request_body();
$ownerId = intval($_GET['owner_id'] ?? ($request['owner_id'] ?? 0));
if ($ownerId <= 0) {
    send_response(false, 'Invalid owner id.', 400);
}

if (!ensure_owner_exists($conn, $ownerId)) {
    send_response(false, 'Owner not found.', 404);
}

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $turfs = load_owner_turfs($conn, $ownerId);
    $summary = [
        'total' => count($turfs),
        'active' => count(array_filter($turfs, function ($item) { return strtolower((string)$item['status'])
=== 'active'; })),
        'inactive' => count(array_filter($turfs, function ($item) { return strtolower((string)$item['status'])
 === 'inactive'; })),
        'pending' => count(array_filter($turfs, function ($item) { return strtolower((string)$item['status'])
=== 'pending'; }))
    ];

    send_response(true, 'Owner turfs loaded.', 200, [
        'turfs' => $turfs,
        'summary' => $summary
    ]);
}

$turfId = intval($request['turf_id'] ?? 0);
$status = strtolower(trim((string)($request['status'] ?? '')));

if ($turfId <= 0 || !in_array($status, ['active', 'inactive'], true)) {
    send_response(false, 'Invalid turf status update request.', 400);
}

$checkStmt = $conn->prepare('SELECT status FROM turfs WHERE turf_id = ? AND owner_id = ? LIMIT 1');
if (!$checkStmt) {
    send_response(false, 'Server error while checking turf.', 500);
}
$checkStmt->bind_param('ii', $turfId, $ownerId);
$checkStmt->execute();
$res = $checkStmt->get_result();
$turf = $res ? $res->fetch_assoc() : null;
$checkStmt->close();

if (!$turf) {
    send_response(false, 'Turf not found for this owner.', 404);
}

$currentStatus = strtolower((string)($turf['status'] ?? ''));
if ($currentStatus === 'pending' || $currentStatus === 'rejected') {
    send_response(false, 'This turf cannot be switched until it is approved.', 400);
}

$stmt = $conn->prepare('UPDATE turfs SET status = ? WHERE turf_id = ? AND owner_id = ? LIMIT 1');
if (!$stmt) {
    send_response(false, 'Server error while updating turf status.', 500);
}
$stmt->bind_param('sii', $status, $turfId, $ownerId);
$stmt->execute();
$stmt->close();
```

```
send_response(true, $status === 'active' ? 'Turf turned on successfully.' : 'Turf turned off successfully.');
?>
```

==========
20. backend/owner_promo_codes.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function parse_request_body() {
    $payload = $_POST;
    if (!count($payload)) {
        $raw = file_get_contents('php://input');
        $decoded = json_decode($raw, true);
        if (is_array($decoded)) {
            $payload = $decoded;
        }
    }
    return $payload;
}

function ensure_owner_exists($conn, $ownerId) {
    $stmt = $conn->prepare('SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1');
    if (!$stmt) {
        return false;
    }
    $stmt->bind_param('i', $ownerId);
    $stmt->execute();
    $res = $stmt->get_result();
    $row = $res ? $res->fetch_assoc() : null;
    $stmt->close();
    return !!$row;
}

function load_owner_promos($conn, $ownerId) {
    $rows = [];
    $stmt = $conn->prepare('
        SELECT promo_id, code, discount_type, discount_value, min_booking_amount, start_date, end_date,
is_active, created_at
        FROM promo_codes
        WHERE owner_id = ?
        ORDER BY promo_id DESC
    ');
    if (!$stmt) {
        return $rows;
    }
    $stmt->bind_param('i', $ownerId);
    $stmt->execute();
    $res = $stmt->get_result();
    while ($row = $res ? $res->fetch_assoc() : null) {
```

```
        if (!$row) break;
        $rows[] = [
            'promo_id' => (int)$row['promo_id'],
            'code' => $row['code'],
            'discount_type' => $row['discount_type'],
            'discount_value' => (float)($row['discount_value'] ?? 0),
            'min_booking_amount' => (float)($row['min_booking_amount'] ?? 0),
            'start_date' => $row['start_date'],
            'end_date' => $row['end_date'],
            'is_active' => intval($row['is_active'] ?? 0) === 1,
            'created_at' => $row['created_at']
        ];
    }
    $stmt->close();
    return $rows;
}

$request = parse_request_body();
$ownerId = intval($_GET['owner_id'] ?? ($request['owner_id'] ?? 0));
if ($ownerId <= 0) {
    send_response(false, 'Invalid owner id.', 400);
}

if (!ensure_owner_exists($conn, $ownerId)) {
    send_response(false, 'Owner not found.', 404);
}

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $promos = load_owner_promos($conn, $ownerId);
    $summary = [
        'total_codes' => count($promos),
        'active_codes' => count(array_filter($promos, function ($item) { return $item['is_active']; })),
        'inactive_codes' => count(array_filter($promos, function ($item) { return !$item['is_active']; }))
    ];
    send_response(true, 'Promo codes loaded.', 200, [
        'promos' => $promos,
        'summary' => $summary
    ]);
}

$action = strtolower(trim((string)($request['action'] ?? 'create')));

if ($action === 'toggle') {
    $promoId = intval($request['promo_id'] ?? 0);
    $isActive = intval($request['is_active'] ?? -1);
    if ($promoId <= 0 || ($isActive !== 0 && $isActive !== 1)) {
        send_response(false, 'Invalid promo toggle request.', 400);
    }

    $stmt = $conn->prepare('UPDATE promo_codes SET is_active = ? WHERE promo_id = ? AND owner_id = ? LIMIT
1');
    if (!$stmt) {
        send_response(false, 'Server error while updating promo status.', 500);
    }
    $stmt->bind_param('iii', $isActive, $promoId, $ownerId);
    $stmt->execute();
    $stmt->close();

    send_response(true, $isActive === 1 ? 'Promo code activated.' : 'Promo code deactivated.');
}

$code = strtoupper(trim((string)($request['code'] ?? '')));
$discountType = strtolower(trim((string)($request['discount_type'] ?? 'percent')));
$discountValue = floatval($request['discount_value'] ?? 0);
$minBookingAmount = floatval($request['min_booking_amount'] ?? 0);
$startDate = trim((string)($request['start_date'] ?? ''));
$endDate = trim((string)($request['end_date'] ?? ''));

if ($code === '' || !preg_match('/^[A-Z0-9_-]{3,20}$/', $code)) {
    send_response(false, 'Promo code must be 3-20 characters using letters, numbers, dash, or underscore.',
400);
}
```

```
if (!in_array($discountType, ['percent', 'fixed'], true)) {
    send_response(false, 'Invalid discount type.', 400);
}
if ($discountValue <= 0) {
    send_response(false, 'Discount value must be greater than zero.', 400);
}
if ($discountType === 'percent' && $discountValue > 100) {
    send_response(false, 'Percent discount cannot be greater than 100.', 400);
}
if ($startDate !== '' && !preg_match('/^\d{4}-\d{2}-\d{2}$/', $startDate)) {
    send_response(false, 'Invalid start date.', 400);
}
if ($endDate !== '' && !preg_match('/^\d{4}-\d{2}-\d{2}$/', $endDate)) {
    send_response(false, 'Invalid end date.', 400);
}
if ($startDate !== '' && $endDate !== '' && $endDate < $startDate) {
    send_response(false, 'End date cannot be earlier than start date.', 400);
}

$stmt = $conn->prepare('
    INSERT INTO promo_codes
    (owner_id, code, discount_type, discount_value, min_booking_amount, start_date, end_date, is_active,
created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, 1, NOW())
');
if (!$stmt) {
    send_response(false, 'Server error while creating promo code.', 500);
}
$stmt->bind_param('issddss', $ownerId, $code, $discountType, $discountValue, $minBookingAmount, $startDate,
$endDate);

if (!$stmt->execute()) {
    $error = strtolower((string)$stmt->error);
    $stmt->close();
    if (strpos($error, 'duplicate') !== false || strpos($error, 'unique') !== false) {
        send_response(false, 'This promo code already exists.', 409);
    }
    send_response(false, 'Failed to create promo code.', 500);
}
$promoId = (int)$stmt->insert_id;
$stmt->close();

send_response(true, 'Promo code created successfully.', 200, [
    'promo_id' => $promoId
]);
?>
```

==========

21. backend/owner_slot_control.php

==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';
date_default_timezone_set('Asia/Dhaka');

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
```

```
    ], $extra));
    exit();
}

function parse_json_body() {
    $raw = file_get_contents('php://input');
    if (!$raw) {
        return [];
    }
    $decoded = json_decode($raw, true);
    return is_array($decoded) ? $decoded : [];
}

function seed_default_slots_if_missing($conn, $turfId, $slotDate, $basePrice) {
    $countStmt = $conn->prepare('SELECT COUNT(*) AS total FROM slots WHERE turf_id = ? AND slot_date = ?');
    if (!$countStmt) {
        return false;
    }
    $countStmt->bind_param('is', $turfId, $slotDate);
    $countStmt->execute();
    $countRes = $countStmt->get_result();
    $countRow = $countRes ? $countRes->fetch_assoc() : null;
    $countStmt->close();

    if (($countRow && intval($countRow['total'] ?? 0) > 0) || $basePrice <= 0) {
        return true;
    }

    $insertStmt = $conn->prepare('
        INSERT IGNORE INTO slots
        (turf_id, slot_date, start_time, end_time, base_price, is_enabled, created_at)
        VALUES (?, ?, ?, ?, ?, 1, NOW())
    ');
    if (!$insertStmt) {
        return false;
    }

    for ($hour = 10; $hour <= 23; $hour++) {
        $start = sprintf('%02d:00:00', $hour);
        $end = ($hour === 23) ? '24:00:00' : sprintf('%02d:00:00', $hour + 1);
        $insertStmt->bind_param('isssd', $turfId, $slotDate, $start, $end, $basePrice);
        if (!$insertStmt->execute()) {
            $insertStmt->close();
            return false;
        }
    }

    $insertStmt->close();
    return true;
}

function get_owner_turfs($conn, $ownerId) {
    $turfs = [];
    $stmt = $conn->prepare("
        SELECT turf_id, turf_name, price_per_hour, status
        FROM turfs
        WHERE owner_id = ?
        ORDER BY turf_name ASC
    ");
    if (!$stmt) {
        return $turfs;
    }
    $stmt->bind_param('i', $ownerId);
    $stmt->execute();
    $res = $stmt->get_result();
    while ($row = $res ? $res->fetch_assoc() : null) {
        if (!$row) break;
        $turfs[] = [
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name'],
            'price_per_hour' => (float)($row['price_per_hour'] ?? 0),
            'status' => $row['status']
```

```
        ];
    }
    $stmt->close();
    return $turfs;
}

function get_slots_for_day($conn, $ownerId, $turfId, $slotDate) {
    $turfStmt = $conn->prepare('
        SELECT turf_id, turf_name, price_per_hour
        FROM turfs
        WHERE owner_id = ? AND turf_id = ?
        LIMIT 1
    ');
    if (!$turfStmt) {
        return [null, null];
    }
    $turfStmt->bind_param('ii', $ownerId, $turfId);
    $turfStmt->execute();
    $turfRes = $turfStmt->get_result();
    $turfRow = $turfRes ? $turfRes->fetch_assoc() : null;
    $turfStmt->close();

    if (!$turfRow) {
        return [null, null];
    }

    if (!seed_default_slots_if_missing($conn, $turfId, $slotDate, floatval($turfRow['price_per_hour'] ?? 0)))
{
        return [$turfRow, null];
    }

    $todayDate = date('Y-m-d');
    $nowTime = date('H:i:s');

    $sql = "
    SELECT
      s.slot_id,
      s.slot_date,
      s.start_time,
      s.end_time,
      s.base_price,
      COALESCE(s.is_enabled, 1) AS is_enabled,
      b.booking_id,
      COALESCE(b.booking_status, '') AS booking_status
    FROM slots s
    LEFT JOIN bookings b
      ON b.booking_id = (
          SELECT b2.booking_id
          FROM bookings b2
          WHERE b2.slot_id = s.slot_id
            AND b2.booking_status IN ('pending', 'confirmed', 'completed', 'resell_listed', 'resold')
          ORDER BY b2.booking_id DESC
          LIMIT 1
      )
    WHERE s.turf_id = ?
      AND s.slot_date = ?
      AND COALESCE(b.booking_id, 0) = 0
    ORDER BY s.start_time ASC, s.end_time ASC
    ";

    $stmt = $conn->prepare($sql);
    if (!$stmt) {
        return [$turfRow, null];
    }
    $stmt->bind_param('is', $turfId, $slotDate);
    $stmt->execute();
    $res = $stmt->get_result();

    $slots = [];
    while ($row = $res ? $res->fetch_assoc() : null) {
        if (!$row) break;
        if ($slotDate < $todayDate) {
```

```
            continue;
        }
        if ($slotDate === $todayDate && (string)$row['start_time'] <= $nowTime) {
            continue;
        }
        $slots[] = [
            'slot_id' => (int)$row['slot_id'],
            'slot_date' => $row['slot_date'],
            'start_time' => $row['start_time'],
            'end_time' => $row['end_time'],
            'base_price' => (float)($row['base_price'] ?? 0),
            'is_enabled' => intval($row['is_enabled'] ?? 1) === 1,
            'booking_id' => 0,
            'booking_status' => '',
            'is_booked' => false
        ];
    }
    $stmt->close();

    return [$turfRow, $slots];
}

$requestOwnerId = intval($_GET['owner_id'] ?? ($_POST['owner_id'] ?? 0));
if ($requestOwnerId <= 0) {
    $body = parse_json_body();
    $requestOwnerId = intval($body['owner_id'] ?? 0);
}

if ($requestOwnerId <= 0) {
    send_response(false, 'Invalid owner id.', 400);
}

$ownerStmt = $conn->prepare('SELECT owner_id FROM turf_owners WHERE owner_id = ? LIMIT 1');
if (!$ownerStmt) {
    send_response(false, 'Server error while checking owner.', 500);
}
$ownerStmt->bind_param('i', $requestOwnerId);
$ownerStmt->execute();
$ownerRes = $ownerStmt->get_result();
$ownerRow = $ownerRes ? $ownerRes->fetch_assoc() : null;
$ownerStmt->close();

if (!$ownerRow) {
    send_response(false, 'Owner not found.', 404);
}

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $turfs = get_owner_turfs($conn, $requestOwnerId);
    $selectedTurfId = intval($_GET['turf_id'] ?? 0);
    $slotDate = trim((string)($_GET['slot_date'] ?? date('Y-m-d')));
    $todayDate = date('Y-m-d');

    if (!preg_match('/^\d{4}-\d{2}-\d{2}$/', $slotDate)) {
        $slotDate = date('Y-m-d');
    }

    if ($slotDate < $todayDate) {
        send_response(true, 'Past date slots are not controllable.', 200, [
            'turfs' => $turfs,
            'selected_turf' => null,
            'slot_date' => $slotDate,
            'slots' => []
        ]);
    }

    $selectedTurf = null;
    $slots = [];
    if ($selectedTurfId > 0) {
        [$selectedTurf, $slotRows] = get_slots_for_day($conn, $requestOwnerId, $selectedTurfId, $slotDate);
        if ($selectedTurf && is_array($slotRows)) {
            $slots = $slotRows;
        }
```

```
    }

    send_response(true, 'Owner slot data loaded.', 200, [
        'turfs' => $turfs,
        'selected_turf' => $selectedTurf,
        'slot_date' => $slotDate,
        'slots' => $slots
    ]);
}

$payload = parse_json_body();
$turfId = intval($payload['turf_id'] ?? 0);
$slotDate = trim((string)($payload['slot_date'] ?? ''));
$slotIds = $payload['slot_ids'] ?? [];
$isEnabled = intval($payload['is_enabled'] ?? -1);

if ($turfId <= 0 || !preg_match('/^\d{4}-\d{2}-\d{2}$/', $slotDate) || !is_array($slotIds) || ($isEnabled !==
0 && $isEnabled !== 1)) {
    send_response(false, 'Missing or invalid slot control data.', 400);
}

$todayDate = date('Y-m-d');
if ($slotDate < $todayDate) {
    send_response(false, 'Past date slots cannot be updated.', 400);
}

$slotIds = array_values(array_filter(array_map('intval', $slotIds), function ($id) {
    return $id > 0;
}));
if (!count($slotIds)) {
    send_response(false, 'Please select at least one slot.', 400);
}

$placeholders = implode(',', array_fill(0, count($slotIds), '?'));
$types = 'iis' . str_repeat('i', count($slotIds));
$params = [$requestOwnerId, $turfId, $slotDate];
foreach ($slotIds as $slotId) {
    $params[] = $slotId;
}

$sql = "
SELECT
  s.slot_id,
  COALESCE(b.booking_id, 0) AS booking_id,
  s.start_time
FROM slots s
JOIN turfs t ON t.turf_id = s.turf_id
LEFT JOIN bookings b
  ON b.booking_id = (
      SELECT b2.booking_id
      FROM bookings b2
      WHERE b2.slot_id = s.slot_id
        AND b2.booking_status IN ('pending', 'confirmed', 'completed', 'resell_listed', 'resold')
      ORDER BY b2.booking_id DESC
      LIMIT 1
  )
WHERE t.owner_id = ?
  AND s.turf_id = ?
  AND s.slot_date = ?
  AND s.slot_id IN ($placeholders)
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while validating selected slots.', 500);
}

$bind = [];
$bind[] = &$types;
foreach ($params as $index => $value) {
    $bind[] = &$params[$index];
}
```

```
call_user_func_array([$stmt, 'bind_param'], $bind);
$stmt->execute();
$res = $stmt->get_result();

$allowedSlotIds = [];
$skippedBooked = 0;
$skippedPast = 0;
    $nowTime = date('H:i:s');
while ($row = $res ? $res->fetch_assoc() : null) {
    if (!$row) break;
    if (intval($row['booking_id'] ?? 0) > 0) {
        $skippedBooked++;
        continue;
    }
    if ($slotDate === $todayDate && (string)($row['start_time'] ?? '') <= $nowTime) {
        $skippedPast++;
        continue;
    }
    $allowedSlotIds[] = intval($row['slot_id']);
}
$stmt->close();

if (!count($allowedSlotIds)) {
    send_response(false, 'Selected slots are already booked or unavailable for update.', 409, [
        'updated_count' => 0,
        'skipped_booked' => $skippedBooked,
        'skipped_past' => $skippedPast
    ]);
}

$updatePlaceholders = implode(',', array_fill(0, count($allowedSlotIds), '?'));
$updateTypes = 'is' . str_repeat('i', count($allowedSlotIds));
$updateParams = [$isEnabled, $slotDate];
foreach ($allowedSlotIds as $slotId) {
    $updateParams[] = $slotId;
}

$updateSql = "UPDATE slots SET is_enabled = ? WHERE slot_date = ? AND slot_id IN ($updatePlaceholders)";
$updateStmt = $conn->prepare($updateSql);
if (!$updateStmt) {
    send_response(false, 'Server error while updating slots.', 500);
}

$updateBind = [];
$updateBind[] = &$updateTypes;
foreach ($updateParams as $index => $value) {
    $updateBind[] = &$updateParams[$index];
}
call_user_func_array([$updateStmt, 'bind_param'], $updateBind);
$updateStmt->execute();
$updatedCount = $updateStmt->affected_rows;
$updateStmt->close();

send_response(true, $isEnabled === 1 ? 'Selected slots are now enabled.' : 'Selected slots are now disabled.',
 200, [
    'updated_count' => max(0, $updatedCount),
    'skipped_booked' => $skippedBooked,
    'skipped_past' => $skippedPast
]);
?>
```

==========
22. backend/promo_code_helper.php
==========

```
<?php

function promo_time_to_minutes($timeText) {
    $value = (string)$timeText;
    $parts = explode(':', $value);
```

```
    if (count($parts) < 2) {
        return 0;
    }

    $hours = intval($parts[0] ?? 0);
    $minutes = intval($parts[1] ?? 0);
    return ($hours * 60) + $minutes;
}

function promo_calculate_slot_subtotal($basePrice, $startTime, $endTime) {
    $startMinutes = promo_time_to_minutes($startTime);
    $endMinutes = promo_time_to_minutes($endTime);

    if ($endMinutes > $startMinutes && $basePrice > 0) {
        $durationHours = ($endMinutes - $startMinutes) / 60;
        return round($durationHours * floatval($basePrice), 2);
    }

    return round(floatval($basePrice), 2);
}

function promo_validate_for_owner_turf($conn, $ownerId, $promoCode, $subtotal, $slotDate) {
    $code = strtoupper(trim((string)$promoCode));
    if ($code === '') {
        return [
            'success' => true,
            'promo_code' => '',
            'discount_amount' => 0,
            'final_total' => round(floatval($subtotal), 2),
            'message' => 'No promo code applied.'
        ];
    }

    $existsStmt = $conn->prepare('SELECT promo_id, owner_id, code, discount_type, discount_value,
min_booking_amount, start_date, end_date, is_active FROM promo_codes WHERE code = ? LIMIT 1');
    if (!$existsStmt) {
        return [
            'success' => false,
            'message' => 'Server error while checking promo code.'
        ];
    }

    $existsStmt->bind_param('s', $code);
    $existsStmt->execute();
    $res = $existsStmt->get_result();
    $promo = $res ? $res->fetch_assoc() : null;
    $existsStmt->close();

    if (!$promo) {
        return [
            'success' => false,
            'message' => 'Promo code not found.'
        ];
    }

    if (intval($promo['owner_id'] ?? 0) !== intval($ownerId)) {
        return [
            'success' => false,
            'message' => 'This promo code is not valid for this turf.'
        ];
    }

    if (intval($promo['is_active'] ?? 0) !== 1) {
        return [
            'success' => false,
            'message' => 'This promo code is inactive.'
        ];
    }

    $startDate = trim((string)($promo['start_date'] ?? ''));
    $endDate = trim((string)($promo['end_date'] ?? ''));
    if ($startDate !== '' && $slotDate < $startDate) {
```

```
        return [
            'success' => false,
            'message' => 'This promo code is not active yet.'
        ];
    }
    if ($endDate !== '' && $slotDate > $endDate) {
        return [
            'success' => false,
            'message' => 'This promo code has expired.'
        ];
    }

    $minBookingAmount = floatval($promo['min_booking_amount'] ?? 0);
    if ($minBookingAmount > 0 && floatval($subtotal) < $minBookingAmount) {
        return [
            'success' => false,
            'message' => 'Booking amount is too low for this promo code.'
        ];
    }

    $discountType = (string)($promo['discount_type'] ?? 'fixed');
    $discountValue = floatval($promo['discount_value'] ?? 0);
    $discountAmount = 0;

    if ($discountType === 'percent') {
        $discountAmount = round(floatval($subtotal) * ($discountValue / 100), 2);
    } else {
        $discountAmount = round($discountValue, 2);
    }

    if ($discountAmount < 0) {
        $discountAmount = 0;
    }

    if ($discountAmount > floatval($subtotal)) {
        $discountAmount = round(floatval($subtotal), 2);
    }

    $finalTotal = round(max(0, floatval($subtotal) - $discountAmount), 2);

    return [
        'success' => true,
        'promo_id' => intval($promo['promo_id'] ?? 0),
        'promo_code' => $code,
        'discount_type' => $discountType,
        'discount_value' => $discountValue,
        'discount_amount' => $discountAmount,
        'final_total' => $finalTotal,
        'message' => 'Promo code applied successfully.'
    ];
}
```

==========
23. backend/register.php
==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

include 'db_connection.php';
```

```php
function send_response($success, $status, $code = 200) {
    http_response_code($code);
    echo json_encode([
        'success' => $success,
        'status' => $status
    ]);
    exit();
}

$contentType = $_SERVER['CONTENT_TYPE'] ?? '';
if (stripos($contentType, 'application/json') !== false) {
    $data = json_decode(file_get_contents('php://input'), true);
} else {
    $data = $_POST;
    if (empty($data)) {
        parse_str(file_get_contents('php://input'), $data);
    }
}

if (!is_array($data) || empty($data)) {
    send_response(false, 'No data received.', 400);
}

$full_name = trim($data['full_name'] ?? '');
$email = trim($data['email'] ?? '');
$phone = trim($data['phone'] ?? '');
$password = (string)($data['password'] ?? '');

if ($full_name === '' || $email === '' || $phone === '' || $password === '') {
    send_response(false, 'All fields are required.', 400);
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    send_response(false, 'Invalid email address.', 400);
}

if (strlen($password) < 6) {
    send_response(false, 'Password must be at least 6 characters.', 400);
}

$checkSql = 'SELECT email FROM users WHERE email = ? LIMIT 1';
$checkStmt = $conn->prepare($checkSql);
if (!$checkStmt) {
    send_response(false, 'Server error while checking user.', 500);
}

$checkStmt->bind_param('s', $email);
$checkStmt->execute();
$checkStmt->store_result();
if ($checkStmt->num_rows > 0) {
    $checkStmt->close();
    send_response(false, 'Email already registered.', 409);
}
$checkStmt->close();

$password_hash = password_hash($password, PASSWORD_DEFAULT);
$language_pref = 'en';
$status = 'active';

$sql = 'INSERT INTO users (full_name, email, phone, password_hash, language_pref, status, created_at)
VALUES (?, ?, ?, ?, ?, ?, NOW())';
$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, 'Server error while preparing registration.', 500);
}

$stmt->bind_param('ssssss', $full_name, $email, $phone, $password_hash, $language_pref, $status);

if ($stmt->execute()) {
    $stmt->close();
    send_response(true, 'User registered successfully.', 200);
}
```

```
$stmt->close();
send_response(false, 'Registration failed.', 500);
?>
```

==========

24. backend/review_pending.php

==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $pending, $code = 200) {
    http_response_code($code);
    echo json_encode([
        'success' => $success,
        'pending' => $pending
    ]);
    exit();
}

$userId = (int)($_GET['user_id'] ?? 0);
if ($userId <= 0) {
    send_response(false, null, 400);
}

$sql = "
SELECT
    b.booking_id,
    b.created_at,
    s.turf_id,
    t.turf_name
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
LEFT JOIN reviews r ON r.booking_id = b.booking_id
WHERE b.user_id = ?
  AND b.booking_status = 'completed'
  AND r.review_id IS NULL
ORDER BY b.created_at DESC
LIMIT 1
";

$stmt = $conn->prepare($sql);
if (!$stmt) {
    send_response(false, null, 500);
}

$stmt->bind_param('i', $userId);
$stmt->execute();
$res = $stmt->get_result();
$pending = $res ? $res->fetch_assoc() : null;
$stmt->close();

send_response(true, $pending ?: null);
?>
```

==========

25. backend/review_submit.php
==========

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE '$name'");
    return $res && $res->num_rows > 0;
}

$userId = (int)($_POST['user_id'] ?? 0);
$bookingId = (int)($_POST['booking_id'] ?? 0);
$turfId = (int)($_POST['turf_id'] ?? 0);
$rating = (int)($_POST['rating'] ?? 0);
$comment = trim((string)($_POST['comment'] ?? ''));

if ($userId <= 0 || $bookingId <= 0 || $turfId <= 0 || $rating < 1 || $rating > 5) {
    send_response(false, 'Invalid review data.', 422);
}

if (!table_exists($conn, 'reviews')) {
    $createSql = "CREATE TABLE IF NOT EXISTS reviews (
        review_id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        booking_id INT NOT NULL UNIQUE,
        turf_id INT NOT NULL,
        rating INT NOT NULL,
        comment TEXT NULL,
        created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
    )";

    if (!$conn->query($createSql)) {
        send_response(false, 'Could not prepare review storage.', 500);
    }
}

$bookingSql = "
SELECT b.booking_id, b.user_id, b.booking_status, s.turf_id, t.turf_name
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
WHERE b.booking_id = ? AND b.user_id = ?
LIMIT 1
";
$bookingStmt = $conn->prepare($bookingSql);
if (!$bookingStmt) {
    send_response(false, 'Could not verify booking.', 500);
}

$bookingStmt->bind_param('ii', $bookingId, $userId);
$bookingStmt->execute();
```

```
$bookingRes = $bookingStmt->get_result();
$booking = $bookingRes ? $bookingRes->fetch_assoc() : null;
$bookingStmt->close();

if (!$booking) {
    send_response(false, 'Booking not found for this user.', 404);
}

if ((int)($booking['turf_id'] ?? 0) != $turfId) {
    send_response(false, 'Review turf mismatch.', 422);
}

$status = strtolower((string)($booking['booking_status'] ?? ''));
if ($status !== 'completed') {
    send_response(false, 'You can review only completed bookings.', 422);
}

$duplicateStmt = $conn->prepare("SELECT review_id FROM reviews WHERE booking_id = ? LIMIT 1");
if (!$duplicateStmt) {
    send_response(false, 'Could not validate previous review.', 500);
}

$duplicateStmt->bind_param('i', $bookingId);
$duplicateStmt->execute();
$duplicateRes = $duplicateStmt->get_result();
$alreadyReviewed = $duplicateRes && $duplicateRes->num_rows > 0;
$duplicateStmt->close();

if ($alreadyReviewed) {
    send_response(false, 'This booking is already reviewed.', 409);
}

$insertStmt = $conn->prepare("INSERT INTO reviews (user_id, booking_id, turf_id, rating, comment, created_at)
VALUES (?, ?, ?, ?, ?, NOW())");
if (!$insertStmt) {
    send_response(false, 'Could not save review.', 500);
}

$insertStmt->bind_param('iiiis', $userId, $bookingId, $turfId, $rating, $comment);
$ok = $insertStmt->execute();
$insertStmt->close();

if (!$ok) {
    send_response(false, 'Failed to submit review.', 500);
}

send_response(true, 'Review submitted. Review points added to your dashboard.', 200, [
    'review' => [
        'booking_id' => $bookingId,
        'turf_name' => $booking['turf_name'] ?? 'Turf',
        'rating' => $rating
    ]
]);
?>
```

==========

26. backend/user_dashboard.php

==========

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}
```

```
require_once ’db_connection.php’;

function send_response($success, $status, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        ’success’ => $success,
        ’status’ => $status
    ], $extra));
    exit();
}

function table_exists($conn, $table) {
    $name = $conn->real_escape_string($table);
    $res = $conn->query("SHOW TABLES LIKE ’$name’");
    return $res && $res->num_rows > 0;
}

function column_exists($conn, $table, $column) {
    $safeTable = preg_replace(’/[^a-zA-Z0-9_]/’, ’’, $table);
    if ($safeTable === ’’) {
        return false;
    }

    $safeColumn = $conn->real_escape_string($column);
    $res = $conn->query("SHOW COLUMNS FROM ‘$safeTable’ LIKE ’$safeColumn’");
    return $res && $res->num_rows > 0;
}

$userId = intval($_GET[’user_id’] ?? 0);
if ($userId <= 0) {
    send_response(false, ’Invalid user id.’, 400);
}

$stats = [
    ’total’ => 0,
    ’upcoming’ => 0,
    ’cancelled’ => 0
];


$statsSql = "
SELECT
  COALESCE(SUM(CASE WHEN b.booking_status IN (’confirmed’,’completed’) THEN 1 ELSE 0 END), 0) AS
total_bookings,
  COALESCE(SUM(CASE WHEN b.booking_status = ’confirmed’ AND s.slot_date >= CURDATE() THEN 1 ELSE 0 END), 0) AS
 upcoming_bookings,
  COALESCE(SUM(CASE WHEN b.booking_status = ’cancelled’ THEN 1 ELSE 0 END), 0) AS cancelled_bookings
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
WHERE b.user_id = ?
";
$statsStmt = $conn->prepare($statsSql);
if (!$statsStmt) {
    send_response(false, ’Server error while loading booking stats.’, 500);
}
$statsStmt->bind_param(’i’, $userId);
$statsStmt->execute();
$statsRes = $statsStmt->get_result();
$statsRow = $statsRes ? $statsRes->fetch_assoc() : null;
$statsStmt->close();

if ($statsRow) {
    $stats[’total’] = (int)($statsRow[’total_bookings’] ?? 0);
    $stats[’upcoming’] = (int)($statsRow[’upcoming_bookings’] ?? 0);
    $stats[’cancelled’] = (int)($statsRow[’cancelled_bookings’] ?? 0);
}

$nextBooking = null;
$hasTurfLocationColumn = column_exists($conn, ’turfs’, ’location’);
$locationExpr = $hasTurfLocationColumn
    ? "CONCAT_WS(’, ’, NULLIF(t.area, ’’), NULLIF(t.city, ’’), NULLIF(t.location, ’’), NULLIF(t.address, ’’))
AS location_text"
```

```
    : "CONCAT_WS(', ', NULLIF(t.area, ''), NULLIF(t.city, ''), NULLIF(t.address, '')) AS location_text";
$hasRefundRequests = table_exists($conn, 'refund_requests');
$refundSelectExpr = $hasRefundRequests ? 'rr.status AS refund_request_status,' : 'NULL AS
refund_request_status,';
$refundJoin = $hasRefundRequests ? 'LEFT JOIN refund_requests rr ON rr.booking_id = b.booking_id' : '';

$nextSql = "
SELECT
  b.booking_id,
  b.booking_status,
  b.booked_price,
  s.slot_date,
  s.start_time,
  s.end_time,
  t.turf_id,
  t.turf_name,
  t.cancel_before_hours,
  t.refund_percent,
  $refundSelectExpr
  $locationExpr
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
$refundJoin
WHERE b.user_id = ?
  AND b.booking_status = 'confirmed'
  AND s.slot_date >= CURDATE()
ORDER BY s.slot_date ASC, s.start_time ASC
LIMIT 1
";
$nextStmt = $conn->prepare($nextSql);
if ($nextStmt) {
    $nextStmt->bind_param('i', $userId);
    $nextStmt->execute();
    $nextRes = $nextStmt->get_result();
    $row = $nextRes ? $nextRes->fetch_assoc() : null;
    $nextStmt->close();

    if ($row) {
        $nextBooking = [
            'booking_id' => (int)$row['booking_id'],
            'booking_status' => $row['booking_status'],
            'booked_price' => (float)$row['booked_price'],
            'slot_date' => $row['slot_date'],
            'start_time' => $row['start_time'],
            'end_time' => $row['end_time'],
            'turf_id' => (int)$row['turf_id'],
            'turf_name' => $row['turf_name'],
            'cancel_before_hours' => (int)($row['cancel_before_hours'] ?? 24),
            'refund_percent' => (int)($row['refund_percent'] ?? 80),
            'location' => $row['location_text'],
            'refund_request_status' => $row['refund_request_status']
        ];
    }
}

$activities = [];
$activitySql = "
SELECT
  b.booking_status,
  b.created_at,
  t.turf_name,
  s.slot_date,
  s.start_time,
  s.end_time
FROM bookings b
JOIN slots s ON b.slot_id = s.slot_id
JOIN turfs t ON s.turf_id = t.turf_id
WHERE b.user_id = ?
ORDER BY b.created_at DESC
LIMIT 8
";
```

```
$activityStmt = $conn->prepare($activitySql);
if ($activityStmt) {
    $activityStmt->bind_param('i', $userId);
    $activityStmt->execute();
    $activityRes = $activityStmt->get_result();

    $statusMap = [
        'confirmed' => 'You booked %s',
        'pending' => 'Booking request pending for %s',
        'cancelled' => 'Booking cancelled for %s',
        'completed' => 'Booking completed at %s',
        'resell_listed' => 'Slot listed for resell from %s',
        'resold' => 'Slot resold successfully for %s'
    ];

    while ($a = $activityRes ? $activityRes->fetch_assoc() : null) {
        if (!$a) {
            break;
        }
        $status = strtolower((string)($a['booking_status'] ?? ''));
        $turfName = (string)($a['turf_name'] ?? 'Turf');
        $template = $statusMap[$status] ?? 'Booking activity at %s';

        $activities[] = [
            'message' => sprintf($template, $turfName),
            'time' => $a['created_at'],
            'booking_status' => $status,
            'turf_name' => $turfName,
            'slot_date' => $a['slot_date'],
            'start_time' => $a['start_time'],
            'end_time' => $a['end_time'],
            'detail_message' => ''
        ];
    }

    $activityStmt->close();
}

if (table_exists($conn, 'notifications')) {
    $notifSql = "SELECT title, message, created_at FROM notifications WHERE user_id = ? ORDER BY created_at
DESC LIMIT 8";
    $notifStmt = $conn->prepare($notifSql);
    if ($notifStmt) {
        $notifStmt->bind_param('i', $userId);
        $notifStmt->execute();
        $notifRes = $notifStmt->get_result();

        while ($n = $notifRes ? $notifRes->fetch_assoc() : null) {
            if (!$n) {
                break;
            }

            $title = trim((string)($n['title'] ?? ''));
            $message = trim((string)($n['message'] ?? ''));
            $detailMessage = '';
            if ($message !== '' && stripos($message, 'Admin note:') !== false) {
                $detailMessage = $message;
            }

            $activities[] = [
                'message' => $title !== '' ? $title : ($message !== '' ? $message : 'Notification'),
                'time' => $n['created_at'],
                'booking_status' => 'notification',
                'turf_name' => '',
                'slot_date' => null,
                'start_time' => null,
                'end_time' => null,
                'detail_message' => $detailMessage
            ];
        }

        $notifStmt->close();
```

```
    }
}

usort($activities, function($a, $b) {
    $aTs = strtotime((string)($a['time'] ?? ''));
    $bTs = strtotime((string)($b['time'] ?? ''));
    return $bTs <=> $aTs;
});

if (count($activities) > 5) {
    $activities = array_slice($activities, 0, 5);
}

$bookingPoints = 0;
$reviewPoints = 0;
$resellPoints = 0;
$referralPoints = 0;

$pointsBookingSql = "SELECT COALESCE(SUM(booked_price), 0) AS completed_value FROM bookings WHERE user_id = ?
AND booking_status IN ('confirmed', 'completed')";
$pointsStmt = $conn->prepare($pointsBookingSql);
if ($pointsStmt) {
    $pointsStmt->bind_param('i', $userId);
    $pointsStmt->execute();
    $pointsRes = $pointsStmt->get_result();
    $pointsRow = $pointsRes ? $pointsRes->fetch_assoc() : null;
    $pointsStmt->close();

    $completedValue = (float)($pointsRow['completed_value'] ?? 0);
    $bookingPoints = (int)floor($completedValue / 100);
}

if (table_exists($conn, 'reviews')) {
    $reviewSql = "SELECT COUNT(*) AS review_count FROM reviews WHERE user_id = ?";
    $reviewStmt = $conn->prepare($reviewSql);
    if ($reviewStmt) {
        $reviewStmt->bind_param('i', $userId);
        $reviewStmt->execute();
        $reviewRes = $reviewStmt->get_result();
        $reviewRow = $reviewRes ? $reviewRes->fetch_assoc() : null;
        $reviewStmt->close();

        $reviewCount = (int)($reviewRow['review_count'] ?? 0);
        $reviewPoints = $reviewCount * 5;
    }
}

if (table_exists($conn, 'resell_listings')) {
    $resellSql = "SELECT COALESCE(SUM(CASE WHEN status = 'sold' THEN 1 ELSE 0 END), 0) AS sold_count FROM
resell_listings WHERE seller_user_id = ?";
    $resellStmt = $conn->prepare($resellSql);
    if ($resellStmt) {
        $resellStmt->bind_param('i', $userId);
        $resellStmt->execute();
        $resellRes = $resellStmt->get_result();
        $resellRow = $resellRes ? $resellRes->fetch_assoc() : null;
        $resellStmt->close();

        $soldCount = (int)($resellRow['sold_count'] ?? 0);
        $resellPoints = $soldCount * 8;
    }
}

if (table_exists($conn, 'user_point_logs') && column_exists($conn, 'user_point_logs', 'user_id')) {
    if (column_exists($conn, 'user_point_logs', 'action') && column_exists($conn, 'user_point_logs',
'points_change')) {
        $refSql = "SELECT COALESCE(SUM(CASE WHEN action = 'promo' THEN points_change ELSE 0 END), 0) AS
referral_points FROM user_point_logs WHERE user_id = ?";
    } elseif (column_exists($conn, 'user_point_logs', 'source') && column_exists($conn, 'user_point_logs',
'points')) {
        $refSql = "SELECT COALESCE(SUM(CASE WHEN source = 'admin_adjustment' THEN points ELSE 0 END), 0) AS
referral_points FROM user_point_logs WHERE user_id = ?";
```

```
    } else {
        $refSql = '';
    }

    if ($refSql !== '') {
        $refStmt = $conn->prepare($refSql);
        if ($refStmt) {
            $refStmt->bind_param('i', $userId);
            $refStmt->execute();
            $refRes = $refStmt->get_result();
            $refRow = $refRes ? $refRes->fetch_assoc() : null;
            $refStmt->close();

            $referralPoints = (int)($refRow['referral_points'] ?? 0);
            if ($referralPoints < 0) {
                $referralPoints = 0;
            }
        }
    }
}

$totalPoints = $bookingPoints + $reviewPoints + $resellPoints + $referralPoints;

send_response(true, 'Dashboard loaded.', 200, [
    'stats' => $stats,
    'next_booking' => $nextBooking,
    'activities' => $activities,
    'points' => $totalPoints,
    'points_breakdown' => [
        'booking' => $bookingPoints,
        'review' => $reviewPoints,
        'resell' => $resellPoints,
        'referral' => $referralPoints
    ]
]);
?>
```

==========
27. backend/validate_promo_code.php
==========

```
<?php
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit();
}

require_once 'db_connection.php';
require_once 'promo_code_helper.php';
date_default_timezone_set('Asia/Dhaka');

function send_response($success, $message, $code = 200, $extra = []) {
    http_response_code($code);
    echo json_encode(array_merge([
        'success' => $success,
        'message' => $message
    ], $extra));
    exit();
}

$payload = json_decode(file_get_contents('php://input'), true);
```

```
if (!is_array($payload)) {
    send_response(false, 'Invalid request body.', 400);
}

$slotId = intval($payload['slot_id'] ?? 0);
$promoCode = trim((string)($payload['promo_code'] ?? ''));

if ($slotId <= 0 || $promoCode === '') {
    send_response(false, 'Slot and promo code are required.', 400);
}

$slotStmt = $conn->prepare('
    SELECT s.slot_id, s.slot_date, s.start_time, s.end_time, s.base_price, s.is_enabled, t.owner_id
    FROM slots s
    JOIN turfs t ON t.turf_id = s.turf_id
    WHERE s.slot_id = ?
    LIMIT 1
');
if (!$slotStmt) {
    send_response(false, 'Server error while checking slot.', 500);
}
$slotStmt->bind_param('i', $slotId);
$slotStmt->execute();
$slotRes = $slotStmt->get_result();
$slot = $slotRes ? $slotRes->fetch_assoc() : null;
$slotStmt->close();

if (!$slot || intval($slot['is_enabled'] ?? 0) !== 1) {
    send_response(false, 'Selected slot is not available.', 404);
}

$now = new DateTime('now', new DateTimeZone('Asia/Dhaka'));
$slotStart = DateTime::createFromFormat('Y-m-d H:i:s', $slot['slot_date'] . ' ' .
substr((string)$slot['start_time'], 0, 8), new DateTimeZone('Asia/Dhaka'));
if (!$slotStart || $slotStart <= $now) {
    send_response(false, 'Past date/time slot cannot use promo code.', 400);
}

$bookingStmt = $conn->prepare("SELECT booking_id FROM bookings WHERE slot_id = ? AND booking_status IN
('pending','confirmed','completed','resell_listed','resold') LIMIT 1");
if (!$bookingStmt) {
    send_response(false, 'Server error while checking booking state.', 500);
}
$bookingStmt->bind_param('i', $slotId);
$bookingStmt->execute();
$bookingStmt->store_result();
if ($bookingStmt->num_rows > 0) {
    $bookingStmt->close();
    send_response(false, 'This slot is already booked.', 409);
}
$bookingStmt->close();

$subtotal = promo_calculate_slot_subtotal($slot['base_price'], $slot['start_time'], $slot['end_time']);
$promoResult = promo_validate_for_owner_turf($conn, intval($slot['owner_id'] ?? 0), $promoCode, $subtotal,
(string)$slot['slot_date']);
if (!$promoResult['success']) {
    send_response(false, $promoResult['message'], 400);
}

send_response(true, $promoResult['message'], 200, [
    'promo' => [
        'code' => $promoResult['promo_code'],
        'discount_type' => $promoResult['discount_type'],
        'discount_value' => $promoResult['discount_value'],
        'discount_amount' => $promoResult['discount_amount'],
        'subtotal' => $subtotal,
        'final_total' => $promoResult['final_total']
    ]
]);
?>
```

==========
28. src/App.vue
==========

```
<template>
  <router-view />
</template>

<script>
export default {}
</script>
```

==========
29. src/assets/theme.css
==========

```
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  background: #f8fafc;
  font-family: 'Poppins', sans-serif;
  color: #1f2937;
}
```

==========
30. src/components/AdminAnalyticsSection.vue
==========

```
<template>
  <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
 transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
    <div>
      <h3 class="m-0 text-[18px] font-bold text-slate-900">System Analytics</h3>
      <p class="mt-2 text-slate-600">Top-rated turfs, active users, revenue trends, booking growth.</p>
    </div>

    <div class="mt-3 grid grid-cols-2 gap-3 max-[1200px]:grid-cols-1">
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Top Rated Turfs</h3>
        <ul v-if="analytics.top_rated_turfs.length" class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="t in analytics.top_rated_turfs"
            :key="t.turf_id"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>{{ t.turf_name }}</span>
            <b class="text-slate-900">{{ Number(t.rating_avg || 0).toFixed(2) }}</b>
          </li>
        </ul>
        <p v-else class="mt-3 text-slate-500">No data</p>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Most Active Users</h3>
        <ul v-if="analytics.most_active_users.length" class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="u in analytics.most_active_users"
            :key="u.user_id"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
```

```
          >
            <span>{{ u.full_name }}</span>
            <b class="text-slate-900">{{ u.booking_count }}</b>
          </li>
        </ul>
        <p v-else class="mt-3 text-slate-500">No data</p>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Revenue Trend</h3>
        <ul v-if="analytics.revenue_trend.length" class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="r in analytics.revenue_trend"
            :key="`rev-${r.month}`"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>{{ r.month }}</span>
            <b class="text-slate-900">Tk {{ formatMoney(r.revenue) }}</b>
          </li>
        </ul>
        <p v-else class="mt-3 text-slate-500">No data</p>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Booking Growth</h3>
        <ul v-if="analytics.booking_growth.length" class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="g in analytics.booking_growth"
            :key="`g-${g.month}`"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>{{ g.month }}</span>
            <b class="text-slate-900">{{ g.bookings }}</b>
          </li>
        </ul>
        <p v-else class="mt-3 text-slate-500">No data</p>
      </article>
    </div>
  </section>
</template>

<script>
export default {
  name: 'AdminAnalyticsSection',
  props: {
    analytics: { type: Object, required: true },
    formatMoney: { type: Function, required: true }
  }
}
</script>
```

==========
31. src/components/AdminBookingRequestsSection.vue
==========

```
<template>
  <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
 transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
    <div>
      <h3 class="m-0 text-[18px] font-bold text-slate-900">Accept / Reject Booking Requests</h3>
      <p class="mt-2 text-slate-600">Points are awarded instantly when a booking is confirmed.</p>
    </div>

    <p v-if="!pendingBookings.length" class="mt-3 text-slate-500">No pending booking requests.</p>

    <div v-else class="mt-3 grid grid-cols-2 gap-3 max-[1200px]:grid-cols-1">
```

```
      <article
        v-for="item in pendingBookings"
        :key="item.booking_id"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <h4 class="m-0 text-[18px] font-bold text-slate-900">Booking #{{ item.booking_id }} - {{
item.turf_name }}</h4>
        <div class="mt-3 space-y-1.5 text-sm text-slate-700">
          <p class="m-0"><b class="text-slate-900">User:</b> {{ item.user_name }} ({{ item.user_email }})</p>
          <p class="m-0"><b class="text-slate-900">Date:</b> {{ formatDate(item.slot_date) }}</p>
          <p class="m-0"><b class="text-slate-900">Time:</b> {{ formatTime(item.start_time) }} - {{
formatTime(item.end_time) }}</p>
          <p class="m-0"><b class="text-slate-900">Price:</b> Tk {{ formatMoney(item.booked_price) }}</p>
        </div>

        <div class="mt-3 flex flex-wrap gap-2">
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2 font-semibold text-white
shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200 hover:-translate-y-0.5 hover:bg-[#2f57c2]
hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('booking', item.booking_id)"
            @click="$emit('review', { item, action: 'confirm' })"
          >
            Accept
          </button>
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2 font-semibold
text-[#991b1b] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('booking', item.booking_id)"
            @click="$emit('review', { item, action: 'reject' })"
          >
            Reject
          </button>
        </div>
      </article>
    </div>
  </section>
</template>

<script>
export default {
  name: 'AdminBookingRequestsSection',
  props: {
    pendingBookings: { type: Array, default: () => [] },
    isActionLoading: { type: Function, required: true },
    formatMoney: { type: Function, required: true },
    formatDate: { type: Function, required: true },
    formatTime: { type: Function, required: true }
  },
  emits: ['review']
}
</script>



==========================================================================================
32. src/components/AdminDashboard.vue
==========================================================================================

<template>
  <div class="min-h-screen w-full box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)]
p-2.5 font-poppins text-[#111827]">
    <AppTopbar
      wrapper-class="mb-3 w-full px-4 py-3.5 max-md:flex-wrap"
      left-class="gap-2.5 max-md:flex-wrap"
      right-class="gap-1.5 max-md:flex-wrap"
    >
      <template #left>
```

```
        <GlassButton class="whitespace-nowrap" @click="switchToUserDashboard">Switch User
Dashboard</GlassButton>
      </template>

      <template #right>
        <UserDashboardProfileMenu
          :user-name="adminName"
          @profile="openProfileMenu"
          @settings="openSettingsMenu"
          @logout="logout"
        />
      </template>
    </AppTopbar>

    <div class="grid min-h-[calc(100vh-108px)] grid-cols-[248px_minmax(0,1fr)] items-start gap-3.5
max-[900px]:grid-cols-1">
      <aside class="sticky top-2 rounded-[20px] border border-transparent bg-white/80 p-3 shadow-glass
backdrop-blur-[14px] max-[900px]:static">
        <div class="mb-3 rounded-[16px] border border-transparent bg-white/85 px-3.5 py-3 shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
          <p class="text-[11px] font-semibold uppercase tracking-[0.22em] text-slate-500">Admin Panel</p>
          <h2 class="mt-1 text-lg font-bold text-slate-900">{{ adminName }}</h2>
        </div>

        <nav class="flex flex-col gap-2.5">
          <button
            v-for="item in menuItems"
            :key="item.key"
            type="button"
            class="rounded-[14px] border border-transparent px-3.5 py-3 text-left font-medium text-slate-900
backdrop-blur-[14px] shadow-glass transition duration-200"
            :class="activeTab === item.key ? 'scale-[1.02] bg-slate-900 text-white
shadow-[0_16px_24px_rgba(15,23,42,0.18)]' : 'bg-white/75 hover:-translate-y-0.5 hover:scale-[1.02]
hover:bg-white/95 hover:font-semibold hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]'"
            @click="activeTab = item.key"
          >
            {{ item.label }}
          </button>
        </nav>
      </aside>

      <main class="min-w-0 rounded-[20px] border border-transparent bg-white/80 p-3.5 shadow-glass
backdrop-blur-[14px]">
        <header class="mb-3.5 border-b border-slate-900/10 pb-3.5">
          <h1 class="m-0 text-[40px] font-bold text-slate-900 max-md:text-[22px]">{{ panelTitle }}</h1>
        </header>

        <AdminOverviewSection
          v-if="activeTab === 'overview'"
          :stats="stats"
          :pending-turfs="pendingTurfs"
          :pending-bookings="pendingBookings"
          :pending-refunds="pendingRefunds"
          :format-money="formatMoney"
        />

        <AdminVerifyTurfsSection
          v-else-if="activeTab === 'verifyTurfs'"
          :pending-turfs="pendingTurfs"
          :notes="turfNotes"
          :is-action-loading="isActionLoading"
          :format-money="formatMoney"
          @update-note="turfNotes = { ...turfNotes, [$event.id]: $event.value }"
          @review="reviewTurf($event.item, $event.action)"
        />

        <AdminBookingRequestsSection
          v-else-if="activeTab === 'bookingRequests'"
          :pending-bookings="pendingBookings"
          :is-action-loading="isActionLoading"
          :format-money="formatMoney"
          :format-date="formatDate"
```

```
          :format-time="formatTime"
          @review="reviewBooking($event.item, $event.action)"
        />

        <AdminRefundsSection
          v-else-if="activeTab === 'refunds'"
          :pending-refunds="pendingRefunds"
          :notes="refundNotes"
          :is-action-loading="isActionLoading"
          :format-money="formatMoney"
          @update-note="refundNotes = { ...refundNotes, [$event.id]: $event.value }"
          @review="reviewRefund($event.item, $event.action)"
        />

        <AdminUsersOwnersSection
          v-else-if="activeTab === 'usersOwners'"
          :monitor-users="monitorUsers"
          :monitor-owners="monitorOwners"
          :status-tone="statusTone"
          :is-action-loading="isActionLoading"
          @toggle="toggleUserStatus($event.type, $event.id, $event.action)"
        />

        <AdminAnalyticsSection
          v-else
          :analytics="analytics"
          :format-money="formatMoney"
        />

        <p
          v-if="message"
          class="mt-3 text-sm font-semibold"
          :class="messageType === 'success' ? 'text-green-700' : messageType === 'error' ? 'text-red-700' :
'text-slate-600'"
        >
          {{ message }}
        </p>
      </main>
    </div>
  </div>
</template>

<script>
import AdminAnalyticsSection from '../components/AdminAnalyticsSection.vue'
import AdminBookingRequestsSection from '../components/AdminBookingRequestsSection.vue'
import AdminOverviewSection from '../components/AdminOverviewSection.vue'
import AdminRefundsSection from '../components/AdminRefundsSection.vue'
import AdminUsersOwnersSection from '../components/AdminUsersOwnersSection.vue'
import AdminVerifyTurfsSection from '../components/AdminVerifyTurfsSection.vue'
import AppTopbar from '../components/AppTopbar.vue'
import GlassButton from '../components/GlassButton.vue'
import UserDashboardProfileMenu from '../components/UserDashboardProfileMenu.vue'
import {
  adminDashboardMethods,
  ADMIN_MENU_ITEMS,
  createAdminAnalytics,
  createAdminStats,
  getAdminSessionUser
} from '../support/adminDashboardSupport'

export default {
  name: 'AdminDashboard',
  components: {
    AdminAnalyticsSection,
    AdminBookingRequestsSection,
    AdminOverviewSection,
    AdminRefundsSection,
    AdminUsersOwnersSection,
    AdminVerifyTurfsSection,
    AppTopbar,
    GlassButton,
    UserDashboardProfileMenu
```

```
  },
  data() {
    const user = getAdminSessionUser()
    return {
      activeTab: 'overview',
      menuItems: ADMIN_MENU_ITEMS,
      message: '',
      messageType: 'info',
      actionLoadingMap: {},
      turfNotes: {},
      refundNotes: {},
      adminId: Number(localStorage.getItem('admin_id') || user.admin_id || (user.role === 'admin' ?
user.user_id : 0) || 0),
      adminName: user.full_name || localStorage.getItem('user_name') || 'Admin',
      stats: createAdminStats(),
      monitorUsers: [],
      monitorOwners: [],
      pendingTurfs: [],
      pendingBookings: [],
      pendingRefunds: [],
      analytics: createAdminAnalytics()
    }
  },
  computed: {
    panelTitle() {
      const item = this.menuItems.find((m) => m.key === this.activeTab)
      return item ? 'Admin ${item.label}' : 'Admin Dashboard'
    }
  },
  async mounted() {
    const user = getAdminSessionUser()
    const hasAdminId = Number(localStorage.getItem('admin_id') || user?.admin_id || (user?.role === 'admin' ?
user?.user_id : 0) || 0) > 0
    if (!user || user.role !== 'admin' || !hasAdminId) {
      this.$router.push('/login')
      return
    }

    await this.loadDashboard()
  },
  methods: {
    ...adminDashboardMethods,
    openProfileMenu() {
      this.activeTab = 'overview'
      window.scrollTo({ top: 0, behavior: 'smooth' })
    },
    openSettingsMenu() {
      // Demo placeholder for future settings action.
    },
    logout() {
      localStorage.clear()
      this.$router.push('/login')
    }
  }
}
</script>
```

==========
33. src/components/AdminOverviewSection.vue
==========

```
<template>
  <section class="space-y-3.5">
    <div class="grid grid-cols-4 gap-3 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <article
        v-for="item in topStats"
        :key="item.label"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
```

```
        <p class="m-0 text-sm text-slate-600">{{ item.label }}</p>
        <b class="mt-2 block text-[22px] text-slate-900">{{ item.value }}</b>
      </article>
    </div>

    <div class="grid grid-cols-4 gap-3 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <article
        v-for="item in lowerStats"
        :key="item.label"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <p class="m-0 text-sm text-slate-600">{{ item.label }}</p>
        <b class="mt-2 block text-[22px] text-slate-900">{{ item.value }}</b>
      </article>
    </div>

    <div class="grid grid-cols-3 gap-3 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Pending Turf Queue</h3>
        <p v-if="!pendingTurfs.length" class="mt-3 text-slate-500">No pending turf requests.</p>
        <ul v-else class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="item in pendingTurfs.slice(0, 5)"
            :key="item.turf_id"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>{{ item.turf_name }} ({{ item.owner_name }})</span>
            <b class="text-slate-900">{{ item.sport_type }}</b>
          </li>
        </ul>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Pending Booking Requests</h3>
        <p v-if="!pendingBookings.length" class="mt-3 text-slate-500">No pending bookings.</p>
        <ul v-else class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="item in pendingBookings.slice(0, 5)"
            :key="item.booking_id"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>#{{ item.booking_id }} - {{ item.user_name }}</span>
            <b class="text-slate-900">{{ item.turf_name }}</b>
          </li>
        </ul>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h3 class="m-0 text-[18px] font-bold text-slate-900">Pending Refund Requests</h3>
        <p v-if="!pendingRefunds.length" class="mt-3 text-slate-500">No pending refunds.</p>
        <ul v-else class="mt-3 flex list-none flex-col gap-2.5 p-0">
          <li
            v-for="item in pendingRefunds.slice(0, 5)"
            :key="item.refund_id"
            class="flex items-center justify-between gap-2 border-b border-slate-200/70 pb-2 text-sm
text-slate-700"
          >
            <span>#{{ item.refund_id }} - {{ item.user_name }}</span>
            <b class="text-slate-900">Tk {{ formatMoney(item.requested_amount) }}</b>
          </li>
        </ul>
      </article>
    </div>
  </section>
</template>
```

```
<script>
export default {
  name: 'AdminOverviewSection',
  props: {
    stats: { type: Object, required: true },
    pendingTurfs: { type: Array, default: () => [] },
    pendingBookings: { type: Array, default: () => [] },
    pendingRefunds: { type: Array, default: () => [] },
    formatMoney: { type: Function, required: true }
  },
  computed: {
    topStats() {
      return [
        { label: 'Pending Turfs', value: this.stats.pending_turfs },
        { label: 'Pending Bookings', value: this.stats.pending_bookings },
        { label: 'Pending Refunds', value: this.stats.pending_refunds },
        { label: "Today's Revenue", value: `Tk ${this.formatMoney(this.stats.today_revenue)}` }
      ]
    },
    lowerStats() {
      return [
        { label: 'Active Users', value: this.stats.users_active },
        { label: 'Banned Users', value: this.stats.users_banned },
        { label: 'Verified Owners', value: this.stats.owners_verified },
        { label: 'Suspended Owners', value: this.stats.owners_suspended }
      ]
    }
  }
}
</script>
```

==========
34. src/components/AdminRefundsSection.vue
==========

```
<template>
  <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
 transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
    <div>
      <h3 class="m-0 text-[18px] font-bold text-slate-900">Manage Refund Requests</h3>
      <p class="mt-2 text-slate-600">Review request then approve or reject.</p>
    </div>

    <p v-if="!pendingRefunds.length" class="mt-3 text-slate-500">No pending refund requests.</p>

    <div v-else class="mt-3 grid grid-cols-2 gap-3 max-[1200px]:grid-cols-1">
      <article
        v-for="item in pendingRefunds"
        :key="item.refund_id"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <h4 class="m-0 text-[18px] font-bold text-slate-900">Refund #{{ item.refund_id }} - Booking #{{
item.booking_id }}</h4>
        <div class="mt-3 space-y-1.5 text-sm text-slate-700">
          <p class="m-0"><b class="text-slate-900">User:</b> {{ item.user_name }} ({{ item.user_email }})</p>
          <p class="m-0"><b class="text-slate-900">Booking Status:</b> {{ item.booking_status }}</p>
          <p class="m-0"><b class="text-slate-900">Requested:</b> Tk {{ formatMoney(item.requested_amount)
}}</p>
        </div>

        <textarea
          :value="notes[item.refund_id] || ''"
          rows="2"
          placeholder="Admin note"
          class="mt-3 min-h-[88px] w-full rounded-[12px] border border-white/95 bg-white/90 px-3 py-2.5
text-sm text-slate-900 shadow-glass outline-none transition focus:border-blue-300"
          @input="$emit('update-note', { id: item.refund_id, value: $event.target.value })"
        />
```

```
        <div class="mt-3 flex flex-wrap gap-2">
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2 font-semibold text-white
shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200 hover:-translate-y-0.5 hover:bg-[#2f57c2]
hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('refund', item.refund_id)"
            @click="$emit('review', { item, action: 'approved' })"
          >
            Approve
          </button>
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2 font-semibold
text-[#991b1b] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('refund', item.refund_id)"
            @click="$emit('review', { item, action: 'rejected' })"
          >
            Reject
          </button>
        </div>
      </article>
    </div>
  </section>
</template>

<script>
export default {
  name: 'AdminRefundsSection',
  props: {
    pendingRefunds: { type: Array, default: () => [] },
    notes: { type: Object, required: true },
    isActionLoading: { type: Function, required: true },
    formatMoney: { type: Function, required: true }
  },
  emits: ['update-note', 'review']
}
</script>



==========================================================================================================
35. src/components/AdminUsersOwnersSection.vue
==========================================================================================================

<template>
  <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
 transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
    <div>
      <h3 class="m-0 text-[18px] font-bold text-slate-900">Monitor Users and Turf Owners</h3>
      <p class="mt-2 text-slate-600">Ban or unban in case of violation.</p>
    </div>

    <div class="mt-3 grid grid-cols-2 gap-3 max-[1200px]:grid-cols-1">
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h4 class="m-0 text-[18px] font-bold text-slate-900">Users</h4>
        <div class="mt-2 overflow-auto">
          <table class="min-w-[420px] w-full border-collapse">
            <thead>
              <tr>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">User</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Status</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Bookings</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Action</th>
              </tr>
            </thead>
```

```
            <tbody>
              <tr v-for="u in monitorUsers" :key="u.user_id">
                <td class="border-b border-slate-200/70 px-2 py-2 text-sm text-slate-700">
                  <div class="font-semibold text-slate-900">{{ u.full_name }}</div>
                  <small>{{ u.email }}</small>
                </td>
                <td class="border-b border-slate-200/70 px-2 py-2"><StatusBadge :label="u.status"
:tone="statusTone(u.status)" :bordered="false" /></td>
                <td class="border-b border-slate-200/70 px-2 py-2 text-sm text-slate-700">{{ u.total_bookings
}}</td>
                <td class="border-b border-slate-200/70 px-2 py-2">
                  <button
                    v-if="u.status !== 'banned'"
                    type="button"
                    class="rounded-[10px] border border-transparent bg-white/80 px-3 py-1.5 font-semibold
text-[#991b1b] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
                    :disabled="isActionLoading('user', u.user_id)"
                    @click="$emit('toggle', { type: 'user', id: u.user_id, action: 'ban' })"
                  >
                    Ban
                  </button>
                  <button
                    v-else
                    type="button"
                    class="rounded-[10px] border border-transparent bg-white/80 px-3 py-1.5 font-semibold
text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
                    :disabled="isActionLoading('user', u.user_id)"
                    @click="$emit('toggle', { type: 'user', id: u.user_id, action: 'unban' })"
                  >
                    Unban
                  </button>
                </td>
              </tr>
            </tbody>
          </table>
        </div>
      </article>

      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <h4 class="m-0 text-[18px] font-bold text-slate-900">Turf Owners</h4>
        <div class="mt-2 overflow-auto">
          <table class="min-w-[420px] w-full border-collapse">
            <thead>
              <tr>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Owner</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Status</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Turfs</th>
                <th class="border-b border-slate-200/80 px-2 py-2 text-left text-sm font-semibold
text-slate-900">Action</th>
              </tr>
            </thead>
            <tbody>
              <tr v-for="o in monitorOwners" :key="o.owner_id">
                <td class="border-b border-slate-200/70 px-2 py-2 text-sm text-slate-700">
                  <div class="font-semibold text-slate-900">{{ o.owner_name }}</div>
                  <small>{{ o.email }}</small>
                </td>
                <td class="border-b border-slate-200/70 px-2 py-2"><StatusBadge :label="o.status"
:tone="statusTone(o.status)" :bordered="false" /></td>
                <td class="border-b border-slate-200/70 px-2 py-2 text-sm text-slate-700">{{ o.total_turfs
}}</td>
                <td class="border-b border-slate-200/70 px-2 py-2">
                  <button
                    v-if="o.status !== 'suspended'"
                    type="button"
                    class="rounded-[10px] border border-transparent bg-white/80 px-3 py-1.5 font-semibold
```

```
text-[#991b1b] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
                    :disabled="isActionLoading('owner', o.owner_id)"
                    @click="$emit('toggle', { type: 'owner', id: o.owner_id, action: 'ban' })"
                  >
                    Suspend
                  </button>
                  <button
                    v-else
                    type="button"
                    class="rounded-[10px] border border-transparent bg-white/80 px-3 py-1.5 font-semibold
text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
                    :disabled="isActionLoading('owner', o.owner_id)"
                    @click="$emit('toggle', { type: 'owner', id: o.owner_id, action: 'unban' })"
                  >
                    Activate
                  </button>
                </td>
              </tr>
            </tbody>
          </table>
        </div>
      </article>
    </div>
  </section>
</template>

<script>
import StatusBadge from './StatusBadge.vue'

export default {
  name: 'AdminUsersOwnersSection',
  components: {
    StatusBadge
  },
  props: {
    monitorUsers: { type: Array, default: () => [] },
    monitorOwners: { type: Array, default: () => [] },
    statusTone: { type: Function, required: true },
    isActionLoading: { type: Function, required: true }
  },
  emits: ['toggle']
}
</script>



==========================================================================================================
36. src/components/AdminVerifyTurfsSection.vue
==========================================================================================================

<template>
  <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
 transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
    <div>
      <h3 class="m-0 text-[18px] font-bold text-slate-900">Verify Turf Listings</h3>
      <p class="mt-2 text-slate-600">Approve, reject, or request changes before public browse visibility.</p>
    </div>

    <p v-if="!pendingTurfs.length" class="mt-3 text-slate-500">No pending turf verification requests.</p>

    <div v-else class="mt-3 grid grid-cols-2 gap-3 max-[1200px]:grid-cols-1">
      <article
        v-for="item in pendingTurfs"
        :key="item.turf_id"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <h4 class="m-0 text-[18px] font-bold text-slate-900">{{ item.turf_name }}</h4>
        <div class="mt-3 space-y-1.5 text-sm text-slate-700">
          <p class="m-0"><b class="text-slate-900">Owner:</b> {{ item.owner_name }} ({{ item.owner_email
```

```
}})</p>
          <p class="m-0"><b class="text-slate-900">Sport:</b> {{ item.sport_type }}</p>
          <p class="m-0"><b class="text-slate-900">Price:</b> Tk {{ formatMoney(item.price_per_hour) }}/hr</p>
          <p class="m-0"><b class="text-slate-900">Location:</b> {{ item.city || item.area || item.address
}}</p>
        </div>

        <textarea
          :value="notes[item.turf_id] || ''"
          rows="2"
          placeholder="Optional note"
          class="mt-3 min-h-[88px] w-full rounded-[12px] border border-white/95 bg-white/90 px-3 py-2.5
text-sm text-slate-900 shadow-glass outline-none transition focus:border-blue-300"
          @input="$emit('update-note', { id: item.turf_id, value: $event.target.value })"
        />

        <div class="mt-3 flex flex-wrap gap-2">
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2 font-semibold text-white
shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200 hover:-translate-y-0.5 hover:bg-[#2f57c2]
hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('turf', item.turf_id)"
            @click="$emit('review', { item, action: 'approved' })"
          >
            Approve
          </button>
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2 font-semibold
text-[#854d0e] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('turf', item.turf_id)"
            @click="$emit('review', { item, action: 'requested_changes' })"
          >
            Request Changes
          </button>
          <button
            type="button"
            class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2 font-semibold
text-[#991b1b] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
            :disabled="isActionLoading('turf', item.turf_id)"
            @click="$emit('review', { item, action: 'rejected' })"
          >
            Reject
          </button>
        </div>
      </article>
    </div>
  </section>
</template>

<script>
export default {
  name: 'AdminVerifyTurfsSection',
  props: {
    pendingTurfs: { type: Array, default: () => [] },
    notes: { type: Object, required: true },
    isActionLoading: { type: Function, required: true },
    formatMoney: { type: Function, required: true }
  },
  emits: ['update-note', 'review']
}
</script>
```

==========
37. src/components/AppTopbar.vue
==========

```
<template>
  <header class="relative z-[60] flex items-center justify-between gap-3 rounded-[18px] border border-white/95
 bg-white/80 px-[18px] py-3 backdrop-blur-[14px] shadow-glass" :class="wrapperClass">
    <div class="flex items-center gap-2.5" :class="leftClass">
      <slot name="left" />
      <div class="text-[30px] font-bold tracking-[0.2px] text-slate-900 max-md:text-2xl">{{ brand }}</div>
    </div>

    <div class="flex items-center gap-2.5" :class="rightClass">
      <slot name="right" />
    </div>
  </header>
</template>

<script>
export default {
  name: 'AppTopbar',
  props: {
    brand: {
      type: String,
      default: 'TurfBooking'
    },
    wrapperClass: {
      type: [String, Array, Object],
      default: ''
    },
    leftClass: {
      type: [String, Array, Object],
      default: ''
    },
    rightClass: {
      type: [String, Array, Object],
      default: ''
    }
  }
}
</script>
```

==========
38. src/components/GlassButton.vue
==========

```
<template>
  <component
    :is="tag"
    :type="resolvedType"
    :disabled="tag === 'button' ? disabled : undefined"
    :class="buttonClass"
    v-bind="$attrs"
  >
    <slot />
  </component>
</template>

<script>
export default {
  name: 'GlassButton',
  inheritAttrs: false,
  props: {
    tag: {
      type: String,
      default: 'button'
    },
    type: {
      type: String,
      default: 'button'
    },
    disabled: {
      type: Boolean,
      default: false
```

```
    },
    subtle: {
      type: Boolean,
      default: false
    },
    block: {
      type: Boolean,
      default: false
    },
    active: {
      type: Boolean,
      default: false
    }
  },
  computed: {
    resolvedType() {
      return this.tag === 'button' ? this.type : undefined
    },
    buttonClass() {
      return [
        'rounded-full border border-transparent bg-white/80 px-3.5 py-2 font-semibold text-slate-900
backdrop-blur-[14px] shadow-glass',
        this.subtle ? '' : 'transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]',
        this.block ? 'w-full' : '',
        this.active ? 'ring-2 ring-blue-300' : '',
        this.disabled ? 'disabled:cursor-not-allowed disabled:opacity-65' : ''
      ]
    }
  }
}
</script>
```

==========
39. src/components/OwnerAddTurfsSection.vue
==========

```
<template>
  <section class="min-h-[calc(100vh-190px)] rounded-[14px] border border-transparent bg-white/80 p-3.5
backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] max-[900px]:min-h-0">
    <div class="mb-2.5">
      <h3 class="m-0 text-[22px] text-slate-900">Add Turf</h3>
      <p class="mt-2 text-slate-600">Form fields map directly to your 'turfs' table schema.</p>
    </div>

    <form class="mt-1.5" @submit.prevent="$emit('submit')">
      <div class="grid grid-cols-3 gap-2.5 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Turf Name *
          <input :value="form.turf_name" type="text" maxlength="150" required class="box-border w-full
rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="update('turf_name', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Sport Type *
          <select :value="form.sport_type" required class="box-border w-full rounded-lg border border-white/95
 bg-white/90 px-2.5 py-2.5 text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none
focus:border-blue-500/55 focus:ring-4 focus:ring-blue-500/15" @change="update('sport_type',
$event.target.value)">
            <option value="football">Football</option>
            <option value="cricket">Cricket</option>
            <option value="badminton">Badminton</option>
            <option value="basketball">Basketball</option>
            <option value="tennis">Tennis</option>
            <option value="other">Other</option>
          </select>
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
```

```
          Price Per Hour *
          <input :value="form.price_per_hour" type="number" min="1" step="0.01" required class="box-border
w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="update('price_per_hour', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Address *
          <input :value="form.address" type="text" maxlength="255" required class="box-border w-full
rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="update('address', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Area
          <input :value="form.area" type="text" maxlength="120" class="box-border w-full rounded-lg border
border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none
focus:border-blue-500/55 focus:ring-4 focus:ring-blue-500/15" @input="update('area', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          City
          <input :value="form.city" type="text" maxlength="120" class="box-border w-full rounded-lg border
border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none
focus:border-blue-500/55 focus:ring-4 focus:ring-blue-500/15" @input="update('city', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Location Text
          <input :value="form.location" type="text" maxlength="255" class="box-border w-full rounded-lg border
 border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none
 focus:border-blue-500/55 focus:ring-4 focus:ring-blue-500/15" @input="update('location',
$event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Latitude
          <input :value="form.latitude" type="number" step="0.0000001" min="-90" max="90" class="box-border
w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="update('latitude', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Longitude
          <input :value="form.longitude" type="number" step="0.0000001" min="-180" max="180" class="box-border
 w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="update('longitude', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Status
          <select :value="form.status" class="box-border w-full rounded-lg border border-white/95 bg-white/90
px-2.5 py-2.5 text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55
focus:ring-4 focus:ring-blue-500/15" @change="update('status', $event.target.value)">
            <option value="pending">Pending</option>
            <option value="active">Active</option>
            <option value="inactive">Inactive</option>
            <option value="rejected">Rejected</option>
          </select>
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Cancel Before Hours
          <input :value="form.cancel_before_hours" type="number" min="0" step="1" class="box-border w-full
rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="updateNumber('cancel_before_hours', $event.target.value)" />
        </label>
        <label class="flex flex-col gap-1.5 text-[13px] text-slate-900">
          Refund Percent
          <input :value="form.refund_percent" type="number" min="0" max="100" step="1" class="box-border
w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15" @input="updateNumber('refund_percent', $event.target.value)" />
        </label>
        <label class="col-span-3 flex flex-col gap-1.5 text-[13px] text-slate-900 max-[1200px]:col-span-2
```

```
max-[900px]:col-span-1">
          Turf Image *
          <input
            :key="fileInputKey"
            type="file"
            accept=".jpg,.jpeg,.png,.webp,image/jpeg,image/png,image/webp"
            class="box-border w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5
text-slate-900 shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none"
            @change="$emit('image-change', $event)"
          />
          <small class="text-xs text-slate-600">Allowed: JPG, PNG, WEBP (max 5MB)</small>
          <span v-if="selectedImageName" class="text-xs font-semibold text-blue-900">{{ selectedImageName
}}</span>
        </label>
      </div>

      <label class="mt-2.5 flex flex-col gap-1.5 text-[13px] text-slate-900">
        Description
        <textarea
          :value="form.description"
          rows="4"
          class="box-border w-full rounded-lg border border-white/95 bg-white/90 px-2.5 py-2.5 text-slate-900
shadow-[0_6px_14px_rgba(15,23,42,0.05)] outline-none focus:border-blue-500/55 focus:ring-4
focus:ring-blue-500/15"
          placeholder="Facilities, parking, lighting, and notes"
          @input="update('description', $event.target.value)"
        ></textarea>
      </label>

      <label class="mt-2.5 flex items-center gap-2 text-[13px] text-slate-900">
        <input :checked="form.is_featured" type="checkbox" class="h-4 w-4" @change="update('is_featured',
$event.target.checked)" />
        Featured Turf
      </label>

      <div v-if="imagePreviewUrl" class="mt-3 flex flex-wrap items-center gap-3">
        <img :src="imagePreviewUrl" alt="Turf preview" class="h-[100px] w-[130px] rounded-[10px] border
border-white/95 object-cover shadow-glass" />
        <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3 py-2
font-semibold text-slate-900 backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="$emit('clear-image')">
          Remove Image
        </button>
      </div>

      <div class="mt-3 flex justify-end gap-2">
        <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2.5
font-semibold text-slate-900 backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="$emit('reset')">Reset</button>
        <button type="submit" class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2.5
font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]
disabled:cursor-not-allowed disabled:opacity-60" :disabled="loading">
          {{ loading ? 'Saving...' : 'Save Turf' }}
        </button>
      </div>
    </form>

    <p v-if="message" class="mt-2.5 font-semibold" :class="{ 'text-green-700': messageType === 'success',
'text-red-700': messageType === 'error', 'text-blue-700': messageType === 'info' }">{{ message }}</p>
    <p v-if="lastCreatedTurfId" class="mt-2.5 font-semibold text-green-700">
      Turf created successfully. ID: <b>#{{ lastCreatedTurfId }}</b>
    </p>

    <div class="mt-4 border-t border-slate-900/10 pt-3">
      <h4 class="m-0 text-base text-slate-900">Future Buttons Zone</h4>
      <div class="mt-4 grid grid-cols-3 gap-2.5 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add My Turf Action</button>
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
```

```
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Bulk Action</button>
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Analytics</button>
      </div>
    </div>
  </section>
</template>

<script>
export default {
  name: 'OwnerAddTurfsSection',
  props: {
    form: { type: Object, required: true },
    selectedImageName: { type: String, default: '' },
    imagePreviewUrl: { type: String, default: '' },
    loading: { type: Boolean, default: false },
    message: { type: String, default: '' },
    messageType: { type: String, default: '' },
    lastCreatedTurfId: { type: [Number, null], default: null },
    fileInputKey: { type: Number, default: 0 }
  },
  emits: ['update:form', 'image-change', 'clear-image', 'reset', 'submit'],
  methods: {
    update(field, value) {
      this.$emit('update:form', { ...this.form, [field]: value })
    },
    updateNumber(field, value) {
      this.$emit('update:form', { ...this.form, [field]: Number(value) })
    }
  }
}
</script>



==========================================================================================
40. src/components/OwnerBookingClientDetailsModal.vue
==========================================================================================

<template>
  <div v-if="visible && booking" class="fixed inset-0 z-[120] flex items-center justify-center bg-slate-950/30
 p-4 backdrop-blur-sm" @click.self="$emit('close')">
    <div class="w-full max-w-[760px] rounded-[20px] border border-transparent bg-white/90 p-4
shadow-[0_24px_64px_rgba(15,23,42,0.18)] backdrop-blur-[18px]">
      <div class="flex items-start justify-between gap-3 border-b border-slate-900/10 pb-3">
        <div>
          <h3 class="m-0 text-[26px] font-bold text-slate-900">Client Details</h3>
          <p class="mt-1 text-slate-600">Booking #{{ booking.booking_id }} • {{ booking.turf_name }}</p>
        </div>
        <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="$emit('close')">Close</button>
      </div>

      <div class="mt-4 grid grid-cols-2 gap-3 max-[760px]:grid-cols-1">
        <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 shadow-glass">
          <h4 class="m-0 text-lg font-bold text-slate-900">Client Contact</h4>
          <p class="mt-3 text-sm text-slate-600">Full Name</p>
          <p class="m-0 font-semibold text-slate-900">{{ booking.client.full_name }}</p>
          <p class="mt-3 text-sm text-slate-600">Email</p>
          <p class="m-0 font-semibold text-slate-900">{{ booking.client.email }}</p>
          <p class="mt-3 text-sm text-slate-600">Phone</p>
          <p class="m-0 font-semibold text-slate-900">{{ booking.client.phone || 'No phone added' }}</p>
        </article>

        <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 shadow-glass">
          <h4 class="m-0 text-lg font-bold text-slate-900">Booking Snapshot</h4>
          <p class="mt-3 text-sm text-slate-600">Status</p>
```

```
          <p class="m-0"><span class="inline-flex rounded-full px-2.5 py-1 text-xs font-semibold"
:class="statusTone(booking.booking_status)">{{ booking.booking_status }}</span></p>
          <p class="mt-3 text-sm text-slate-600">Date</p>
          <p class="m-0 font-semibold text-slate-900">{{ formatDate(booking.slot_date) }}</p>
          <p class="mt-3 text-sm text-slate-600">Slot</p>
          <p class="m-0 font-semibold text-slate-900">{{ formatTime(booking.start_time) }} - {{
formatTime(booking.end_time) }}</p>
          <p class="mt-3 text-sm text-slate-600">Amount</p>
          <p class="m-0 font-semibold text-slate-900">Tk {{ formatMoney(booking.booked_price) }}</p>
        </article>
      </div>

      <div class="mt-4 flex flex-wrap justify-end gap-2">
        <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2.5
font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="$emit('email', booking)">Email Client</button>
        <button type="button" class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2.5
font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]"
@click="$emit('call', booking)">Call Client</button>
      </div>
    </div>
  </div>
</template>

<script>
export default {
  name: 'OwnerBookingClientDetailsModal',
  props: {
    visible: { type: Boolean, default: false },
    booking: { type: Object, default: null },
    formatDate: { type: Function, required: true },
    formatTime: { type: Function, required: true },
    formatMoney: { type: Function, required: true },
    statusTone: { type: Function, required: true }
  },
  emits: ['close', 'email', 'call']
}
</script>
```

==========
41. src/components/OwnerBookingsSection.vue
==========

```
<template>
  <section class="space-y-3.5">
    <div class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h3 class="m-0 text-[22px] font-bold text-slate-900">Client Details Access</h3>
          <p class="mt-2 text-slate-600">Open any booking and instantly see that customer?s contact details
without leaving the owner panel.</p>
        </div>
        <div class="rounded-[12px] border border-transparent bg-white/85 px-4 py-3 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Total Bookings</p>
          <p class="mt-1 text-base font-bold text-slate-900">{{ filteredBookings.length }}</p>
        </div>
      </div>

      <div class="mt-4 flex flex-wrap items-end gap-3">
        <label class="flex min-w-[280px] flex-1 flex-col gap-1.5 max-[640px]:min-w-0">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Search Booking or
Client</span>
          <input v-model.trim="search" type="text" placeholder="Booking ID, client name, email, turf..."
class="rounded-[10px] border border-transparent bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass
outline-none" />
        </label>
        <button type="button" class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2.5
font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
```

```
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]
disabled:cursor-not-allowed disabled:opacity-60" :disabled="loading" @click="loadBookings">
          {{ loading ? 'Refreshing...' : 'Refresh List' }}
        </button>
      </div>

      <p v-if="message" class="mt-3 rounded-[10px] px-3 py-2 text-sm font-semibold" :class="messageType ===
'error' ? 'bg-rose-100 text-rose-700' : 'bg-slate-100 text-slate-700'">
        {{ message }}
      </p>
    </div>

    <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div v-if="!filteredBookings.length" class="rounded-[12px] border border-dashed border-slate-300
bg-white/70 px-4 py-10 text-center text-slate-600">
        No owner bookings found for client access.
      </div>

      <div v-else class="overflow-x-auto">
        <table class="min-w-full border-separate border-spacing-y-2 text-left">
          <thead>
            <tr class="text-xs uppercase tracking-[0.18em] text-slate-500">
              <th class="px-3 py-2">Booking</th>
              <th class="px-3 py-2">Turf</th>
              <th class="px-3 py-2">Date & Slot</th>
              <th class="px-3 py-2">Client</th>
              <th class="px-3 py-2">Status</th>
              <th class="px-3 py-2">Action</th>
            </tr>
          </thead>
          <tbody>
            <tr
              v-for="booking in filteredBookings"
              :key="booking.booking_id"
              class="rounded-[12px] bg-white/75 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white/90 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
            >
              <td class="rounded-l-[12px] px-3 py-3 align-top">
                <div class="font-semibold text-slate-900">#{{ booking.booking_id }}</div>
                <p class="mt-1 text-xs text-slate-500">Tk {{ formatMoney(booking.booked_price) }}</p>
              </td>
              <td class="px-3 py-3 align-top">
                <div class="font-semibold text-slate-900">{{ booking.turf_name }}</div>
                <p class="mt-1 text-sm text-slate-600">{{ booking.sport_type || 'General' }}</p>
              </td>
              <td class="px-3 py-3 align-top">
                <div class="font-semibold text-slate-900">{{ formatDate(booking.slot_date) }}</div>
                <p class="mt-1 text-sm text-slate-600">{{ formatTime(booking.start_time) }} - {{
formatTime(booking.end_time) }}</p>
              </td>
              <td class="px-3 py-3 align-top">
                <div class="font-semibold text-slate-900">{{ booking.client.full_name }}</div>
                <p class="mt-1 text-sm text-slate-600">{{ booking.client.email }}</p>
              </td>
              <td class="px-3 py-3 align-top">
                <span class="inline-flex rounded-full px-2.5 py-1 text-xs font-semibold"
:class="statusTone(booking.booking_status)">
                  {{ booking.booking_status }}
                </span>
              </td>
              <td class="rounded-r-[12px] px-3 py-3 align-top">
                <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
text-sm font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="openClientDetails(booking)">View
Client</button>
              </td>
            </tr>
          </tbody>
        </table>
      </div>
    </section>
```

```
    <OwnerBookingClientDetailsModal
      :visible="detailsOpen"
      :booking="selectedBooking"
      :format-date="formatDate"
      :format-time="formatTime"
      :format-money="formatMoney"
      :status-tone="statusTone"
      @close="closeClientDetails"
      @email="emailClient"
      @call="callClient"
    />
  </section>
</template>

<script>
import {
  fetchOwnerBookingClients,
  filterOwnerClientBookings,
  formatOwnerClientDate,
  formatOwnerClientMoney,
  formatOwnerClientTime,
  getOwnerClientStatusTone
} from '../support/ownerBookingsSupport'
import OwnerBookingClientDetailsModal from './OwnerBookingClientDetailsModal.vue'

export default {
  name: 'OwnerBookingsSection',
  components: {
    OwnerBookingClientDetailsModal
  },
  props: {
    ownerId: { type: Number, required: true }
  },
  data() {
    return {
      loading: false,
      message: '',
      messageType: 'info',
      bookings: [],
      search: '',
      detailsOpen: false,
      selectedBooking: null
    }
  },
  computed: {
    filteredBookings() {
      return filterOwnerClientBookings(this.bookings, this.search)
    }
  },
  async mounted() {
    await this.loadBookings()
  },
  methods: {
    formatDate: formatOwnerClientDate,
    formatTime: formatOwnerClientTime,
    formatMoney: formatOwnerClientMoney,
    statusTone: getOwnerClientStatusTone,
    async loadBookings() {
      if (!this.ownerId) return
      this.loading = true
      this.message = ''
      try {
        const data = await fetchOwnerBookingClients(this.ownerId)
        if (!data?.success) {
          this.messageType = 'error'
          this.message = data?.message || 'Failed to load client details.'
          return
        }
        this.bookings = Array.isArray(data.bookings) ? data.bookings : []
      } catch (error) {
        this.messageType = 'error'
```

```
        this.message = error.response?.data?.message || 'Server connection failed while loading booking
clients.'
      } finally {
        this.loading = false
      }
    },
    openClientDetails(booking) {
      this.selectedBooking = booking
      this.detailsOpen = true
    },
    closeClientDetails() {
      this.detailsOpen = false
      this.selectedBooking = null
    },
    emailClient(booking) {
      const email = booking?.client?.email || ''
      if (!email) return
      window.location.href = 'mailto:${email}?subject=${encodeURIComponent('Booking #${booking.booking_id} -
TurfBooking')}'
    },
    callClient(booking) {
      const phone = booking?.client?.phone || ''
      if (!phone) return
      window.location.href = 'tel:${phone}'
    }
  }
}
</script>



==========================================================================================
42. src/components/OwnerDashboard.vue
==========================================================================================

<template>
  <div class="min-h-screen w-full box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)]
p-2.5 font-poppins text-[#111827]">
    <AppTopbar
      wrapper-class="mb-3 w-full px-4 py-3.5 max-md:flex-wrap"
      left-class="gap-2.5 max-md:flex-wrap"
      right-class="gap-1.5 max-md:flex-wrap"
    >
      <template #left>
        <GlassButton class="whitespace-nowrap" @click="switchToUserDashboard">Switch User
Dashboard</GlassButton>
      </template>
      <template #right>
        <UserDashboardProfileMenu
          :user-name="ownerName"
          @profile="openProfileMenu"
          @settings="openSettingsMenu"
          @logout="logout"
        />
      </template>
    </AppTopbar>

    <div class="grid min-h-[calc(100vh-108px)] grid-cols-[248px_minmax(0,1fr)] items-start gap-3.5
max-[900px]:grid-cols-1">
      <aside class="sticky top-2 rounded-[20px] border border-transparent bg-white/80 p-3 shadow-glass
backdrop-blur-[14px] max-[900px]:static">
        <div class="mb-3 rounded-[16px] border border-transparent bg-white/85 px-3.5 py-3 shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
          <p class="text-[11px] font-semibold uppercase tracking-[0.22em] text-slate-500">Owner Panel</p>
          <h2 class="mt-1 text-lg font-bold text-slate-900">{{ ownerName }}</h2>
        </div>

        <nav class="flex flex-col gap-2.5">
          <button
            v-for="item in menuItems"
            :key="item.key"
            type="button"
```

```
            class="rounded-[14px] border border-transparent px-3.5 py-3 text-left font-medium text-slate-900
backdrop-blur-[14px] shadow-glass transition duration-200"
            :class="activeTab === item.key ? 'scale-[1.02] bg-slate-900 text-white
shadow-[0_16px_24px_rgba(15,23,42,0.18)]' : 'bg-white/75 hover:-translate-y-0.5 hover:scale-[1.02]
hover:bg-white/95 hover:font-semibold hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]'"
            @click="activeTab = item.key"
          >
            {{ item.label }}
          </button>
        </nav>
      </aside>

      <main class="min-w-0 rounded-[20px] border border-transparent bg-white/80 p-3.5 shadow-glass
backdrop-blur-[14px]">
        <header class="mb-3.5 border-b border-slate-900/10 pb-3.5">
          <h1 class="m-0 text-[40px] font-bold text-slate-900 max-md:text-[22px]">{{ panelTitle }}</h1>
        </header>

        <OwnerOverviewSection
          v-if="activeTab === 'dashboard'"
          :owner-finance="ownerFinance"
          :format-money="formatMoney"
        />

        <OwnerAddTurfsSection
          v-else-if="activeTab === 'myTurfs'"
          :form="form"
          :selected-image-name="selectedImageName"
          :image-preview-url="imagePreviewUrl"
          :loading="loading"
          :message="message"
          :message-type="messageType"
          :last-created-turf-id="lastCreatedTurfId"
          :file-input-key="fileInputKey"
          @update:form="form = $event"
          @image-change="handleImageChange"
          @clear-image="clearSelectedImage"
          @reset="resetForm"
          @submit="submitTurf"
        />

        <OwnerMyTurfsSection
          v-else-if="activeTab === 'ownerTurfs'"
          :owner-id="ownerIdFromSession"
        />

        <OwnerRevenueSection
          v-else-if="activeTab === 'revenue'"
          :owner-finance="ownerFinance"
          :owner-transactions="ownerTransactions"
          :finance-loading="financeLoading"
          :format-money="formatMoney"
          @refresh="loadOwnerFinance"
        />

        <OwnerBookingsSection
          v-else-if="activeTab === 'bookings'"
          :owner-id="ownerIdFromSession"
        />

        <OwnerSlotControlSection
          v-else-if="activeTab === 'slotControl'"
          :owner-id="ownerIdFromSession"
        />

        <OwnerPromoCodesSection
          v-else-if="activeTab === 'promoCodes'"
          :owner-id="ownerIdFromSession"
          :owner-name="ownerName"
        />

        <OwnerPlaceholderSection v-else :panel-title="panelTitle" />
```

```
      </main>
    </div>
  </div>
</template>

<script>
import AppTopbar from '../components/AppTopbar.vue'
import GlassButton from '../components/GlassButton.vue'
import OwnerAddTurfsSection from '../components/OwnerAddTurfsSection.vue'
import OwnerBookingsSection from '../components/OwnerBookingsSection.vue'
import OwnerMyTurfsSection from '../components/OwnerMyTurfsSection.vue'
import OwnerOverviewSection from '../components/OwnerOverviewSection.vue'
import OwnerPlaceholderSection from '../components/OwnerPlaceholderSection.vue'
import OwnerPromoCodesSection from '../components/OwnerPromoCodesSection.vue'
import OwnerRevenueSection from '../components/OwnerRevenueSection.vue'
import OwnerSlotControlSection from '../components/OwnerSlotControlSection.vue'
import UserDashboardProfileMenu from '../components/UserDashboardProfileMenu.vue'
import {
  createOwnerFinanceSummary,
  createOwnerForm,
  getOwnerSessionUser,
  ownerDashboardMethods,
  OWNER_MENU_ITEMS
} from '../support/ownerDashboardSupport'

export default {
  name: 'OwnerDashboard',
  components: {
    AppTopbar,
    GlassButton,
    OwnerAddTurfsSection,
    OwnerBookingsSection,
    OwnerMyTurfsSection,
    OwnerOverviewSection,
    OwnerPlaceholderSection,
    OwnerPromoCodesSection,
    OwnerRevenueSection,
    OwnerSlotControlSection,
    UserDashboardProfileMenu
  },
  data() {
    const sessionUser = getOwnerSessionUser()
    return {
      activeTab: 'dashboard',
      menuItems: OWNER_MENU_ITEMS,
      loading: false,
      message: '',
      messageType: '',
      lastCreatedTurfId: null,
      selectedImageFile: null,
      selectedImageName: '',
      imagePreviewUrl: '',
      fileInputKey: 0,
      ownerName: localStorage.getItem('user_name') || sessionUser.full_name || 'Owner',
      ownerEmail: localStorage.getItem('user_email') || sessionUser.email || '',
      ownerIdFromSession: Number(localStorage.getItem('owner_id') || sessionUser.owner_id || 0),
      financeLoading: false,
      ownerFinance: createOwnerFinanceSummary(),
      ownerTransactions: [],
      form: createOwnerForm()
    }
  },
  computed: {
    panelTitle() {
      const current = this.menuItems.find((item) => item.key === this.activeTab)
      if (!current || current.key === 'dashboard') return 'Owner Dashboard Overview'
      return current.label
    }
  },
  async mounted() {
    await this.loadOwnerFinance()
  },
```

```
  methods: {
    ...ownerDashboardMethods,
    openProfileMenu() {
      this.activeTab = 'dashboard'
      window.scrollTo({ top: 0, behavior: 'smooth' })
    },
    openSettingsMenu() {
      // Demo placeholder for future settings action.
    },
    logout() {
      localStorage.clear()
      this.$router.push('/login')
    }
  },
  beforeUnmount() {
    if (this.imagePreviewUrl) {
      URL.revokeObjectURL(this.imagePreviewUrl)
    }
  }
}
</script>
```

==========
43. src/components/OwnerMyTurfsSection.vue
==========

```
<template>
  <section class="space-y-3.5">
    <div class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h3 class="m-0 text-[22px] font-bold text-slate-900">My Turfs</h3>
          <p class="mt-2 text-slate-600">See all turfs created by this owner and quickly turn them active or
inactive.</p>
        </div>
        <button
          type="button"
          class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2 font-semibold text-slate-900
 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
          :disabled="loading"
          @click="loadTurfs"
        >
          {{ loading ? 'Refreshing...' : 'Refresh List' }}
        </button>
      </div>

      <p v-if="message" class="mt-3 rounded-[10px] px-3 py-2 text-sm font-semibold" :class="messageType ===
'error' ? 'bg-rose-100 text-rose-700' : messageType === 'success' ? 'bg-emerald-100 text-emerald-700' :
'bg-slate-100 text-slate-700'">
        {{ message }}
      </p>
    </div>

    <div class="grid grid-cols-4 gap-3 max-[1100px]:grid-cols-2 max-[640px]:grid-cols-1">
      <article
        v-for="card in summaryCards"
        :key="card.label"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <p class="m-0 text-sm text-slate-600">{{ card.label }}</p>
        <strong class="mt-2 block text-[24px] leading-none text-slate-900">{{ card.value }}</strong>
      </article>
    </div>

    <div v-if="!turfs.length && !loading" class="rounded-[14px] border border-transparent bg-white/80 p-8
text-center text-slate-600 backdrop-blur-[14px] shadow-glass">
      No turfs found for this owner yet.
```

```
    </div>

    <div v-else class="grid grid-cols-2 gap-3 max-[1100px]:grid-cols-1">
      <article
        v-for="turf in turfs"
        :key="turf.turf_id"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <div class="flex items-start justify-between gap-3 max-[640px]:flex-col">
          <div>
            <h4 class="m-0 text-xl font-bold text-slate-900">{{ turf.turf_name }}</h4>
            <p class="mt-1 text-sm text-slate-600">{{ turf.sport_type }} • {{ turf.city || turf.area ||
turf.address }}</p>
          </div>
          <span class="inline-flex rounded-full px-2.5 py-1 text-xs font-semibold"
:class="statusTone(turf.status)">
            {{ turf.status }}
          </span>
        </div>

        <div class="mt-3 grid grid-cols-2 gap-2.5 text-sm text-slate-700 max-[640px]:grid-cols-1">
          <p class="m-0"><b class="text-slate-900">Price:</b> Tk {{ formatMoney(turf.price_per_hour) }}/hr</p>
          <p class="m-0"><b class="text-slate-900">Created:</b> {{ formatDate(turf.created_at) }}</p>
          <p class="m-0"><b class="text-slate-900">Refund:</b> {{ turf.refund_percent }}%</p>
          <p class="m-0"><b class="text-slate-900">Cancel Window:</b> {{ turf.cancel_before_hours }} hrs</p>
        </div>

        <p v-if="turf.status === 'pending'" class="mt-3 text-sm font-medium text-amber-700">
          This turf is waiting for admin approval, so it cannot be switched yet.
        </p>
        <p v-else-if="turf.status === 'rejected'" class="mt-3 text-sm font-medium text-rose-700">
          This turf was rejected by admin. Activate is blocked until it is approved again.
        </p>

        <div class="mt-3 flex justify-end">
          <button
            type="button"
            class="rounded-[10px] border border-transparent px-4 py-2.5 font-semibold shadow-glass transition
duration-200 hover:-translate-y-0.5 disabled:cursor-not-allowed disabled:opacity-60"
            :class="turf.status === 'active' ? 'bg-rose-100 text-rose-700 hover:bg-rose-200
hover:shadow-[0_14px_20px_rgba(190,24,93,0.14)]' : 'bg-emerald-100 text-emerald-700 hover:bg-emerald-200
hover:shadow-[0_14px_20px_rgba(22,163,74,0.14)]'"
            :disabled="loading || !canToggle(turf)"
            @click="toggleTurf(turf)"
          >
            {{ turf.status === 'active' ? 'Turn Off Turf' : 'Turn On Turf' }}
          </button>
        </div>
      </article>
    </div>
  </section>
</template>

<script>
import {
  buildOwnerTurfSummary,
  fetchOwnerTurfs,
  formatOwnerTurfDate,
  formatOwnerTurfMoney,
  getOwnerTurfStatusTone,
  toggleOwnerTurfStatus
} from '../support/ownerTurfsSupport'

export default {
  name: 'OwnerMyTurfsSection',
  props: {
    ownerId: { type: Number, required: true }
  },
  data() {
    return {
      loading: false,
```

```
      message: '',
      messageType: 'info',
      turfs: [],
      summary: {
        total: 0,
        active: 0,
        inactive: 0,
        pending: 0
      }
    }
  },
  computed: {
    summaryCards() {
      return buildOwnerTurfSummary(this.summary)
    }
  },
  async mounted() {
    await this.loadTurfs()
  },
  methods: {
    formatMoney: formatOwnerTurfMoney,
    formatDate: formatOwnerTurfDate,
    statusTone: getOwnerTurfStatusTone,
    canToggle(turf) {
      const status = String(turf?.status || '').toLowerCase()
      return status === 'active' || status === 'inactive'
    },
    async loadTurfs() {
      if (!this.ownerId) return
      this.loading = true
      this.message = ''
      try {
        const data = await fetchOwnerTurfs(this.ownerId)
        if (!data?.success) {
          this.messageType = 'error'
          this.message = data?.message || 'Failed to load owner turfs.'
          return
        }
        this.turfs = Array.isArray(data.turfs) ? data.turfs : []
        this.summary = { ...this.summary, ...(data.summary || {}) }
      } catch (error) {
        this.messageType = 'error'
        this.message = error.response?.data?.message || 'Server connection failed while loading owner turfs.'
      } finally {
        this.loading = false
      }
    },
    async toggleTurf(turf) {
      if (!this.canToggle(turf)) return
      this.loading = true
      this.message = ''
      const nextStatus = String(turf.status || '').toLowerCase() === 'active' ? 'inactive' : 'active'

      try {
        const data = await toggleOwnerTurfStatus(this.ownerId, turf.turf_id, nextStatus)
        this.messageType = data?.success ? 'success' : 'error'
        this.message = data?.message || 'Turf status updated.'
        if (data?.success) {
          await this.loadTurfs()
        }
      } catch (error) {
        this.messageType = 'error'
        this.message = error.response?.data?.message || 'Server connection failed while updating turf status.'
      } finally {
        this.loading = false
      }
    }
  }
}
</script>
```

==========
44. src/components/OwnerOverviewSection.vue
==========

```
<template>
  <section class="space-y-3.5">
    <div class="grid grid-cols-4 gap-3 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <article
        v-for="item in statCards"
        :key="item.label"
        class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
      >
        <p class="m-0 text-sm text-slate-600">{{ item.label }}</p>
        <b class="mt-2 block text-[22px] text-slate-900">{{ item.value }}</b>
      </article>
    </div>

    <section class="min-h-[calc(100vh-264px)] rounded-[14px] border border-transparent bg-white/80 p-3.5
backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] max-[900px]:min-h-0">
      <h3 class="m-0 text-[22px] font-bold text-slate-900">Module Workspace</h3>
      <p class="mt-2 text-slate-600">You can add new owner feature buttons and modules here later.</p>

      <div class="mt-4 grid grid-cols-3 gap-2.5 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:border-blue-500/70 hover:bg-white hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Feature Button</button>
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:border-blue-500/70 hover:bg-white hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Shortcut Button</button>
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:border-blue-500/70 hover:bg-white hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Report Button</button>
        <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45
bg-white/80 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:border-blue-500/70 hover:bg-white hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Custom Module</button>
      </div>
    </section>
  </section>
</template>

<script>
export default {
  name: 'OwnerOverviewSection',
  props: {
    ownerFinance: {
      type: Object,
      required: true
    },
    formatMoney: {
      type: Function,
      required: true
    }
  },
  computed: {
    statCards() {
      return [
        { label: 'Owner Wallet Balance', value: 'Tk ${this.formatMoney(this.ownerFinance.wallet_balance)}' },
        { label: 'Cancellation Earnings (20%)', value: 'Tk
${this.formatMoney(this.ownerFinance.total_cancellation_earnings)}' },
        { label: 'Active Turfs', value: this.ownerFinance.active_turfs },
        { label: 'Pending Refund Requests', value: this.ownerFinance.pending_refund_requests }
      ]
    }
  }
}
```

```
</script>


==========================================================================================================
45. src/components/OwnerPlaceholderSection.vue
==========================================================================================================

<template>
  <section class="min-h-[calc(100vh-190px)] rounded-[14px] border border-transparent bg-white/80 p-3.5
backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] max-[900px]:min-h-0">
    <h3 class="m-0 text-[22px] text-slate-900">{{ panelTitle }}</h3>
    <p class="mt-2 text-slate-600">This section is ready for your custom logic and action buttons.</p>
    <div class="mt-4 grid grid-cols-3 gap-2.5 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45 bg-white/80
 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition duration-200
hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add New Button</button>
      <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45 bg-white/80
 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition duration-200
hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Table</button>
      <button type="button" class="min-h-11 rounded-[10px] border border-dashed border-blue-500/45 bg-white/80
 p-[10px_12px] font-semibold text-slate-700 backdrop-blur-[14px] shadow-glass transition duration-200
hover:-translate-y-0.5 hover:bg-white hover:border-blue-500/70 hover:text-blue-900
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">+ Add Action Panel</button>
    </div>
  </section>
</template>

<script>
export default {
  name: 'OwnerPlaceholderSection',
  props: {
    panelTitle: {
      type: String,
      required: true
    }
  }
}
</script>



==========================================================================================================
46. src/components/OwnerPromoCodesSection.vue
==========================================================================================================

<template>
  <section class="space-y-3.5">
    <div class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h3 class="m-0 text-[22px] font-bold text-slate-900">Create Promo Codes & Offers</h3>
          <p class="mt-2 text-slate-600">Create owner-specific offers that only work on this owner?s turfs
during booking.</p>
        </div>
        <div class="rounded-[12px] border border-transparent bg-white/85 px-4 py-3 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Owner</p>
          <p class="mt-1 text-base font-bold text-slate-900">{{ ownerName }}</p>
        </div>
      </div>

      <p v-if="message" class="mt-3 rounded-[10px] px-3 py-2 text-sm font-semibold" :class="messageType ===
'error' ? 'bg-rose-100 text-rose-700' : messageType === 'success' ? 'bg-emerald-100 text-emerald-700' :
'bg-slate-100 text-slate-700'">
        {{ message }}
      </p>
    </div>
```

```
    <div class="grid grid-cols-3 gap-3 max-[900px]:grid-cols-1">
      <article v-for="card in summaryCards" :key="card.label" class="rounded-[14px] border border-transparent
bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-600">{{ card.label }}</p>
        <strong class="mt-2 block text-[24px] leading-none text-slate-900">{{ card.value }}</strong>
      </article>
    </div>

    <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <h4 class="m-0 text-[22px] font-bold text-slate-900">New Promo Code</h4>
      <div class="mt-4 grid grid-cols-3 gap-3 max-[1100px]:grid-cols-2 max-[640px]:grid-cols-1">
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Promo Code</span>
          <input v-model.trim="form.code" type="text" placeholder="AXR10" class="rounded-[10px] border
border-transparent bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass outline-none" />
        </label>
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Discount Type</span>
          <select v-model="form.discount_type" class="rounded-[10px] border border-transparent bg-white/85
px-3 py-2.5 text-sm text-slate-900 shadow-glass outline-none">
            <option value="percent">Percent</option>
            <option value="fixed">Fixed</option>
          </select>
        </label>
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Discount Value</span>
          <input v-model="form.discount_value" type="number" min="1" step="0.01" placeholder="10"
class="rounded-[10px] border border-transparent bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass
outline-none" />
        </label>
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Min Booking
Amount</span>
          <input v-model="form.min_booking_amount" type="number" min="0" step="0.01" placeholder="0"
class="rounded-[10px] border border-transparent bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass
outline-none" />
        </label>
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Start Date</span>
          <input v-model="form.start_date" type="date" class="rounded-[10px] border border-transparent
bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass outline-none" />
        </label>
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">End Date</span>
          <input v-model="form.end_date" type="date" class="rounded-[10px] border border-transparent
bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass outline-none" />
        </label>
      </div>

      <div class="mt-4 flex justify-end">
        <button type="button" class="rounded-[10px] border border-transparent bg-[#3361d8] px-4 py-2.5
font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]
disabled:cursor-not-allowed disabled:opacity-60" :disabled="loading" @click="submitPromo">
          {{ loading ? 'Saving...' : 'Create Promo Code' }}
        </button>
      </div>
    </section>

    <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h4 class="m-0 text-[22px] font-bold text-slate-900">Promo Code List</h4>
          <p class="mt-1 text-slate-600">These offers only work for turfs created by this owner.</p>
        </div>
        <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
:disabled="loading" @click="loadPromos">Refresh List</button>
```

```
      </div>

      <div v-if="!promos.length" class="mt-4 rounded-[12px] border border-dashed border-slate-300 bg-white/70
px-4 py-10 text-center text-slate-600">
        No promo codes created yet.
      </div>

      <div v-else class="mt-4 overflow-x-auto">
        <table class="min-w-full border-separate border-spacing-y-2 text-left">
          <thead>
            <tr class="text-xs uppercase tracking-[0.18em] text-slate-500">
              <th class="px-3 py-2">Code</th>
              <th class="px-3 py-2">Offer</th>
              <th class="px-3 py-2">Min Amount</th>
              <th class="px-3 py-2">Validity</th>
              <th class="px-3 py-2">Status</th>
              <th class="px-3 py-2">Action</th>
            </tr>
          </thead>
          <tbody>
            <tr v-for="promo in promos" :key="promo.promo_id" class="rounded-[12px] bg-white/75 shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:bg-white/90
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
              <td class="rounded-l-[12px] px-3 py-3 align-top font-semibold text-slate-900">{{ promo.code
}}</td>
              <td class="px-3 py-3 align-top text-slate-700">
                {{ promo.discount_type === 'percent' ? '${promo.discount_value}% off' : 'Tk
${formatMoney(promo.discount_value)} off' }}
              </td>
              <td class="px-3 py-3 align-top text-slate-700">Tk {{ formatMoney(promo.min_booking_amount)
}}</td>
              <td class="px-3 py-3 align-top text-slate-700">
                {{ formatDate(promo.start_date) }} - {{ formatDate(promo.end_date) }}
              </td>
              <td class="px-3 py-3 align-top">
                <span class="inline-flex rounded-full px-2.5 py-1 text-xs font-semibold"
:class="promo.is_active ? 'bg-emerald-100 text-emerald-800' : 'bg-slate-200 text-slate-700'">
                  {{ promo.is_active ? 'Active' : 'Inactive' }}
                </span>
              </td>
              <td class="rounded-r-[12px] px-3 py-3 align-top">
                <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3 py-1.5
text-sm font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
 :disabled="loading" @click="togglePromo(promo)">
                  {{ promo.is_active ? 'Deactivate' : 'Activate' }}
                </button>
              </td>
            </tr>
          </tbody>
        </table>
      </div>
    </section>
  </section>
</template>

<script>
import {
  buildOwnerPromoSummary,
  createOwnerPromoCode,
  createOwnerPromoForm,
  fetchOwnerPromoCodes,
  formatOwnerPromoDate,
  formatOwnerPromoMoney,
  toggleOwnerPromoCode
} from '../support/ownerPromoCodesSupport'

export default {
  name: 'OwnerPromoCodesSection',
  props: {
    ownerId: { type: Number, required: true },
    ownerName: { type: String, default: 'Owner' }
```

```
},
data() {
  return {
    loading: false,
    message: '',
    messageType: 'info',
    form: createOwnerPromoForm(),
    promos: [],
    summary: {
      total_codes: 0,
      active_codes: 0,
      inactive_codes: 0
    }
  }
},
computed: {
  summaryCards() {
    return buildOwnerPromoSummary(this.summary)
  }
},
async mounted() {
  await this.loadPromos()
},
methods: {
  formatMoney: formatOwnerPromoMoney,
  formatDate: formatOwnerPromoDate,
  async loadPromos() {
    if (!this.ownerId) return
    this.loading = true
    this.message = ''
    try {
      const data = await fetchOwnerPromoCodes(this.ownerId)
      if (!data?.success) {
        this.messageType = 'error'
        this.message = data?.message || 'Failed to load promo codes.'
        return
      }
      this.promos = Array.isArray(data.promos) ? data.promos : []
      this.summary = { ...this.summary, ...(data.summary || {}) }
    } catch (error) {
      this.messageType = 'error'
      this.message = error.response?.data?.message || 'Server connection failed while loading promo codes.'
    } finally {
      this.loading = false
    }
  },
  async submitPromo() {
    if (!this.ownerId) return
    this.loading = true
    this.message = ''
    try {
      const data = await createOwnerPromoCode(this.ownerId, this.form)
      this.messageType = data?.success ? 'success' : 'error'
      this.message = data?.message || 'Promo code request finished.'
      if (data?.success) {
        this.form = createOwnerPromoForm()
        await this.loadPromos()
      }
    } catch (error) {
      this.messageType = 'error'
      this.message = error.response?.data?.message || 'Server connection failed while creating promo code.'
    } finally {
      this.loading = false
    }
  },
  async togglePromo(promo) {
    this.loading = true
    this.message = ''
    try {
      const data = await toggleOwnerPromoCode(this.ownerId, promo.promo_id, !promo.is_active)
      this.messageType = data?.success ? 'success' : 'error'
      this.message = data?.message || 'Promo code updated.'
```

```
        await this.loadPromos()
      } catch (error) {
        this.messageType = 'error'
        this.message = error.response?.data?.message || 'Server connection failed while updating promo code.'
      } finally {
        this.loading = false
      }
    }
  }
}
</script>
```

==========
47. src/components/OwnerRevenueSection.vue
==========

```
<template>
  <section class="min-h-[calc(100vh-190px)] rounded-[14px] border border-transparent bg-white/80 p-3.5
backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] max-[900px]:min-h-0">
    <div class="mb-2.5">
      <h3 class="m-0 text-[22px] text-slate-900">Owner Wallet & Cancellation Earnings</h3>
      <p class="mt-2 text-slate-600">Admin approved cancellation gives 20% share to owner wallet.</p>
    </div>

    <div class="mt-2.5 grid grid-cols-4 gap-3 max-[1200px]:grid-cols-2 max-[900px]:grid-cols-1">
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-900">Current Wallet Balance: <b>Tk {{
formatMoney(ownerFinance.wallet_balance) }}</b></p>
      </article>
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-900">Total Cancellation Earnings: <b>Tk {{
formatMoney(ownerFinance.total_cancellation_earnings) }}</b></p>
      </article>
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-900">Cancelled Bookings: <b>{{ ownerFinance.cancelled_bookings
}}</b></p>
      </article>
      <article class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-900">Pending Refund Requests: <b>{{
ownerFinance.pending_refund_requests }}</b></p>
      </article>
    </div>

    <div class="mt-4 border-t border-slate-900/10 pt-3">
      <h4 class="m-0 text-base text-slate-900">Recent Wallet Transactions</h4>
      <p v-if="financeLoading" class="mt-2 text-slate-600">Loading wallet data...</p>
      <p v-else-if="!ownerTransactions.length" class="mt-2 text-slate-600">No wallet transactions yet.</p>
      <div v-else class="mt-2.5 flex flex-col gap-2">
        <div v-for="txn in ownerTransactions" :key="txn.txn_id" class="flex items-center justify-between gap-3
 rounded-[10px] border border-white/95 bg-white/85 p-[10px_12px] shadow-[0_6px_14px_rgba(15,23,42,0.07)]">
          <span class="text-[13px] text-slate-700">{{ txn.reference_note || txn.txn_type }}</span>
          <b>Tk {{ formatMoney(txn.amount) }}</b>
        </div>
      </div>
      <div class="mt-3 flex justify-end gap-2">
        <button
          type="button"
          class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2.5 font-semibold
text-slate-900 backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
          @click="$emit('refresh')"
          :disabled="financeLoading"
        >
          {{ financeLoading ? 'Refreshing...' : 'Refresh Wallet' }}
        </button>
```

```
      </div>
    </div>
  </section>
</template>

<script>
export default {
  name: 'OwnerRevenueSection',
  props: {
    ownerFinance: { type: Object, required: true },
    ownerTransactions: { type: Array, default: () => [] },
    financeLoading: { type: Boolean, default: false },
    formatMoney: { type: Function, required: true }
  },
  emits: ['refresh']
}
</script>
```

==========
48. src/components/OwnerSlotControlSection.vue
==========

```
<template>
  <section class="space-y-3.5">
    <div class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h3 class="m-0 text-[22px] font-bold text-slate-900">Control Slot Availability</h3>
          <p class="mt-2 text-slate-600">Choose a turf and day, then turn selected slots on or off without
breaking booked reservations.</p>
        </div>
        <div class="rounded-[12px] border border-transparent bg-white/85 px-4 py-3 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Selected Day</p>
          <p class="mt-1 text-base font-bold text-slate-900">{{ formatDate(filters.slotDate) }}</p>
        </div>
      </div>

      <div class="mt-4 grid grid-cols-[minmax(0,1.2fr)_200px_auto] gap-3 max-[1100px]:grid-cols-1">
        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Select Turf</span>
          <select v-model="filters.turfId" class="rounded-[10px] border border-transparent bg-white/85 px-3
py-2.5 text-sm text-slate-900 shadow-glass outline-none">
            <option value="">Choose a turf</option>
            <option v-for="turf in turfs" :key="turf.turf_id" :value="String(turf.turf_id)">
              {{ turf.turf_name }}
            </option>
          </select>
        </label>

        <label class="flex flex-col gap-1.5">
          <span class="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Select Day</span>
          <input v-model="filters.slotDate" type="date" class="rounded-[10px] border border-transparent
bg-white/85 px-3 py-2.5 text-sm text-slate-900 shadow-glass outline-none" />
        </label>

        <div class="flex items-end">
          <button type="button" class="w-full rounded-[10px] border border-transparent bg-[#3361d8] px-3.5
py-2.5 font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]
disabled:cursor-not-allowed disabled:opacity-60" :disabled="loading || !filters.turfId" @click="loadData">
            {{ loading ? 'Loading...' : 'Load Slots' }}
          </button>
        </div>
      </div>

      <p v-if="message" class="mt-3 rounded-[10px] px-3 py-2 text-sm font-semibold" :class="messageType ===
'error' ? 'bg-rose-100 text-rose-700' : messageType === 'success' ? 'bg-emerald-100 text-emerald-700' :
'bg-slate-100 text-slate-700'">
        {{ message }}
```

```
      </p>
    </div>

    <div class="grid grid-cols-3 gap-3 max-[1200px]:grid-cols-2 max-[640px]:grid-cols-1">
      <article v-for="card in summaryCards" :key="card.label" class="rounded-[14px] border border-transparent
bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
        <p class="m-0 text-sm text-slate-600">{{ card.label }}</p>
        <strong class="mt-2 block text-[24px] leading-none text-slate-900">{{ card.value }}</strong>
      </article>
    </div>

    <section class="rounded-[14px] border border-transparent bg-white/80 p-3.5 backdrop-blur-[14px]
shadow-glass transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
      <div class="flex items-start justify-between gap-3 max-[900px]:flex-col">
        <div>
          <h4 class="m-0 text-[22px] font-bold text-slate-900">Slot Grid</h4>
          <p class="mt-1 text-slate-600">Select free slots and switch them on or off for the chosen day.</p>
        </div>
        <div class="flex flex-wrap gap-2">
          <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
text-sm font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="selectAllAvailable">Select
Available</button>
          <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
text-sm font-semibold text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]" @click="clearSelection">Clear</button>
          <button type="button" class="rounded-[10px] border border-transparent bg-white/80 px-3.5 py-2
text-sm font-semibold text-rose-700 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)] disabled:cursor-not-allowed disabled:opacity-60"
:disabled="actionLoading || !selectedSlotIds.length" @click="updateAvailability(0)">Turn Off Selected</button>
          <button type="button" class="rounded-[10px] border border-transparent bg-[#3361d8] px-3.5 py-2
text-sm font-semibold text-white shadow-[0_10px_20px_rgba(51,97,216,0.3)] transition duration-200
hover:-translate-y-0.5 hover:bg-[#2f57c2] hover:shadow-[0_14px_20px_rgba(51,97,216,0.28)]
disabled:cursor-not-allowed disabled:opacity-60" :disabled="actionLoading || !selectedSlotIds.length"
@click="updateAvailability(1)">Turn On Selected</button>
        </div>
      </div>

      <div v-if="!slots.length" class="mt-4 rounded-[12px] border border-dashed border-slate-300 bg-white/70
px-4 py-10 text-center text-slate-600">
        Choose a turf and date to load slots.
      </div>

      <div v-else class="mt-4 grid grid-cols-4 gap-3 max-[1200px]:grid-cols-3 max-[900px]:grid-cols-2
max-[640px]:grid-cols-1">
        <label
          v-for="slot in slots"
          :key="slot.slot_id"
          class="flex cursor-pointer items-start gap-3 rounded-[14px] border border-transparent bg-white/80
p-3.5 shadow-glass transition duration-200 hover:-translate-y-0.5 hover:bg-white
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
          :class="slot.is_booked ? 'opacity-80' : ''"
        >
          <input
            :checked="selectedSlotIds.includes(slot.slot_id)"
            type="checkbox"
            class="mt-1 h-4 w-4 accent-[#3361d8]"
            :disabled="slot.is_booked"
            @change="toggleSlot(slot.slot_id, $event.target.checked)"
          />
          <div class="min-w-0 flex-1">
            <div class="flex items-start justify-between gap-3">
              <strong class="text-slate-900">{{ formatTime(slot.start_time) }} - {{ formatTime(slot.end_time)
}}</strong>
              <span class="inline-flex rounded-full px-2.5 py-1 text-xs font-semibold"
:class="statusTone(slot)">
                {{ slot.is_booked ? 'Booked' : slot.is_enabled ? 'Enabled' : 'Disabled' }}
              </span>
            </div>
            <p class="mt-2 text-sm text-slate-600">Tk {{ formatMoney(slot.base_price) }}</p>
            <p class="mt-1 text-xs text-slate-500">
```

```
              {{ slot.is_booked ? ‘Locked by ${slot.booking_status || ’booking’}‘ : ’Available for owner
control’ }}
            </p>
          </div>
        </label>
      </div>
    </section>
  </section>
</template>

<script>
import {
  createOwnerSlotControlState,
  fetchOwnerSlotControl,
  formatOwnerSlotDate,
  formatOwnerSlotMoney,
  formatOwnerSlotTime,
  getOwnerSlotStatusTone,
  updateOwnerSlotAvailability
} from ’../support/ownerSlotControlSupport’

export default {
  name: ’OwnerSlotControlSection’,
  props: {
    ownerId: {
      type: Number,
      required: true
    }
  },
  data() {
    return {
      loading: false,
      actionLoading: false,
      message: ’’,
      messageType: ’info’,
      filters: createOwnerSlotControlState(),
      turfs: [],
      slots: [],
      selectedSlotIds: []
    }
  },
  computed: {
    summaryCards() {
      const total = this.slots.length
      const enabled = this.slots.filter((slot) => slot.is_enabled && !slot.is_booked).length
      const disabled = this.slots.filter((slot) => !slot.is_enabled && !slot.is_booked).length
      return [
        { label: ’Total Slots’, value: total },
        { label: ’Enabled’, value: enabled },
        { label: ’Disabled’, value: disabled }
      ]
    }
  },
  async mounted() {
    await this.loadData()
  },
  methods: {
    formatDate: formatOwnerSlotDate,
    formatTime: formatOwnerSlotTime,
    formatMoney: formatOwnerSlotMoney,
    statusTone: getOwnerSlotStatusTone,
    async loadData() {
      if (!this.ownerId) return
      this.loading = true
      this.message = ’’
      try {
        const data = await fetchOwnerSlotControl(this.ownerId, this.filters.turfId, this.filters.slotDate)
        if (!data?.success) {
          this.messageType = ’error’
          this.message = data?.message || ’Failed to load owner slot data.’
          return
        }
```

```
        this.turfs = Array.isArray(data.turfs) ? data.turfs : []
        if (!this.filters.turfId && this.turfs.length) {
          this.filters.turfId = String(this.turfs[0].turf_id)
          const retry = await fetchOwnerSlotControl(this.ownerId, this.filters.turfId, this.filters.slotDate)
          this.slots = Array.isArray(retry?.slots) ? retry.slots : []
        } else {
          this.slots = Array.isArray(data.slots) ? data.slots : []
        }
        this.selectedSlotIds = []
      } catch (error) {
        this.messageType = 'error'
        this.message = error.response?.data?.message || 'Server connection failed while loading slots.'
      } finally {
        this.loading = false
      }
    },
    toggleSlot(slotId, checked) {
      if (checked) {
        this.selectedSlotIds = Array.from(new Set([...this.selectedSlotIds, slotId]))
        return
      }
      this.selectedSlotIds = this.selectedSlotIds.filter((id) => id !== slotId)
    },
    selectAllAvailable() {
      this.selectedSlotIds = this.slots.filter((slot) => !slot.is_booked).map((slot) => slot.slot_id)
    },
    clearSelection() {
      this.selectedSlotIds = []
    },
    async updateAvailability(nextState) {
      if (!this.ownerId || !this.filters.turfId || !this.selectedSlotIds.length) return
      this.actionLoading = true
      this.message = ''
      try {
        const data = await updateOwnerSlotAvailability(
          this.ownerId,
          Number(this.filters.turfId),
          this.filters.slotDate,
          this.selectedSlotIds,
          nextState
        )
        this.messageType = data?.success ? 'success' : 'error'
        this.message = data?.message || 'Slot availability updated.'
        if (typeof data?.skipped_booked === 'number' && data.skipped_booked > 0) {
          this.message += ` ${data.skipped_booked} booked slot(s) were skipped.`
        }
        await this.loadData()
      } catch (error) {
        this.messageType = 'error'
        this.message = error.response?.data?.message || 'Server connection failed while updating slot
availability.'
      } finally {
        this.actionLoading = false
      }
    }
  }
}
</script>
```

==========
49. src/components/StatCard.vue
==========

```
<template>
  <component
    :is="tag"
    :type="tag === 'button' ? 'button' : undefined"
    class="appearance-none rounded-[14px] border border-transparent bg-white/80 p-3.5 text-left text-slate-900
 backdrop-blur-[14px] shadow-glass outline-none transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
```

```
    :class="customClass"
    v-bind="$attrs"
  >
    <h3 class="m-0 text-[24px] font-semibold tracking-[-0.03em] text-slate-900">{{ title }}</h3>
    <p class="mt-2 text-[52px] font-bold leading-none tracking-[-0.04em] text-slate-900">{{ value }}</p>
    <slot />
  </component>
</template>

<script>
export default {
  name: 'StatCard',
  inheritAttrs: false,
  props: {
    tag: {
      type: String,
      default: 'button'
    },
    title: {
      type: String,
      default: ''
    },
    value: {
      type: [String, Number],
      default: ''
    },
    customClass: {
      type: [String, Array, Object],
      default: ''
    }
  }
}
</script>
```

==========
50. src/components/StatusBadge.vue
==========

```
<template>
  <span :class="badgeClass">
    <slot>{{ label }}</slot>
  </span>
</template>

<script>
export default {
  name: 'StatusBadge',
  props: {
    label: {
      type: String,
      default: ''
    },
    tone: {
      type: String,
      default: 'neutral'
    },
    bordered: {
      type: Boolean,
      default: true
    }
  },
  computed: {
    badgeClass() {
      const toneMap = {
        success: 'bg-green-200/90 text-green-800',
        warning: 'bg-yellow-100/90 text-yellow-800',
        danger: 'bg-red-100/90 text-red-800',
        info: 'bg-blue-200/90 text-blue-800',
        neutral: 'bg-slate-100/90 text-slate-700'
      }
```

```
      return [
        'inline-block rounded-full px-2.5 py-1 text-xs font-bold capitalize',
        this.bordered ? 'border border-white/75' : '',
        toneMap[this.tone] || toneMap.neutral
      ]
    }
  }
}
</script>



==========
51. src/components/UserActivityCard.vue
==========

<template>
  <article class="flex min-h-[420px] flex-col overflow-visible rounded-2xl border border-slate-100 bg-white
p-[32px_14px_18px] text-slate-900 shadow-[0_8px_20px_rgba(0,0,0,0.08)] transition duration-200
hover:-translate-y-0.5 hover:shadow-[0_12px_22px_rgba(0,0,0,0.1)]">
    <h3 class="mb-4 px-1 text-[24px] font-semibold tracking-[-0.03em] text-[#141b33]">Recent Activity</h3>

    <div v-if="activities.length" class="flex flex-1 flex-col gap-1.5 overflow-visible">
      <div v-for="(a, i) in activities" :key="i" class="relative overflow-visible rounded-2xl border
border-slate-100 bg-white p-[14px_14px_12px] shadow-[0_8px_20px_rgba(0,0,0,0.06)]">
        <div class="flex items-start justify-between gap-2.5">
          <div class="flex min-w-0 flex-1 items-start gap-3">
            <span class="mt-0.5 flex h-11 w-11 shrink-0 items-center justify-center rounded-full
bg-slate-50/90 shadow-[0_8px_16px_rgba(15,23,42,0.08)]">
              <img :src="getActivityIcon(a)" alt="" class="h-7 w-7 object-contain" aria-hidden="true" />
            </span>
            <div class="min-w-0 flex-1">
              <div class="text-[17px] font-semibold text-[#1c2337]">{{ a.message }}</div>
              <div class="mt-1 text-[13px] text-[#4c566d]">{{ formatDateTime(a.time) }}</div>
            </div>
          </div>
          <div v-if="a.detail_message" class="group relative ml-1 mt-0.5 shrink-0">
            <span class="inline-flex h-6 w-6 items-center justify-center rounded-full border border-slate-100
bg-white text-[11px] leading-none text-slate-900 shadow-[0_6px_14px_rgba(0,0,0,0.06)]" aria-label="Admin
note">&#9993;</span>
            <div class="pointer-events-none invisible absolute right-0 top-[calc(100%+6px)] z-[120]
w-[min(320px,75vw)] translate-y-1 rounded-[10px] border border-slate-100 bg-white px-2.5 py-2 text-xs
leading-[1.45] text-[#111827] opacity-0 shadow-[0_8px_20px_rgba(0,0,0,0.08)] transition duration-200
whitespace-normal break-words group-hover:pointer-events-auto group-hover:visible group-hover:translate-y-0
group-hover:opacity-100">
              {{ a.detail_message }}
            </div>
          </div>
        </div>
      </div>
    </div>
    <div v-else class="flex-1 px-1 text-[17px] text-[#30384d]">No activity yet</div>

    <div class="mt-3 flex flex-wrap gap-1.5">
      <button type="button" class="appearance-none rounded-full border border-transparent bg-white/85 px-4
py-3 font-semibold text-slate-900 shadow-glass outline-none backdrop-blur-[14px] transition duration-200
hover:-translate-y-0.5 hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
@click="$emit('refresh')">Refresh Activity</button>
    </div>
  </article>
</template>

<script>
import pendingIcon from '../assets/pending.svg'
import confirmedIcon from '../assets/confirmed.svg'
import disabledIcon from '../assets/disabled.svg'
import cancelledIcon from '../assets/cancelled.svg'

export default {
  name: 'UserActivityCard',
  props: {
```

```
    activities: {
      type: Array,
      default: () => []
    },
    formatDateTime: {
      type: Function,
      required: true
    }
  },
  emits: ['refresh'],
  data() {
    return {
      pendingIcon,
      confirmedIcon,
      disabledIcon,
      cancelledIcon
    }
  },
  methods: {
    getActivityIcon(activity) {
      const text = '${activity?.message || ''} ${activity?.detail_message || ''}'.toLowerCase()
      if (text.includes('approved') || text.includes('confirmed') || text.includes('success') ||
text.includes('refund approved')) {
        return this.confirmedIcon
      }
      if (text.includes('cancel') || text.includes('rejected') || text.includes('failed') ||
text.includes('declined')) {
        return this.cancelledIcon
      }
      if (text.includes('disabled') || text.includes('unavailable') || text.includes('expired') ||
text.includes('blocked')) {
        return this.disabledIcon
      }
      return this.pendingIcon
    }
  }
}
</script>
```

==========
52. src/components/UserBrowseTurfCard.vue
==========

```
<template>
  <article class="overflow-hidden rounded-2xl border border-white/95 bg-white/80 backdrop-blur-[14px]
shadow-glass">
    <div class="relative h-[170px] w-full bg-slate-200">
      <img v-if="turf.image_full_url" :src="turf.image_full_url" :alt="turf.turf_name" class="h-full w-full
object-cover" />
      <div v-else class="grid h-full w-full place-items-center font-semibold text-slate-600">No Image</div>

      <span v-if="turf.is_new" class="absolute left-2 top-2 rounded-full bg-green-500/90 px-2 py-1 text-xs
font-bold text-green-950">New Turf</span>
    </div>

    <div class="p-3.5">
      <h3 class="mb-2 text-[22px] font-semibold">{{ turf.turf_name }}</h3>
      <p class="my-1 text-slate-700"><b>Sport:</b> {{ turf.sport_type }}</p>
      <p class="my-1 text-slate-700"><b>Location:</b> {{ turf.city || turf.area || turf.location ||
turf.address }}</p>
      <p class="my-1 text-slate-700"><b>Price:</b> Tk {{ Number(turf.price_per_hour || 0).toFixed(2) }}/hr</p>

      <div class="mt-3 flex flex-col gap-2.5">
        <div class="text-sm font-bold text-slate-900">Book Slot</div>

        <div class="grid grid-cols-2 gap-2 max-[760px]:grid-cols-1">
          <label class="flex flex-col gap-1 text-xs text-slate-700">
            Select Date
            <input
              type="date"
```

```
              :min="todayDate"
              :value="turf.booking_date"
              class="rounded-lg border border-white/95 bg-white/80 px-2 py-2 text-slate-900
backdrop-blur-[14px] shadow-glass"
              @input="emitUpdate('booking_date', $event.target.value)"
              @change="$emit('date-change', turf)"
            />
          </label>

          <label class="flex flex-col gap-1 text-xs text-slate-700">
            Select Available Slot
            <select
              :value="turf.selected_slot_id"
              class="rounded-lg border border-white/95 bg-white/80 px-2 py-2 text-slate-900
backdrop-blur-[14px] shadow-glass"
              :disabled="slotLoading || !turf.available_slots.length"
              @change="onSlotSelect($event)"
            >
              <option :value="0">{{ slotLoading ? 'Loading...' : 'Select slot' }}</option>
              <option v-for="slot in turf.available_slots" :key="slot.slot_id" :value="slot.slot_id">
                {{ formatSlotLabel(slot) }}
              </option>
            </select>
          </label>
        </div>

        <p class="m-0 text-xs font-semibold text-blue-800">Available Slots: {{ turf.available_slots.length
}}</p>
        <div v-if="selectedSlot" class="rounded-xl border border-slate-200/80 bg-white/75 p-2.5">
          <h4 class="mb-2 text-sm font-semibold text-slate-900">Confirm Booking</h4>

          <div class="grid grid-cols-3 gap-1.5 max-[1100px]:grid-cols-1">
            <p class="m-0 text-xs text-slate-700"><b>Date:</b> {{ turf.booking_date }}</p>
            <p class="m-0 text-xs text-slate-700"><b>Time:</b> {{ formatSlotLabel(selectedSlot) }}</p>
            <p class="m-0 text-xs text-slate-700"><b>Price/hr:</b> Tk {{ Number(turf.price_per_hour ||
0).toFixed(2) }}</p>
          </div>

        <div v-if="priceBreakdown" class="mt-2 flex flex-col gap-1 border-t border-slate-200/90 pt-2">
            <div class="flex items-center justify-between text-[13px] text-slate-700">
              <span>Duration</span>
              <b>{{ formatDuration(priceBreakdown.durationHours) }}</b>
            </div>
            <div class="flex items-center justify-between text-[13px] text-slate-700">
              <span>Subtotal</span>
              <b>Tk {{ formatMoney(priceBreakdown.subtotal) }}</b>
            </div>
            <div v-if="priceBreakdown.promoCode" class="flex items-center justify-between text-[13px]
text-emerald-700">
              <span>Promo ({{ priceBreakdown.promoCode }})</span>
              <b>- Tk {{ formatMoney(priceBreakdown.discountAmount) }}</b>
            </div>
            <div class="flex items-center justify-between text-[13px] text-slate-700">
              <span>Service Fee</span>
              <b>Tk {{ formatMoney(priceBreakdown.serviceFee) }}</b>
            </div>
            <div class="mt-0.5 flex items-center justify-between text-[13px] font-bold text-slate-900">
              <span>Total</span>
              <b>Tk {{ formatMoney(priceBreakdown.total) }}</b>
            </div>
          </div>

          <div class="mt-2 grid grid-cols-[minmax(0,1fr)_auto] gap-2 max-[760px]:grid-cols-1">
            <label class="flex flex-col gap-1 text-xs text-slate-700">
              Promo Code
              <input
                :value="turf.promo_code"
                type="text"
                class="rounded-lg border border-white/95 bg-white/80 px-2 py-2 text-slate-900
backdrop-blur-[14px] shadow-glass"
                placeholder="Enter promo code"
                @input="emitUpdate('promo_code', $event.target.value.toUpperCase())"
```

```
              />
            </label>

            <div class="flex items-end">
              <GlassButton
                class="w-full rounded-[10px] px-3 py-[9px] font-bold"
                :disabled="bookingLoading || promoLoading"
                @click="$emit(’apply-promo’, turf)"
              >
                {{ promoLoading ? ’Applying...’ : ’Apply Promo’ }}
              </GlassButton>
            </div>
          </div>

          <label class="mt-2 flex flex-col gap-1 text-xs text-slate-700">
            Payment Method
            <select
              :value="turf.payment_method"
              class="rounded-lg border border-white/95 bg-white/80 px-2 py-2 text-slate-900
backdrop-blur-[14px] shadow-glass"
              @change="emitUpdate(’payment_method’, $event.target.value)"
            >
              <option value="cash">Cash</option>
              <option value="bkash">bKash</option>
              <option value="nagad">Nagad</option>
              <option value="card">Card</option>
            </select>
          </label>

          <GlassButton
            class="mt-2 w-full rounded-[10px] px-3 py-[9px] font-bold"
            :disabled="bookingLoading"
            @click="$emit(’book’, turf)"
          >
            {{ bookingLoading ? ’Booking...’ : ’Confirm Booking’ }}
          </GlassButton>
        </div>
      </div>

      <p v-if="turf.message" class="mt-2.5 text-sm font-semibold"
:class="statusTextClass(turf.messageType)">{{ turf.message }}</p>
    </div>
  </article>
</template>

<script>
import GlassButton from ’./GlassButton.vue’

export default {
  name: ’UserBrowseTurfCard’,
  components: {
    GlassButton
  },
  props: {
    turf: {
      type: Object,
      required: true
    },
    todayDate: {
      type: String,
      required: true
    },
    slotLoading: {
      type: Boolean,
      default: false
    },
    bookingLoading: {
      type: Boolean,
      default: false
    },
    promoLoading: {
      type: Boolean,
```

```
      default: false
    },
    selectedSlot: {
      type: Object,
      default: null
    },
    priceBreakdown: {
      type: Object,
      default: null
    },
    formatSlotLabel: {
      type: Function,
      required: true
    },
    formatDuration: {
      type: Function,
      required: true
    },
    formatMoney: {
      type: Function,
      required: true
    },
    statusTextClass: {
      type: Function,
      required: true
    }
  },
  emits: ['update-field', 'date-change', 'slot-change', 'book', 'apply-promo'],
  methods: {
    emitUpdate(field, value) {
      this.$emit('update-field', this.turf, field, value)
    },
    onSlotSelect(event) {
      this.emitUpdate('selected_slot_id', Number(event.target.value || 0))
      this.$emit('slot-change', this.turf)
    }
  }
}
</script>



==========================================================================================================
53. src/components/UserBrowseTurfs.vue
==========================================================================================================

<template>
  <div class="min-h-screen w-full box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)]
p-2.5 font-poppins text-[#111827]">
    <AppTopbar wrapper-class="max-[760px]:flex-wrap">
      <template #right>
        <div class="font-semibold">Browse Turfs</div>
        <GlassButton @click="goBack">Back</GlassButton>
      </template>
    </AppTopbar>

    <main class="mt-3 rounded-[20px] border border-white/95 bg-white/80 p-4 backdrop-blur-[14px]
shadow-glass">
      <div class="flex items-center justify-between gap-2.5 max-[760px]:flex-wrap">
        <h1 class="m-0 text-[34px] font-bold">Available Turfs</h1>
        <GlassButton @click="loadTurfs" :disabled="loading">
          {{ loading ? 'Loading...' : 'Refresh' }}
        </GlassButton>
      </div>

      <div class="mt-3 grid grid-cols-[2fr_repeat(5,minmax(0,1fr))] gap-2 rounded-[14px] border
border-white/95 bg-white/80 p-2.5 backdrop-blur-[14px] shadow-glass max-[1100px]:grid-cols-3
max-[760px]:grid-cols-1">
        <input
          v-model.trim="searchText"
          type="text"
          class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px] text-slate-900
```

```
backdrop-blur-[14px] shadow-glass"
          placeholder="Search turf name..."
        />

        <select v-model="sportFilter" class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px]
 text-slate-900 backdrop-blur-[14px] shadow-glass">
          <option value="all">All Sports</option>
          <option v-for="sport in availableSports" :key="sport" :value="sport">
            {{ sport }}
          </option>
        </select>

        <input
          v-model.trim="locationFilter"
          type="text"
          class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px] text-slate-900
backdrop-blur-[14px] shadow-glass"
          placeholder="Filter by location"
        />

        <input
          v-model.number="minPrice"
          type="number"
          min="0"
          step="1"
          class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px] text-slate-900
backdrop-blur-[14px] shadow-glass"
          placeholder="Min price"
        />

        <input
          v-model.number="maxPrice"
          type="number"
          min="0"
          step="1"
          class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px] text-slate-900
backdrop-blur-[14px] shadow-glass"
          placeholder="Max price"
        />

        <select v-model="sortBy" class="rounded-[10px] border border-white/95 bg-white/80 px-2.5 py-[9px]
text-slate-900 backdrop-blur-[14px] shadow-glass">
          <option value="newest">Sort: Newest</option>
          <option value="price_low">Sort: Price Low to High</option>
          <option value="price_high">Sort: Price High to Low</option>
          <option value="name_az">Sort: Name A-Z</option>
        </select>
      </div>

      <p v-if="errorText" class="mt-2.5 text-sm font-semibold text-red-700">{{ errorText }}</p>
      <p v-else-if="statusText" class="mt-2.5 text-sm font-semibold text-blue-700">{{ statusText }}</p>

      <div v-if="!loading && filteredTurfs.length === 0" class="mt-3.5 rounded-[14px] border border-white/95
bg-white/80 p-[18px] backdrop-blur-[14px] shadow-glass">
        No turfs matched your filters.
      </div>

      <section v-else class="mt-3.5 grid grid-cols-3 gap-3.5 max-[1100px]:grid-cols-2
max-[760px]:grid-cols-1">
        <UserBrowseTurfCard
          v-for="turf in filteredTurfs"
          :key="turf.turf_id"
          :turf="turf"
          :today-date="todayDate"
          :slot-loading="slotLoadingMap[turf.turf_id] === true"
          :booking-loading="bookingMap[turf.turf_id] === true"
          :promo-loading="promoLoadingMap[turf.turf_id] === true"
          :selected-slot="getSelectedSlot(turf)"
          :price-breakdown="getPriceBreakdown(turf)"
          :format-slot-label="formatSlotLabel"
          :format-duration="formatDuration"
          :format-money="formatMoney"
```

```
          :status-text-class="statusTextClass"
          @update-field="updateTurfField"
          @date-change="onDateChange"
          @slot-change="onSlotChange"
          @apply-promo="applyPromoCode"
          @book="bookTurf"
        />
      </section>
    </main>
  </div>
</template>

<script>
import AppTopbar from '../components/AppTopbar.vue'
import UserBrowseTurfCard from '../components/UserBrowseTurfCard.vue'
import GlassButton from '../components/GlassButton.vue'
import {
  getAvailableSports,
  getDhakaTodayDate,
  getFilteredTurfs,
  userBrowseTurfsMethods
} from '../support/userBrowseTurfsSupport'

export default {
  name: 'UserBrowseTurfs',
  components: {
    AppTopbar,
    UserBrowseTurfCard,
    GlassButton
  },
  data() {
    return {
      turfs: [],
      loading: false,
      statusText: '',
      errorText: '',
      bookingMap: {},
      promoLoadingMap: {},
      slotLoadingMap: {},
      dateInputDebounceMap: {},
      searchText: '',
      sportFilter: 'all',
      locationFilter: '',
      minPrice: '',
      maxPrice: '',
      sortBy: 'newest',
      todayDate: getDhakaTodayDate()
    }
  },
  computed: {
    availableSports() {
      return getAvailableSports(this.turfs)
    },
    filteredTurfs() {
      return getFilteredTurfs(this.turfs, {
        searchText: this.searchText,
        sportFilter: this.sportFilter,
        locationFilter: this.locationFilter,
        minPrice: this.minPrice,
        maxPrice: this.maxPrice,
        sortBy: this.sortBy
      })
    }
  },
  async mounted() {
    await this.loadTurfs()
  },
  methods: {
    ...userBrowseTurfsMethods
  }
}
</script>
```

==========
54. src/components/UserDashboard.vue
==========

```
<template>
  <div class="min-h-screen w-full box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)]
p-2.5 font-poppins text-[#111827]">
    <AppTopbar
      wrapper-class="mb-3 w-full rounded-2xl border border-slate-100 bg-white px-4 py-3.5
shadow-[0_8px_20px_rgba(0,0,0,0.08)] backdrop-blur-0 max-md:flex-wrap"
      left-class="gap-2.5 max-md:flex-wrap"
      right-class="gap-1.5 max-md:flex-wrap"
    >
      <template #left>
        <GlassButton v-if="isOwner" class="whitespace-nowrap" @click="switchToOwnerDashboard">Switch Owner
Dashboard</GlassButton>
        <GlassButton v-if="isAdmin" class="whitespace-nowrap" @click="switchToAdminDashboard">Switch Admin
Dashboard</GlassButton>
        <div class="text-[26px] font-bold tracking-[-0.05em] text-[#18223f]">TurfBooking</div>
      </template>

      <template #right>
        <UserDashboardProfileMenu
          :user-name="userName || 'User'"
          @profile="openProfileMenu"
          @settings="openSettingsMenu"
          @logout="logout"
        />
      </template>
    </AppTopbar>

    <div class="grid min-h-[calc(100vh-108px)] grid-cols-[248px_minmax(0,1fr)] items-start gap-3.5
max-[900px]:grid-cols-1">
      <aside class="sticky top-2 rounded-[20px] border border-transparent bg-white/80 p-3 shadow-glass
backdrop-blur-[14px] max-[900px]:static">
        <div class="mb-3 rounded-[16px] border border-transparent bg-white/85 px-3.5 py-3 shadow-glass
transition duration-200 hover:-translate-y-0.5 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]">
          <p class="text-[11px] font-semibold uppercase tracking-[0.22em] text-slate-500">User Panel</p>
          <h2 class="mt-1 text-lg font-bold text-slate-900">{{ userName || 'User' }}</h2>
        </div>

        <nav class="flex flex-col gap-2.5">
          <button
            v-for="item in sidebarItems"
            :key="item.key"
            type="button"
            class="rounded-[14px] border border-transparent px-3.5 py-3 text-left font-medium text-slate-900
backdrop-blur-[14px] shadow-glass transition duration-200"
            :class="item.key === 'dashboard'
              ? 'scale-[1.02] bg-slate-900 text-white shadow-[0_16px_24px_rgba(15,23,42,0.18)]'
              : 'bg-white/75 hover:-translate-y-0.5 hover:scale-[1.02] hover:bg-white/95 hover:font-semibold
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]'"
            @click="handleSidebarAction(item.key)"
          >
            {{ item.label }}
          </button>
        </nav>
      </aside>

      <main class="min-w-0 rounded-2xl border border-slate-100 bg-white p-4
shadow-[0_8px_20px_rgba(0,0,0,0.08)]">
        <section class="flex items-start justify-between gap-3 max-[760px]:flex-col">
          <div>
            <h1 class="m-0 text-[28px] font-bold tracking-[-0.04em] text-[#141b33]">User Dashboard</h1>
            <p class="mt-3 text-[16px] text-[#2d3446]">Welcome, <b>{{ userName || 'User' }}</b> - manage your
bookings easily.</p>
          </div>

          <div class="min-w-[120px] rounded-2xl border border-slate-100 bg-white px-4 py-3 text-center
```

```
shadow-[0_8px_20px_rgba(0,0,0,0.08)] transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_24px_rgba(15,23,42,0.1)]">
            <p class="text-[13px] text-slate-600">Rising</p>
            <p class="mt-1 text-[20px] font-bold text-[#141b33]">{{ points }}</p>
            <p class="text-[15px] font-semibold text-[#141b33]">Reward Points</p>
          </div>
        </section>

        <section class="mt-3 grid grid-cols-4 gap-3 max-[1180px]:grid-cols-2 max-[700px]:grid-cols-1">
          <article class="rounded-[14px] border border-white/95 bg-white/88 p-4 shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:shadow-[0_16px_24px_rgba(20,32,89,0.14)]">
            <div class="flex items-center gap-2 text-[14px] font-semibold text-[#25314d]">
              <span class="flex h-7 w-7 items-center justify-center rounded-full bg-[#d8f0eb] text-[#249780]
shadow-[0_6px_12px_rgba(36,151,128,0.18)]">
                <img :src="bookingsIcon" alt="" class="h-4 w-4 object-contain" aria-hidden="true" />
              </span>
              <span>Total Bookings</span>
            </div>
            <p class="mt-3 text-[20px] font-bold leading-none text-[#18223f]">{{ stats.total }}</p>
            <p class="mt-3 text-[13px] font-medium text-[#25a28a]">+12% this month</p>
          </article>

          <article class="rounded-[14px] border border-white/95 bg-white/88 p-4 shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:shadow-[0_16px_24px_rgba(20,32,89,0.14)]">
            <div class="flex items-center gap-2 text-[14px] font-semibold text-[#25314d]">
              <span class="flex h-7 w-7 items-center justify-center rounded-full bg-[#e7edf7] text-[#6f7c97]
shadow-[0_6px_12px_rgba(87,102,138,0.14)]">
                <img :src="upcomingIcon" alt="" class="h-4 w-4 object-contain" aria-hidden="true" />
              </span>
              <span>Upcoming Bookings</span>
            </div>
            <p class="mt-3 text-[20px] font-bold leading-none text-[#18223f]">{{ stats.upcoming }}</p>
          </article>

          <article class="rounded-[14px] border border-white/95 bg-white/88 p-4 shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:shadow-[0_16px_24px_rgba(20,32,89,0.14)]">
            <div class="flex items-center gap-2 text-[14px] font-semibold text-[#25314d]">
              <span class="flex h-7 w-7 items-center justify-center rounded-full bg-[#e3e8f4] text-[#5d6a86]
shadow-[0_6px_12px_rgba(87,102,138,0.14)]">
                <img :src="walletIcon" alt="" class="h-4 w-4 object-contain" aria-hidden="true" />
              </span>
              <span>Wallet Balance</span>
            </div>
            <p class="mt-3 text-[20px] font-bold leading-none text-[#18223f]">Tk 200</p>
            <p class="mt-3 text-[13px] text-[#687389]">Demo wallet balance</p>
          </article>

          <article class="rounded-[14px] border border-white/95 bg-white/88 p-4 shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:shadow-[0_16px_24px_rgba(20,32,89,0.14)]">
            <div class="flex items-center gap-2 text-[14px] font-semibold text-[#25314d]">
              <span class="flex h-7 w-7 items-center justify-center rounded-full bg-[#fff0d9] text-[#d39b35]
shadow-[0_6px_12px_rgba(211,155,53,0.16)]">
                <img :src="rewardIcon" alt="" class="h-4 w-4 object-contain" aria-hidden="true" />
              </span>
              <span>Reward Points</span>
            </div>
            <p class="mt-3 text-[20px] font-bold leading-none text-[#18223f]">{{ points }}</p>
            <p class="mt-3 text-[13px] text-[#687389]">Reward Points</p>
          </article>
        </section>

        <section class="mt-3 grid grid-cols-[1.05fr_1.05fr_1fr] gap-3 max-[1180px]:grid-cols-1">
          <UserUpcomingCard
            ref="upcomingCard"
            :next-booking="nextBooking"
            :pending-review="pendingReview"
            :cancel-loading="cancelUpcomingLoading"
            :action-message="upcomingActionMsg"
            :format-date="formatDate"
            :format-time="formatTime"
            :format-money="formatMoney"
            :status-class="statusClass"
```

```
            :status-badge-tone="statusBadgeTone"
            @browse-turfs="goToTurfs"
            @cancel-booking="cancelUpcomingBooking"
            @open-review="openReview"
          />

          <UserActivityCard
            :activities="visibleActivities"
            :format-date-time="formatDateTime"
            @refresh="loadDashboard"
          />

          <UserWalletCard
            ref="walletCard"
            :points="points"
            :points-breakdown="pointsBreakdown"
          />
        </section>

        <div class="mt-3 flex justify-end gap-2 max-md:justify-start">
          <button type="button" class="appearance-none rounded-full border border-transparent bg-white/85 px-7
 py-3 text-[17px] font-semibold text-slate-900 shadow-glass outline-none backdrop-blur-[14px] transition
duration-200 hover:-translate-y-0.5 hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
@click="loadAll">Refresh</button>
          <button type="button" class="appearance-none rounded-full border border-transparent bg-white/85 px-7
 py-3 text-[17px] font-semibold text-slate-900 shadow-glass outline-none backdrop-blur-[14px] transition
duration-200 hover:-translate-y-0.5 hover:bg-white hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
@click="logout">Logout</button>
        </div>
      </main>
    </div>

    <UserReviewModal
      :visible="showReviewModal"
      :pending-review="pendingReview"
      :rating="rating"
      :comment="comment"
      :message="reviewMsg"
      :loading="reviewLoading"
      @close="closeReview"
      @submit="submitReview"
      @set-rating="rating = $event"
      @update:comment="comment = $event"
    />
  </div>
</template>

<script>
import AppTopbar from ’./AppTopbar.vue’
import GlassButton from ’./GlassButton.vue’
import UserUpcomingCard from ’./UserUpcomingCard.vue’
import UserActivityCard from ’./UserActivityCard.vue’
import UserDashboardProfileMenu from ’./UserDashboardProfileMenu.vue’
import UserWalletCard from ’./UserWalletCard.vue’
import UserReviewModal from ’./UserReviewModal.vue’
import bookingsIcon from ’../assets/total-bookings.svg’
import upcomingIcon from ’../assets/upcoming-bookings.svg’
import walletIcon from ’../assets/wallet-balance.svg’
import rewardIcon from ’../assets/reward-points.svg’
import {
  createUserDashboardStats,
  createUserPointsBreakdown,
  getUserSession,
  userDashboardMethods
} from ’../support/userDashboardSupport’

export default {
  name: ’UserDashboard’,
  components: {
    AppTopbar,
    GlassButton,
    UserUpcomingCard,
```

```
    UserActivityCard,
    UserDashboardProfileMenu,
    UserWalletCard,
    UserReviewModal
  },
  data() {
    return {
      userId: localStorage.getItem('user_id'),
      userName: localStorage.getItem('user_name'),
      bookingsIcon,
      upcomingIcon,
      walletIcon,
      rewardIcon,
      stats: createUserDashboardStats(),
      points: 0,
      pointsBreakdown: createUserPointsBreakdown(),
      nextBooking: null,
      activities: [],
      pendingReview: null,
      rating: 0,
      comment: '',
      reviewLoading: false,
      reviewMsg: '',
      showReviewModal: false,
      cancelUpcomingLoading: false,
      upcomingActionMsg: ''
    }
  },
  computed: {
    currentUser() {
      return getUserSession()
    },
    isOwner() {
      return this.currentUser?.role === 'owner'
    },
    isAdmin() {
      return this.currentUser?.role === 'admin'
    },
    visibleActivities() {
      return this.activities.slice(0, 5)
    },
    sidebarItems() {
      return [
        { key: 'dashboard', label: 'User Dashboard' },
        { key: 'browse', label: 'Browse Turfs' },
        { key: 'bookings', label: 'My Bookings' },
        { key: 'wallet', label: 'My Wallet' },
        { key: 'review', label: 'Rate a Turf' },
        { key: 'transactions', label: 'Transaction History' },
        { key: 'support', label: 'Support' }
      ]
    }
  },
  async mounted() {
    if (!this.userId) {
      this.$router.push('/login')
      return
    }
    await this.loadAll()
  },
  methods: {
    ...userDashboardMethods,
    openProfileMenu() {
      window.scrollTo({ top: 0, behavior: 'smooth' })
    },
    openSettingsMenu() {
      // Demo placeholder for future settings action.
    },
    handleSidebarAction(key) {
      if (key === 'dashboard') return
      if (key === 'browse') {
        this.goToTurfs()
```

```
        return
      }
      if (key === 'bookings') {
        this.goToBookings()
        return
      }
      if (key === 'transactions') {
        this.goToTransactions()
        return
      }
      if (key === 'wallet') {
        this.scrollToWallet()
        return
      }
      if (key === 'review') {
        this.openReview()
      }
    }
  }
}
</script>
```

==========
55. src/components/UserDashboardProfileMenu.vue
==========

```
<template>
  <div ref="root" class="relative z-[70]">
    <button
      type="button"
      class="group flex min-w-[190px] items-center justify-between gap-3 rounded-[14px] border
border-transparent px-3.5 py-3 text-slate-900 backdrop-blur-[14px] shadow-glass transition duration-200"
      :class="menuOpen
        ? 'scale-[1.02] bg-white/95 shadow-[0_16px_24px_rgba(15,23,42,0.18)]'
        : 'bg-white/75 hover:-translate-y-0.5 hover:scale-[1.02] hover:bg-white/95
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]'"
      @click="toggleMenu"
    >
      <div class="flex items-center gap-3">
        <span class="flex h-9 w-9 items-center justify-center rounded-full bg-white/85
shadow-[0_8px_16px_rgba(20,32,89,0.12)]">
          <img :src="avatarSrc" alt="Profile" class="h-8 w-8 rounded-full object-contain" />
        </span>
        <span class="text-[18px] font-semibold tracking-[-0.02em] text-[#1b2441]">{{ userName }}</span>
      </div>
      <img :src="chevronIcon" alt="" class="h-5 w-5 object-contain text-slate-500 transition duration-200
group-hover:text-slate-700" :class="menuOpen ? 'rotate-180' : ''" aria-hidden="true" />
    </button>

    <div
      v-if="menuOpen"
      class="absolute right-0 top-[calc(100%+12px)] z-[80] min-w-[220px] overflow-hidden rounded-[20px] border
 border-[#d8def0] bg-[linear-gradient(180deg,rgba(255,255,255,0.98),rgba(244,247,255,0.95))] p-2.5
shadow-[0_26px_46px_rgba(30,41,84,0.18)] backdrop-blur-[22px]"
    >
      <button
        type="button"
        class="group flex w-full items-center gap-3 rounded-[14px] border border-transparent bg-white/75
px-3.5 py-3 text-left text-[17px] font-medium text-[#22304f] backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:scale-[1.02] hover:bg-white/95
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
        @click="emitAction('profile')"
      >
        <span class="flex h-9 w-9 items-center justify-center rounded-full bg-white/85
shadow-[0_8px_16px_rgba(20,32,89,0.12)] transition duration-200 group-hover:scale-105">
          <img :src="profileIcon" alt="" class="h-5 w-5 object-contain" aria-hidden="true" />
        </span>
        <span>Profile</span>
      </button>
```

```
      <button
        type="button"
        class="group mt-2 flex w-full items-center gap-3 rounded-[14px] border border-transparent bg-white/75
px-3.5 py-3 text-left text-[17px] font-medium text-[#22304f] backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:scale-[1.02] hover:bg-white/95
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
        @click="emitAction('settings')"
      >
        <span class="flex h-9 w-9 items-center justify-center rounded-full bg-white/85
shadow-[0_8px_16px_rgba(20,32,89,0.12)] transition duration-200 group-hover:scale-105">
          <img :src="settingsIcon" alt="" class="h-5 w-5 object-contain" aria-hidden="true" />
        </span>
        <span>Settings</span>
      </button>

      <button
        type="button"
        class="group mt-2 flex w-full items-center gap-3 rounded-[14px] border border-transparent bg-white/75
px-3.5 py-3 text-left text-[17px] font-semibold text-[#17203a] backdrop-blur-[14px] shadow-glass transition
duration-200 hover:-translate-y-0.5 hover:scale-[1.02] hover:bg-white/95
hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
        @click="emitAction('logout')"
      >
        <span class="flex h-9 w-9 items-center justify-center rounded-full bg-[#f8e6e6]
shadow-[0_8px_16px_rgba(176,72,72,0.12)] transition duration-200 group-hover:scale-105">
          <img :src="logoutIcon" alt="" class="h-5 w-5 object-contain" aria-hidden="true" />
        </span>
        <span class="tracking-[-0.01em] text-[#1d2440]">Logout</span>
      </button>
    </div>
  </div>
</template>

<script>
import avatarSrc from '../assets/white avatar.svg'
import chevronIcon from '../assets/chevron.svg'
import profileIcon from '../assets/profile.svg'
import settingsIcon from '../assets/settings.svg'
import logoutIcon from '../assets/logout.svg'

export default {
  name: 'UserDashboardProfileMenu',
  props: {
    userName: {
      type: String,
      default: 'User'
    }
  },
  emits: ['profile', 'settings', 'logout'],
  data() {
    return {
      menuOpen: false,
      avatarSrc,
      chevronIcon,
      profileIcon,
      settingsIcon,
      logoutIcon
    }
  },
  mounted() {
    document.addEventListener('click', this.handleDocumentClick)
    document.addEventListener('keydown', this.handleEscape)
  },
  beforeUnmount() {
    document.removeEventListener('click', this.handleDocumentClick)
    document.removeEventListener('keydown', this.handleEscape)
  },
  methods: {
    toggleMenu() {
      this.menuOpen = !this.menuOpen
    },
    emitAction(type) {
```

```
      this.menuOpen = false
      this.$emit(type)
    },
    handleDocumentClick(event) {
      if (!this.menuOpen) return
      if (this.$refs.root?.contains(event.target)) return
      this.menuOpen = false
    },
    handleEscape(event) {
      if (event.key === 'Escape') {
        this.menuOpen = false
      }
    }
  }
}
</script>
```

==========
56. src/components/UserDashboardSummaryCard.vue
==========

```
<template>
  <button
    type="button"
    class="flex min-h-[154px] w-full flex-col items-start rounded-[22px] border border-white/95 bg-white/82
p-5 text-left text-slate-900 shadow-glass transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_14px_24px_rgba(20,32,89,0.16)]"
    @click="$emit('click')"
  >
    <div class="flex items-start gap-3">
      <div class="grid h-[44px] w-[44px] place-items-center rounded-[14px]" :class="iconShellClass">
        <img :src="iconSrc" :alt="iconAlt || title" class="h-6 w-6 object-contain" />
      </div>
      <div>
        <h3 class="m-0 text-[20px] font-semibold tracking-[-0.03em] text-[#16203b]">{{ title }}</h3>
        <p class="mt-3 text-[56px] font-bold leading-none tracking-[-0.05em] text-[#16203b]">{{ value }}</p>
      </div>
    </div>

    <p v-if="helperText" class="mt-auto pt-4 text-[14px] font-medium" :class="helperClass">{{ helperText
}}</p>
  </button>
</template>

<script>
export default {
  name: 'UserDashboardSummaryCard',
  props: {
    title: { type: String, default: '' },
    value: { type: [String, Number], default: '' },
    helperText: { type: String, default: '' },
    helperClass: { type: String, default: 'text-slate-500' },
    iconSrc: { type: String, required: true },
    iconAlt: { type: String, default: '' },
    iconShellClass: { type: String, default: 'bg-slate-100' }
  },
  emits: ['click']
}
</script>
```

==========
57. src/components/UserLogin.vue
==========

```
<template>
  <div class="relative flex min-h-screen items-center justify-center bg-gray-100 px-4 font-poppins">
    <button
      type="button"
```

```
      class="absolute right-4 top-4 rounded-full border border-white/95 bg-white/80 px-4 py-2 font-semibold
text-slate-900 backdrop-blur-[14px] shadow-glass transition hover:bg-white"
      @click="goBack"
    >
      Back
    </button>
    <div class="w-full max-w-[350px] rounded-xl bg-white p-[30px] shadow-[0_10px_25px_rgba(0,0,0,0.1)]">
      <h2>Login</h2>

      <form @submit.prevent="login">
        <input
          v-model.trim="email"
          type="email"
          class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5"
          placeholder="Email"
          autocomplete="email"
        />
        <input
          v-model="password"
          type="password"
          class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5"
          placeholder="Password"
          autocomplete="current-password"
        />

        <button
          type="submit"
          class="mt-2 w-full rounded-md bg-[#2f428f] px-3 py-2.5 font-bold text-white transition-colors
hover:bg-[#5a7ee8] focus-visible:bg-[#5a7ee8] disabled:cursor-not-allowed disabled:opacity-85"
          :disabled="loading || !canSubmit"
        >
          {{ loading ? 'Checking...' : 'Login' }}
        </button>
      </form>

      <p
        v-if="message"
        class="mt-2.5 text-center"
        :class="{
          'text-green-700': messageType === 'success',
          'text-red-700': messageType === 'error',
          'text-blue-800': messageType === 'info'
        }"
      >
        {{ message }}
      </p>
    </div>
  </div>
</template>

<script>
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const LOGIN_ENDPOINT = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend/login.php`

export default {
  name: 'UserLogin',
  data() {
    return {
      email: '',
      password: '',
      message: '',
      messageType: '',
      loading: false
    }
  },
  mounted() {
    if (!this.email) {
```

```
    const savedEmail = localStorage.getItem('last_registered_email')
    if (savedEmail) {
      this.email = savedEmail
    }
  }
},
computed: {
  canSubmit() {
    return this.email.includes('@') && this.password.length > 0
  }
},
methods: {
  saveUserSession(user) {
    localStorage.setItem('user_id', String(user.user_id || ''))
    localStorage.setItem('user_name', user.full_name || 'User')
    localStorage.setItem('user_email', user.email || this.email)

    if (user.owner_id) {
      localStorage.setItem('owner_id', String(user.owner_id))
    } else {
      localStorage.removeItem('owner_id')
    }

    const resolvedAdminId = Number(user.admin_id || 0) > 0
      ? Number(user.admin_id)
      : (user.role === 'admin' ? Number(user.user_id || 0) : 0)

    if (resolvedAdminId > 0) {
      localStorage.setItem('admin_id', String(resolvedAdminId))
    } else {
      localStorage.removeItem('admin_id')
    }

    localStorage.setItem('user', JSON.stringify(user))
    localStorage.setItem('mode', user.role === 'admin' ? 'admin' : 'user')
  },
  redirectAfterLogin(user) {
    if (user?.role === 'admin') {
      this.$router.push('/admin-dashboard')
      return
    }

    this.$router.push('/dashboard')
  },
  goBack() {
    if (window.history.length > 1) {
      this.$router.back()
      return
    }
    this.$router.push('/')
  },
  async login() {
    if (this.loading) return
    if (!this.canSubmit) {
      this.messageType = 'error'
      this.message = 'Please provide valid email and password.'
      return
    }

    this.loading = true
    this.messageType = 'info'
    this.message = 'Checking credentials...'

    const payload = new URLSearchParams()
    payload.append('email', this.email)
    payload.append('password', this.password)

    try {
      const response = await axios.post(LOGIN_ENDPOINT, payload, {
        timeout: 5000,
        headers: {
          'Content-Type': 'application/x-www-form-urlencoded'
```

```
          }
        })

        if (response.data?.success) {
          const user = response.data.user || {}
          this.saveUserSession(user)
          this.messageType = 'success'
          this.message = response.data.status || 'Login successful.'
          this.redirectAfterLogin(user)
          return
        }

        this.messageType = 'error'
        this.message = response.data?.status || 'Invalid email or password.'
      } catch (error) {
        console.error(error)
        this.messageType = 'error'
        if (error.response?.data?.status) {
          this.message = error.response.data.status
        } else {
          this.message = error.code === 'ECONNABORTED'
            ? 'Server response timeout.'
            : 'Server connection failed.'
        }
      } finally {
        this.loading = false
      }
    }
  }
}
</script>
```

==========
58. src/components/UserMyBookingsPage.vue
==========

```
<template>
  <div class="min-h-screen box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)] p-2.5
font-poppins">
    <AppTopbar>
      <template #right>
        <div class="font-semibold text-slate-900">My Bookings</div>
        <GlassButton @click="goBack">Back</GlassButton>
      </template>
    </AppTopbar>

    <main class="mt-3 rounded-[20px] border border-white/95 bg-white/80 p-4 backdrop-blur-[14px]
shadow-glass">
      <div class="flex flex-wrap items-center justify-between gap-2.5">
        <div>
          <h1 class="m-0 text-[26px] font-bold tracking-[-0.03em] text-slate-900">My Bookings</h1>
          <p class="mt-1 text-sm text-slate-600">Track your reserved slots, booking status, and match
schedule.</p>
        </div>
        <div class="flex flex-wrap gap-2">
          <GlassButton
            v-for="item in statusFilters"
            :key="item.value"
            class="px-3 py-[7px]"
            :active="activeFilter === item.value"
            @click="activeFilter = item.value"
          >
            {{ item.label }}
          </GlassButton>
        </div>
        <GlassButton @click="loadBookings" :disabled="loading">
          {{ loading ? 'Loading...' : 'Refresh' }}
        </GlassButton>
      </div>
```

```
      <p
        v-if="message"
        class="mt-2.5 font-semibold"
        :class="{
          'text-green-700': messageType === 'success',
          'text-red-700': messageType === 'error',
          'text-blue-700': messageType === 'info'
        }"
      >
        {{ message }}
      </p>

      <div v-if="!loading && filteredBookings.length === 0" class="mt-3 rounded-xl border border-white/95
bg-white/80 p-4 backdrop-blur-[14px] shadow-glass">
        No bookings found.
      </div>

      <section class="mt-3 flex flex-col gap-3">
        <article
          v-for="row in filteredBookings"
          :key="row.booking_id"
          class="rounded-[14px] border border-white/95 bg-white/80 p-3.5 backdrop-blur-[14px] shadow-glass"
        >
          <div class="mb-2 flex items-center justify-between gap-2.5">
            <h3 class="m-0 text-xl font-semibold text-slate-900">{{ row.turf_name }}</h3>
            <StatusBadge :label="row.booking_status" :tone="badgeTone(row.booking_status)" />
          </div>

          <div class="grid grid-cols-3 gap-2 text-sm text-slate-700 max-[900px]:grid-cols-1">
            <div><b>Date:</b> {{ formatDate(row.slot_date) }}</div>
            <div><b>Time:</b> {{ formatTime(row.start_time) }} - {{ formatTime(row.end_time) }}</div>
            <div><b>Booking ID:</b> #{{ row.booking_id }}</div>
            <div><b>Price:</b> Tk {{ Number(row.booked_price || 0).toFixed(2) }}</div>
            <div><b>Location:</b> {{ row.location || 'N/A' }}</div>
          </div>

          <p v-if="row.booking_status === 'cancelled' && Number(row.refund_amount || 0) > 0" class="mt-2
text-sm text-slate-500">
            Refund details are available in Transaction History.
          </p>

          <div class="mt-2.5 flex justify-end">
            <GlassButton
              v-if="row.can_cancel"
              class="px-3 py-2 font-bold text-red-700"
              :disabled="cancelLoadingId === row.booking_id"
              @click="cancelBooking(row)"
            >
              {{ cancelLoadingId === row.booking_id ? 'Cancelling...' : 'Cancel Booking' }}
            </GlassButton>
          </div>
        </article>
      </section>
    </main>
  </div>
</template>

<script>
import axios from 'axios'
import AppTopbar from '../components/AppTopbar.vue'
import GlassButton from '../components/GlassButton.vue'
import StatusBadge from '../components/StatusBadge.vue'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const API_BASE = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend'

export default {
  name: 'UserMyBookingsPage',
  components: {
```

```
    AppTopbar,
    GlassButton,
    StatusBadge
  },
  data() {
    return {
      userId: Number(localStorage.getItem('user_id') || 0),
      bookings: [],
      loading: false,
      cancelLoadingId: null,
      message: '',
      messageType: 'info',
      activeFilter: 'all',
      statusFilters: [
        { label: 'All', value: 'all' },
        { label: 'Confirmed', value: 'confirmed' },
        { label: 'Pending', value: 'pending' },
        { label: 'Completed', value: 'completed' },
        { label: 'Cancelled', value: 'cancelled' }
      ]
    }
  },
  computed: {
    filteredBookings() {
      if (this.activeFilter === 'all') return this.bookings
      return this.bookings.filter((b) => String(b.booking_status).toLowerCase() === this.activeFilter)
    }
  },
  async mounted() {
    if (!this.userId) {
      this.$router.push('/login')
      return
    }
    await this.loadBookings()
  },
  methods: {
    goBack() {
      if (window.history.length > 1) {
        this.$router.back()
        return
      }
      this.$router.push('/dashboard')
    },
    badgeTone(status) {
      const value = String(status || '').toLowerCase()
      if (value === 'confirmed') return 'success'
      if (value === 'pending') return 'warning'
      if (value === 'cancelled') return 'danger'
      if (value === 'completed') return 'info'
      return 'neutral'
    },
    formatDate(value) {
      if (!value) return 'N/A'
      const d = new Date(value)
      if (Number.isNaN(d.getTime())) return value
      return d.toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' })
    },
    formatTime(value) {
      if (!value) return 'N/A'
      const txt = String(value)
      const parts = txt.split(':')
      if (parts.length < 2) return txt
      let hour = Number(parts[0])
      const minute = parts[1]
      if (Number.isNaN(hour)) return txt
      const ampm = hour >= 12 ? 'PM' : 'AM'
      hour %= 12
      if (hour === 0) hour = 12
      return `${hour}:${minute} ${ampm}`
    },
    async loadBookings() {
      this.loading = true
```

```
      this.message = ''
      try {
        const response = await axios.get(`${API_BASE}/get_user_bookings.php?user_id=${this.userId}`)
        if (response.data?.success) {
          this.bookings = Array.isArray(response.data.bookings) ? response.data.bookings : []
        } else {
          this.message = response.data?.message || 'Could not load bookings.'
          this.messageType = 'error'
        }
      } catch (error) {
        this.message = error.response?.data?.message || 'Server connection failed.'
        this.messageType = 'error'
      } finally {
        this.loading = false
      }
    },
    async cancelBooking(row) {
      this.cancelLoadingId = row.booking_id
      this.message = ''
      try {
        const payload = {
          user_id: this.userId,
          booking_id: row.booking_id
        }
        const response = await axios.post(`${API_BASE}/cancel_booking.php`, payload, {
          headers: { 'Content-Type': 'application/json' }
        })

        if (response.data?.success) {
          this.message = `Cancellation request sent. After admin approval: Refund Tk
${Number(response.data.refund_amount || 0).toFixed(2)} (80%)`
          this.messageType = 'success'
          await this.loadBookings()
        } else {
          this.message = response.data?.message || 'Cancellation failed.'
          this.messageType = 'error'
        }
      } catch (error) {
        this.message = error.response?.data?.message || 'Server connection failed.'
        this.messageType = 'error'
      } finally {
        this.cancelLoadingId = null
      }
    }
  }
}
</script>
```

==========
59. src/components/UserRegister.vue
==========

```
<template>
  <div class="relative flex min-h-screen items-center justify-center bg-gray-100 px-4 font-poppins">
    <button
      type="button"
      class="absolute right-4 top-4 rounded-full border border-white/95 bg-white/80 px-4 py-2 font-semibold
text-slate-900 backdrop-blur-[14px] shadow-glass transition hover:bg-white"
      @click="goBack"
    >
      Back
    </button>

    <div class="w-full max-w-[350px] rounded-xl bg-white p-[30px] shadow-[0_10px_25px_rgba(0,0,0,0.1)]">
      <h2>Create Account</h2>

      <form @submit.prevent="register">
        <input
          v-model.trim="full_name"
          class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5"
```

```
          placeholder="Full Name"
          autocomplete="name"
        />
        <input
          v-model.trim="email"
          type="email"
          class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5"
          placeholder="Email"
          autocomplete="email"
        />
        <input
          v-model.trim="phone"
          type="tel"
          class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5"
          placeholder="Phone"
          autocomplete="tel"
        />

        <div class="relative">
          <input
            v-model="password"
            :type="showPassword ? 'text' : 'password'"
            class="my-2 w-full box-border border border-gray-500 px-2.5 py-2.5 pr-12"
            placeholder="Password"
            autocomplete="new-password"
          />
          <span
            class="absolute right-3 top-1/2 -translate-y-1/2 cursor-pointer select-none text-xs font-semibold
text-gray-800"
            @click="showPassword = !showPassword"
          >
            {{ showPassword ? 'Hide' : 'Show' }}
          </span>
        </div>

        <button
          type="submit"
          class="mt-2 inline-flex w-full items-center justify-center rounded-md bg-[#2f428f] px-3 py-2.5
font-bold text-white transition-colors hover:bg-[#5a7ee8] focus-visible:bg-[#5a7ee8] disabled:cursor-wait
disabled:opacity-85"
          :disabled="loading"
        >
          <span
            v-if="loading"
            class="mr-2 inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-white
border-t-transparent"
          ></span>
          {{ loading ? 'Registering...' : 'Register' }}
        </button>
      </form>

      <p class="mt-4 text-center">
        Already have an account?
        <button class="ml-1 w-auto border-none bg-transparent p-0 font-bold text-blue-900 underline"
@click="goToLogin">
          Login
        </button>
      </p>

      <p
        v-if="message"
        class="mt-2.5 text-center"
        :class="{
          'text-green-700': messageType === 'success',
          'text-red-700': messageType === 'error',
          'text-blue-800': messageType === 'info'
        }"
      >
        {{ message }}
      </p>
    </div>
  </div>
```

```
</template>

<script>
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const REGISTER_ENDPOINT = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend/register.php'

export default {
  name: 'UserRegister',
  data() {
    return {
      full_name: '',
      email: '',
      phone: '',
      password: '',
      message: '',
      messageType: '',
      loading: false,
      showPassword: false
    }
  },
  computed: {
    canSubmit() {
      return (
        this.full_name.length > 1 &&
        this.email.includes('@') &&
        this.phone.length >= 6 &&
        this.password.length >= 6
      )
    }
  },
  methods: {
    async register() {
      if (this.loading) return

      if (!this.canSubmit) {
        this.messageType = 'error'
        this.message = 'Please fill all fields correctly.'
        return
      }

      this.loading = true
      this.messageType = 'info'
      this.message = 'Processing...'

      try {
        const payload = new URLSearchParams()
        payload.append('full_name', this.full_name)
        payload.append('email', this.email)
        payload.append('phone', this.phone)
        payload.append('password', this.password)

        const response = await axios.post(REGISTER_ENDPOINT, payload, {
          timeout: 4000,
          headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
        })

        this.messageType = response.data?.success === false ? 'error' : 'success'
        this.message = response.data?.status || 'Registration completed.'

        if (response.data?.success !== false) {
          localStorage.setItem('last_registered_email', this.email)

          this.full_name = ''
          this.email = ''
          this.phone = ''
          this.password = ''
```

```
          setTimeout(() => {
            this.$router.push('/login')
          }, 1500)
        }
      } catch (error) {
        console.error(error)
        this.messageType = 'error'
        if (error.response?.data?.status) {
          this.message = error.response.data.status
        } else {
          this.message =
            error.code === 'ECONNABORTED'
              ? 'Server response timeout.'
              : 'Server connection failed.'
        }
      }

      this.loading = false
    },

    goToLogin() {
      this.$router.push('/login')
    },
    goBack() {
      if (window.history.length > 1) {
        this.$router.back()
        return
      }
      this.$router.push('/login')
    }
  }
}
</script>
```

==========
60. src/components/UserReviewModal.vue
==========

```
<template>
  <div v-if="visible && pendingReview" class="fixed inset-0 z-50 grid place-items-center bg-slate-900/30 p-4"
@click.self="$emit('close')">
    <div class="w-full max-w-[460px] rounded-[18px] border border-white/95 bg-white/80 p-[18px] text-slate-900
 backdrop-blur-[14px] shadow-glass">
      <h3 class="m-0">Rate your last booking</h3>
      <p class="mt-2 text-[#1f2937]">{{ pendingReview.turf_name }}</p>

      <div class="my-3 flex gap-1.5">
        <button
          v-for="n in 5"
          :key="n"
          type="button"
          class="h-[38px] w-[38px] cursor-pointer rounded-[10px] border border-white/75 bg-white/80 text-lg
text-slate-900"
          :class="{ 'bg-[#fde68a] text-[#3b2f03]': rating >= n }"
          @click="$emit('set-rating', n)"
        >
          *
        </button>
      </div>

      <textarea
        :value="comment"
        class="min-h-[80px] w-full resize-y rounded-xl border border-white/80 bg-white/70 p-2.5
text-slate-900"
        placeholder="Write a short comment (optional)"
        @input="$emit('update:comment', $event.target.value)"
      ></textarea>

      <p v-if="message" class="mt-2.5 text-sm text-[#111827]">{{ message }}</p>
```

```
      <div class="mt-3 flex justify-end gap-2.5">
        <GlassButton @click="$emit('submit')" :disabled="loading">
          {{ loading ? 'Submitting...' : 'Submit Review' }}
        </GlassButton>
        <GlassButton @click="$emit('close')">Close</GlassButton>
      </div>
    </div>
  </div>
</template>

<script>
import GlassButton from './GlassButton.vue'

export default {
  name: 'UserReviewModal',
  components: {
    GlassButton
  },
  props: {
    visible: {
      type: Boolean,
      default: false
    },
    pendingReview: {
      type: Object,
      default: null
    },
    rating: {
      type: Number,
      default: 0
    },
    comment: {
      type: String,
      default: ''
    },
    message: {
      type: String,
      default: ''
    },
    loading: {
      type: Boolean,
      default: false
    }
  },
  emits: ['close', 'submit', 'set-rating', 'update:comment']
}
</script>
```

==========
61. src/components/UserTransactionHistoryPage.vue
==========

```
<template>
  <div class="min-h-screen box-border bg-[linear-gradient(135deg,#f8fafc_0%,#eef2ff_50%,#f3f4f6_100%)] p-2.5
font-poppins">
    <AppTopbar>
      <template #right>
        <div class="font-semibold text-slate-900">Transaction History</div>
        <GlassButton @click="goBack">Back</GlassButton>
      </template>
    </AppTopbar>

    <main class="mt-3 rounded-[20px] border border-white/95 bg-white/80 p-4 backdrop-blur-[14px]
shadow-glass">
      <div class="flex flex-wrap items-center justify-between gap-2.5">
        <div>
          <h1 class="m-0 text-[26px] font-bold tracking-[-0.03em] text-slate-900">Transaction History</h1>
          <p class="mt-1 text-sm text-slate-600">Review your booking payments, wallet refunds, and
credits.</p>
        </div>
```

```
        <GlassButton @click="refreshTransactions" :disabled="loading">
          {{ loading ? 'Loading...' : 'Refresh' }}
        </GlassButton>
      </div>

      <section class="mt-4 grid grid-cols-3 gap-3 max-[900px]:grid-cols-1">
        <article class="rounded-[16px] border border-white/95 bg-white/85 p-4 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Total Paid</p>
          <p class="mt-2 text-[30px] font-bold text-slate-900">Tk {{ totalPaid }}</p>
        </article>
        <article class="rounded-[16px] border border-white/95 bg-white/85 p-4 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Refunded</p>
          <p class="mt-2 text-[30px] font-bold text-slate-900">Tk {{ totalRefunded }}</p>
        </article>
        <article class="rounded-[16px] border border-white/95 bg-white/85 p-4 shadow-glass">
          <p class="m-0 text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">Entries</p>
          <p class="mt-2 text-[30px] font-bold text-slate-900">{{ rows.length }}</p>
        </article>
      </section>

      <div v-if="!loading && rows.length === 0" class="mt-4 rounded-[16px] border border-white/95 bg-white/85
p-4 text-slate-600 shadow-glass">
        No transactions found yet.
      </div>

      <section v-else class="mt-4 flex flex-col gap-3">
        <article
          v-for="row in rows"
          :key="row.id"
          class="rounded-[16px] border border-white/95 bg-white/85 p-4 shadow-glass"
        >
          <div class="flex items-center justify-between gap-3 max-[760px]:flex-col max-[760px]:items-start">
            <div>
              <div class="flex items-center gap-2">
                <StatusBadge :label="row.type" :tone="row.tone" />
                <span class="text-sm text-slate-500">{{ row.reference }}</span>
              </div>
              <h3 class="mt-2 text-lg font-semibold text-slate-900">{{ row.title }}</h3>
              <p class="mt-1 text-sm text-slate-600">{{ row.description }}</p>
              <div class="mt-2 flex flex-wrap gap-x-4 gap-y-1 text-sm text-slate-500">
                <span><b>Method:</b> {{ row.method }}</span>
                <span><b>Status:</b> {{ row.status }}</span>
              </div>
            </div>

            <div class="text-right max-[760px]:text-left">
              <p class="text-[22px] font-bold" :class="row.amount > 0 ? 'text-slate-900' : row.amount < 0 ?
'text-rose-700' : 'text-amber-700'">
                {{ row.amount > 0 ? '+' : row.amount < 0 ? '-' : '' }}Tk {{ Math.abs(row.amount).toFixed(2) }}
              </p>
              <p class="mt-1 text-sm text-slate-500">{{ row.time }}</p>
            </div>
          </div>
        </article>
      </section>
    </main>
  </div>
</template>

<script>
import axios from 'axios'
import AppTopbar from '../components/AppTopbar.vue'
import GlassButton from '../components/GlassButton.vue'
import StatusBadge from '../components/StatusBadge.vue'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const API_BASE = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend'

export default {
```

```
  name: 'UserTransactionHistoryPage',
  components: {
    AppTopbar,
    GlassButton,
    StatusBadge
  },
  data() {
    return {
      userId: Number(localStorage.getItem('user_id') || 0),
      rows: [],
      loading: false
    }
  },
  computed: {
    totalPaid() {
      return this.rows.filter((row) => row.amount < 0).reduce((sum, row) => sum + Math.abs(row.amount),
0).toFixed(2)
    },
    totalRefunded() {
      return this.rows.filter((row) => row.amount > 0).reduce((sum, row) => sum + row.amount, 0).toFixed(2)
    }
  },
  async mounted() {
    if (!this.userId) {
      this.$router.push('/login')
      return
    }
    await this.loadTransactions()
  },
  methods: {
    goBack() {
      if (window.history.length > 1) {
        this.$router.back()
        return
      }
      this.$router.push('/dashboard')
    },
    async loadTransactions() {
      this.loading = true
      try {
        const response = await axios.get(`${API_BASE}/get_user_transactions.php?user_id=${this.userId}`)
        if (response.data?.success) {
          this.rows = Array.isArray(response.data.transactions) ? response.data.transactions : []
        } else {
          this.rows = []
        }
      } catch (error) {
        this.rows = []
      } finally {
        this.loading = false
      }
    },
    async refreshTransactions() {
      await this.loadTransactions()
    }
  }
}
</script>
```

==========
62. src/components/UserUpcomingCard.vue
==========

```
<template>
  <article class="flex min-h-[420px] flex-col rounded-2xl border border-slate-100 bg-white p-[32px_22px_18px]
text-slate-900 shadow-[0_8px_20px_rgba(0,0,0,0.08)] transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_12px_22px_rgba(0,0,0,0.1)]">
    <h3 class="mb-4 text-[24px] font-semibold tracking-[-0.03em] text-[#141b33]">Upcoming Booking</h3>

    <div v-if="nextBooking" class="flex-1 text-[17px] text-[#30384d]">
```

```
      <div class="mb-1.5"><b class="text-[#1b2236]">{{ nextBooking.turf_name || 'Turf' }}</b></div>
      <div class="mb-1.5">{{ formatDate(nextBooking.slot_date) }}</div>
      <div class="mb-1.5">
        {{ formatTime(nextBooking.start_time) }} - {{ formatTime(nextBooking.end_time) }}
      </div>
      <div class="mb-1.5">Price: Tk {{ formatMoney(nextBooking.booked_price) }}</div>
      <StatusBadge
        class="mt-2"
        :label="nextBooking.booking_status || 'pending'"
        :tone="statusBadgeTone(nextBooking.booking_status)"
      />
    </div>

    <div v-else class="flex flex-1 flex-col items-center justify-center text-center text-[#30384d]">
      <img :src="footballImage" alt="" class="h-[88px] w-auto object-contain" aria-hidden="true" />
      <p class="mt-4 text-[20px] font-semibold tracking-[-0.03em] text-[#1b2236]">No upcoming booking</p>
      <p class="mt-2 text-[15px] text-[#5c667d]">Browse turfs to reserve your next slot</p>
      <button
        type="button"
        class="mt-6 appearance-none rounded-[16px] border border-[#16213d]
bg-[linear-gradient(180deg,#16213d_0%,#0f1830_100%)] px-6 py-3 text-[16px] font-semibold text-white
shadow-[0_16px_28px_rgba(15,24,48,0.28)] outline-none transition duration-200 hover:-translate-y-0.5
hover:bg-[linear-gradient(180deg,#1a2748_0%,#121d37_100%)] hover:shadow-[0_18px_30px_rgba(15,24,48,0.34)]"
        @click="$emit('browse-turfs')"
      >
        Browse Turfs
      </button>
    </div>

    <div v-if="nextBooking && statusClass(nextBooking.booking_status) === 'confirmed' &&
nextBooking.refund_request_status !== 'pending'" class="mt-auto flex flex-wrap gap-2">
      <button
        type="button"
        class="appearance-none rounded-[14px] border border-transparent bg-white/75 px-4 py-3 font-semibold
text-slate-900 shadow-glass outline-none backdrop-blur-[14px] transition duration-200 hover:-translate-y-0.5
hover:scale-[1.02] hover:bg-white/95 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]
disabled:cursor-not-allowed disabled:opacity-60"
        :disabled="cancelLoading"
        @click="$emit('cancel-booking')"
      >
        {{ cancelLoading ? 'Cancelling...' : 'Cancel Booking' }}
      </button>
    </div>
    <div v-if="pendingReview" class="mt-3 flex flex-wrap gap-2">
      <button
        type="button"
        class="appearance-none rounded-[14px] border border-transparent bg-white/75 px-4 py-3 font-semibold
text-slate-900 shadow-glass outline-none backdrop-blur-[14px] transition duration-200 hover:-translate-y-0.5
hover:scale-[1.02] hover:bg-white/95 hover:shadow-[0_14px_20px_rgba(20,32,89,0.18)]"
        @click="$emit('open-review')"
      >
        Rate Completed Booking
      </button>
    </div>
    <p v-if="nextBooking && nextBooking.refund_request_status === 'pending'" class="mt-2 text-sm
text-[#111827]">Cancellation request is pending admin approval.</p>
    <p v-if="actionMessage" class="mt-2 text-sm text-[#111827]">{{ actionMessage }}</p>
  </article>
</template>

<script>
import StatusBadge from './StatusBadge.vue'
import footballImage from '../assets/football.png'

export default {
  name: 'UserUpcomingCard',
  components: {
    StatusBadge
  },
  data() {
    return {
      footballImage
```

```
    }
  },
  props: {
    nextBooking: {
      type: Object,
      default: null
    },
    pendingReview: {
      type: Object,
      default: null
    },
    cancelLoading: {
      type: Boolean,
      default: false
    },
    actionMessage: {
      type: String,
      default: ''
    },
    formatDate: {
      type: Function,
      required: true
    },
    formatTime: {
      type: Function,
      required: true
    },
    formatMoney: {
      type: Function,
      required: true
    },
    statusClass: {
      type: Function,
      required: true
    },
    statusBadgeTone: {
      type: Function,
      required: true
    }
  },
  emits: ['browse-turfs', 'cancel-booking', 'open-review']
}
</script>



==========================================================================================
63. src/components/UserWalletCard.vue
==========================================================================================

<template>
  <article class="flex min-h-[420px] flex-col rounded-2xl border border-slate-100 bg-white p-[32px_18px_18px]
text-slate-900 shadow-[0_8px_20px_rgba(0,0,0,0.08)] transition duration-200 hover:-translate-y-0.5
hover:shadow-[0_12px_22px_rgba(0,0,0,0.1)]">
    <h3 class="mb-4 text-[24px] font-semibold tracking-[-0.03em] text-[#141b33]">Wallet / Points
Breakdown</h3>

    <div v-for="item in rows" :key="item.key" class="mb-4">
      <div class="flex items-center justify-between gap-2.5 text-[17px] text-[#2f374a]">
        <span>{{ item.label }}</span>
        <b class="text-[#1a2135]">{{ item.value }}</b>
      </div>
    </div>

    <div class="mt-auto flex items-center justify-between pt-3 text-[18px] font-bold text-[#1a2135]">
      <span>Total Reward Points</span>
      <b>{{ points }}</b>
    </div>
  </article>
</template>

<script>
```

```
export default {
  name: 'UserWalletCard',
  computed: {
    rows() {
      return [
        {
          key: 'booking',
          label: 'From confirmed/completed bookings',
          value: this.pointsBreakdown.booking
        },
        {
          key: 'review',
          label: 'Reviews',
          value: this.pointsBreakdown.review
        },
        {
          key: 'referral',
          label: 'Referral / Promo bonus',
          value: this.pointsBreakdown.referral
        },
        {
          key: 'resell',
          label: 'Slot resell',
          value: this.pointsBreakdown.resell
        }
      ]
    }
  },
  props: {
    points: {
      type: Number,
      default: 0
    },
    pointsBreakdown: {
      type: Object,
      default: () => ({
        booking: 0,
        review: 0,
        referral: 0,
        resell: 0
      })
    }
  }
}
</script>
```

==========
64. src/main.js
==========

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './assets/theme.css'

createApp(App).use(router).mount('#app')
```

==========
65. src/router/index.js
==========

```
import { createRouter, createWebHistory } from 'vue-router'

import UserRegister from '../components/UserRegister.vue'
import UserLogin from '../components/UserLogin.vue'
import UserDashboard from '../components/UserDashboard.vue'
import OwnerDashboard from '../components/OwnerDashboard.vue'
import AdminDashboard from '../components/AdminDashboard.vue'
```

```
import UserBrowseTurfs from '../components/UserBrowseTurfs.vue'
import UserMyBookingsPage from '../components/UserMyBookingsPage.vue'
import UserTransactionHistoryPage from '../components/UserTransactionHistoryPage.vue'

function getUser() {
  const raw = localStorage.getItem('user')
  return raw ? JSON.parse(raw) : null
}

const authGuard = (next) => {
  const user = getUser()
  if (!user) {
    next('/login')
    return false
  }
  return true
}

const routes = [
  {
    path: '/',
    component: UserRegister
  },
  {
    path: '/login',
    component: UserLogin
  },
  {
    path: '/dashboard',
    component: UserDashboard,
    beforeEnter: (to, from, next) => {
      if (!authGuard(next)) return
      localStorage.setItem('mode', 'user')
      next()
    }
  },
  {
    path: '/owner-dashboard',
    component: OwnerDashboard,
    beforeEnter: (to, from, next) => {
      const user = getUser()
      if (!user) {
        next('/login')
      } else if (user.role !== 'owner') {
        next('/dashboard')
      } else {
        localStorage.setItem('mode', 'owner')
        next()
      }
    }
  },
  {
    path: '/admin-dashboard',
    component: AdminDashboard,
    beforeEnter: (to, from, next) => {
      const user = getUser()
      if (!user) {
        next('/login')
      } else if (user.role !== 'admin') {
        next('/dashboard')
      } else {
        localStorage.setItem('mode', 'admin')
        next()
      }
    }
  },
  {
    path: '/browse',
    alias: '/turfs',
    component: UserBrowseTurfs,
    beforeEnter: (to, from, next) => {
      if (!authGuard(next)) return
```

```
      next()
    }
  },
  {
    path: '/bookings',
    component: UserMyBookingsPage,
    beforeEnter: (to, from, next) => {
      if (!authGuard(next)) return
      next()
    }
  },
  {
    path: '/transactions',
    component: UserTransactionHistoryPage,
    beforeEnter: (to, from, next) => {
      if (!authGuard(next)) return
      next()
    }
  },
  {
    path: '/:pathMatch(.*)*',
    redirect: '/'
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

==========
66. src/support/adminDashboardSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const API_BASE = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend'

export const ADMIN_MENU_ITEMS = [
  { key: 'overview', label: 'Overview' },
  { key: 'verifyTurfs', label: 'Verify Turfs' },
  { key: 'bookingRequests', label: 'Booking Requests' },
  { key: 'refunds', label: 'Refund Requests' },
  { key: 'usersOwners', label: 'Users & Owners' },
  { key: 'analytics', label: 'Analytics' }
]

export function getAdminSessionUser() {
  try {
    const raw = localStorage.getItem('user')
    return raw ? JSON.parse(raw) : {}
  } catch (error) {
    return {}
  }
}

export function createAdminStats() {
  return {
    users_active: 0,
    users_banned: 0,
    owners_verified: 0,
    owners_suspended: 0,
    owners_pending: 0,
    pending_turfs: 0,
```

```
    pending_bookings: 0,
    pending_refunds: 0,
    today_revenue: 0
  }
}

export function createAdminAnalytics() {
  return {
    top_rated_turfs: [],
    most_active_users: [],
    revenue_trend: [],
    booking_growth: []
  }
}

export const adminDashboardMethods = {
  switchToUserDashboard() {
    localStorage.setItem('mode', 'user')
    this.$router.push('/dashboard')
  },
  goBack() {
    if (window.history.length > 1) {
      this.$router.back()
      return
    }
    this.$router.push('/dashboard')
  },
  setMessage(type, text) {
    this.messageType = type
    this.message = text
  },
  getApiErrorMessage(error, fallback) {
    const responseData = error?.response?.data
    if (typeof responseData === 'string' && responseData.trim()) {
      const compact = responseData.replace(/<[^>]*>/g, ' ').replace(/\s+/g, ' ').trim()
      return compact || fallback
    }
    if (responseData?.message) return responseData.message
    if (responseData?.status) return responseData.status
    if (error?.message) return error.message
    return fallback
  },
  keyFor(type, id) {
    return `${type}:${id}`
  },
  isActionLoading(type, id) {
    return !!this.actionLoadingMap[this.keyFor(type, id)]
  },
  setActionLoading(type, id, value) {
    this.actionLoadingMap = {
      ...this.actionLoadingMap,
      [this.keyFor(type, id)]: value
    }
  },
  formatMoney(v) {
    const n = Number(v || 0)
    return Number.isFinite(n) ? n.toFixed(2) : '0.00'
  },
  formatDate(v) {
    if (!v) return 'N/A'
    const d = new Date(v)
    if (Number.isNaN(d.getTime())) return v
    return d.toLocaleDateString('en-US', { year: 'numeric', month: 'short', day: 'numeric' })
  },
  formatTime(value) {
    if (!value) return 'N/A'
    const txt = String(value)
    const parts = txt.split(':')
    if (parts.length < 2) return txt
    let hour = Number(parts[0])
    const minute = parts[1]
    if (Number.isNaN(hour)) return txt
```

```
    const ampm = hour >= 12 ? 'PM' : 'AM'
    hour %= 12
    if (hour === 0) hour = 12
    return `${hour}:${minute} ${ampm}`
  },
  statusTone(value) {
    const status = String(value || '').toLowerCase()
    if (['active', 'verified', 'confirmed', 'approved'].includes(status)) return 'success'
    if (status === 'pending') return 'warning'
    if (['banned', 'suspended', 'cancelled', 'rejected'].includes(status)) return 'danger'
    return 'neutral'
  },
  async loadDashboard() {
    if (!this.adminId) {
      this.setMessage('error', 'Admin session not found. Please login again.')
      return
    }

    this.setMessage('info', 'Loading...')

    try {
      const response = await axios.get(`${API_BASE}/admin_dashboard.php`, {
        params: { admin_id: this.adminId },
        timeout: 10000
      })

      if (!response.data?.success) {
        this.setMessage('error', response.data?.message || 'Failed to load dashboard.')
        return
      }

      this.stats = response.data.stats || this.stats
      this.monitorUsers = Array.isArray(response.data.monitor_users) ? response.data.monitor_users : []
      this.monitorOwners = Array.isArray(response.data.monitor_owners) ? response.data.monitor_owners : []
      this.pendingTurfs = Array.isArray(response.data.pending_turfs) ? response.data.pending_turfs : []
      this.pendingBookings = Array.isArray(response.data.pending_bookings) ? response.data.pending_bookings :
[]
      this.pendingRefunds = Array.isArray(response.data.pending_refunds) ? response.data.pending_refunds : []
      this.analytics = response.data.analytics || this.analytics

      this.setMessage('success', 'Refreshed')
    } catch (error) {
      this.setMessage('error', this.getApiErrorMessage(error, 'Server connection failed.'))
    }
  },
  async reviewTurf(item, action) {
    this.setActionLoading('turf', item.turf_id, true)
    try {
      const response = await axios.post(`${API_BASE}/admin_turf_action.php`, {
        admin_id: this.adminId,
        turf_id: item.turf_id,
        action,
        note: this.turfNotes[item.turf_id] || ''
      }, {
        headers: { 'Content-Type': 'application/json' },
        timeout: 10000
      })

      if (response.data?.success) {
        this.turfNotes = { ...this.turfNotes, [item.turf_id]: '' }
        this.setMessage('success', `Turf #${item.turf_id} ${action.replace('_', ' ')} successfully.`)
        await this.loadDashboard()
      } else {
        this.setMessage('error', response.data?.message || 'Turf action failed.')
      }
    } catch (error) {
      this.setMessage('error', this.getApiErrorMessage(error, 'Turf action failed.'))
    } finally {
      this.setActionLoading('turf', item.turf_id, false)
    }
  },
  async reviewBooking(item, action) {
```

```
    this.setActionLoading('booking', item.booking_id, true)
    try {
      const response = await axios.post(`${API_BASE}/admin_booking_action.php`, {
        admin_id: this.adminId,
        booking_id: item.booking_id,
        action
      }, {
        headers: { 'Content-Type': 'application/json' },
        timeout: 10000
      })

      if (response.data?.success) {
        const plus = Number(response.data.points_awarded || 0)
        this.setMessage('success', `Booking #${item.booking_id} ${response.data.status}.${plus > 0 ? `
+${plus} points` : ''}`)
        await this.loadDashboard()
      } else {
        this.setMessage('error', response.data?.message || 'Booking action failed.')
      }
    } catch (error) {
      this.setMessage('error', this.getApiErrorMessage(error, 'Booking action failed.'))
    } finally {
      this.setActionLoading('booking', item.booking_id, false)
    }
  },
  async reviewRefund(item, action) {
    this.setActionLoading('refund', item.refund_id, true)
    try {
      const response = await axios.post(`${API_BASE}/admin_refund_action.php`, {
        admin_id: this.adminId,
        refund_id: item.refund_id,
        action,
        admin_note: this.refundNotes[item.refund_id] || ''
      }, {
        headers: { 'Content-Type': 'application/json' },
        timeout: 10000
      })

      if (response.data?.success) {
        this.refundNotes = { ...this.refundNotes, [item.refund_id]: '' }
        this.setMessage('success', `Refund #${item.refund_id} updated to ${response.data.status}.`)
        await this.loadDashboard()
      } else {
        this.setMessage('error', response.data?.message || 'Refund action failed.')
      }
    } catch (error) {
      this.setMessage('error', this.getApiErrorMessage(error, 'Refund action failed.'))
    } finally {
      this.setActionLoading('refund', item.refund_id, false)
    }
  },
  async toggleUserStatus(type, id, action) {
    this.setActionLoading(type, id, true)
    try {
      const response = await axios.post(`${API_BASE}/admin_user_action.php`, {
        admin_id: this.adminId,
        target_type: type,
        target_id: id,
        action
      }, {
        headers: { 'Content-Type': 'application/json' },
        timeout: 10000
      })

      if (response.data?.success) {
        this.setMessage('success', `${type} #${id} status updated to ${response.data.status}.`)
        await this.loadDashboard()
      } else {
        this.setMessage('error', response.data?.message || 'Status update failed.')
      }
    } catch (error) {
      this.setMessage('error', this.getApiErrorMessage(error, 'Status update failed.'))
```

```
    } finally {
      this.setActionLoading(type, id, false)
    }
  }
}
```

==========
67. src/support/ownerBookingsSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const OWNER_BOOKING_CLIENTS_ENDPOINT =
`${API_PROTOCOL}//${API_HOST}/turfbooking/backend/owner_booking_clients.php`

export function formatOwnerClientDate(value) {
  if (!value) return 'N/A'
  const date = new Date(`${value}T00:00:00`)
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleDateString('en-GB', {
    day: 'numeric',
    month: 'short',
    year: 'numeric'
  })
}

export function formatOwnerClientTime(value) {
  if (!value) return 'N/A'
  const date = new Date(`1970-01-01T${value}`)
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleTimeString('en-US', {
    hour: 'numeric',
    minute: '2-digit'
  })
}

export function formatOwnerClientMoney(value) {
  const amount = Number(value || 0)
  return Number.isFinite(amount) ? amount.toFixed(2) : '0.00'
}

export function getOwnerClientStatusTone(status) {
  const value = String(status || '').toLowerCase()
  if (value === 'completed') return 'bg-emerald-100 text-emerald-800'
  if (value === 'cancelled') return 'bg-rose-100 text-rose-700'
  if (value === 'confirmed') return 'bg-sky-100 text-sky-800'
  return 'bg-amber-100 text-amber-800'
}

export function filterOwnerClientBookings(bookings, search) {
  const query = String(search || '').trim().toLowerCase()
  if (!query) return bookings

  return bookings.filter((booking) => {
    const haystack = [
      booking.booking_id,
      booking.turf_name,
      booking.sport_type,
      booking.client?.full_name,
      booking.client?.email,
      booking.client?.phone,
      booking.booking_status
    ].join(' ').toLowerCase()

    return haystack.includes(query)
  })
```

```
}

export async function fetchOwnerBookingClients(ownerId) {
  const response = await axios.get(OWNER_BOOKING_CLIENTS_ENDPOINT, {
    params: { owner_id: ownerId },
    timeout: 12000
  })
  return response.data
}
```

==========
68. src/support/ownerDashboardSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const ADD_TURF_ENDPOINT = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend/add_turf.php`
const OWNER_FINANCE_ENDPOINT = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend/owner_finance.php`

export const OWNER_MENU_ITEMS = [
  { key: 'dashboard', label: 'Dashboard' },
  { key: 'myTurfs', label: 'Add Turfs' },
  { key: 'ownerTurfs', label: 'My Turfs' },
  { key: 'bookings', label: 'Bookings' },
  { key: 'revenue', label: 'Revenue' },
  { key: 'slotControl', label: 'Slot Control' },
  { key: 'promoCodes', label: 'Promo Codes' },
  { key: 'reviews', label: 'Reviews' }
]

export function getOwnerSessionUser() {
  try {
    const raw = localStorage.getItem('user')
    return raw ? JSON.parse(raw) : {}
  } catch (error) {
    return {}
  }
}

export function createOwnerFinanceSummary() {
  return {
    wallet_balance: 0,
    total_cancellation_earnings: 0,
    pending_refund_requests: 0,
    cancelled_bookings: 0,
    active_turfs: 0
  }
}

export function createOwnerForm() {
  return {
    turf_name: '',
    sport_type: 'football',
    address: '',
    area: '',
    city: '',
    location: '',
    latitude: '',
    longitude: '',
    price_per_hour: '',
    description: '',
    is_featured: false,
    status: 'pending',
    cancel_before_hours: 24,
    refund_percent: 80
  }
```

```
}

function toPositiveNumber(value) {
  const n = Number(value)
  return Number.isFinite(n) && n > 0 ? n : null
}

export const ownerDashboardMethods = {
  formatMoney(value) {
    const n = Number(value || 0)
    return Number.isFinite(n) ? n.toFixed(2) : '0.00'
  },
  async loadOwnerFinance() {
    if (!this.ownerIdFromSession) return
    this.financeLoading = true
    try {
      const response = await axios.get(OWNER_FINANCE_ENDPOINT, {
        params: { owner_id: this.ownerIdFromSession },
        timeout: 10000
      })
      if (response.data?.success) {
        this.ownerFinance = { ...this.ownerFinance, ...(response.data.summary || {}) }
        this.ownerTransactions = Array.isArray(response.data.transactions) ? response.data.transactions : []
      }
    } catch (error) {
      // keep old stats if request fails
    } finally {
      this.financeLoading = false
    }
  },
  switchToUserDashboard() {
    localStorage.setItem('mode', 'user')
    this.$router.push('/dashboard')
  },
  goBack() {
    if (window.history.length > 1) {
      this.$router.back()
      return
    }
    this.$router.push('/dashboard')
  },
  resetForm() {
    this.form = createOwnerForm()
    this.message = ''
    this.messageType = ''
    this.lastCreatedTurfId = null
    this.clearSelectedImage()
  },
  buildPayload() {
    const payload = new FormData()
    payload.append('owner_id', String(this.ownerIdFromSession || 0))
    payload.append('owner_email', this.ownerEmail || '')
    payload.append('turf_name', this.form.turf_name)
    payload.append('sport_type', this.form.sport_type)
    payload.append('address', this.form.address)
    payload.append('area', this.form.area)
    payload.append('city', this.form.city)
    payload.append('location', this.form.location)
    payload.append('latitude', this.form.latitude)
    payload.append('longitude', this.form.longitude)
    payload.append('price_per_hour', String(this.form.price_per_hour))
    payload.append('description', this.form.description)
    payload.append('is_featured', this.form.is_featured ? '1' : '0')
    payload.append('status', this.form.status)
    payload.append('cancel_before_hours', String(this.form.cancel_before_hours))
    payload.append('refund_percent', String(this.form.refund_percent))
    if (this.selectedImageFile) {
      payload.append('turf_image', this.selectedImageFile)
    }
    return payload
  },
  handleImageChange(event) {
```

```
    const files = event?.target?.files
    const file = files && files[0] ? files[0] : null

    if (!file) {
      this.clearSelectedImage()
      return
    }

    const allowedTypes = ['image/jpeg', 'image/png', 'image/webp']
    if (!allowedTypes.includes(file.type)) {
      this.messageType = 'error'
      this.message = 'Only JPG, PNG, and WEBP images are allowed.'
      this.clearSelectedImage()
      return
    }

    const maxSize = 5 * 1024 * 1024
    if (file.size > maxSize) {
      this.messageType = 'error'
      this.message = 'Image size must be 5MB or less.'
      this.clearSelectedImage()
      return
    }

    if (this.imagePreviewUrl) {
      URL.revokeObjectURL(this.imagePreviewUrl)
    }

    this.selectedImageFile = file
    this.selectedImageName = file.name
    this.imagePreviewUrl = URL.createObjectURL(file)
  },
  clearSelectedImage() {
    if (this.imagePreviewUrl) {
      URL.revokeObjectURL(this.imagePreviewUrl)
    }
    this.imagePreviewUrl = ''
    this.selectedImageFile = null
    this.selectedImageName = ''
    this.fileInputKey += 1
  },
  validateTurfForm() {
    if (!this.form.turf_name.trim()) {
      return 'Turf name is required.'
    }

    if (!this.form.address.trim()) {
      return 'Address is required.'
    }

    if (toPositiveNumber(this.form.price_per_hour) === null) {
      return 'Price per hour must be a positive number.'
    }

    if (!this.selectedImageFile) {
      return 'Please choose a turf image (JPG, PNG, or WEBP) before saving.'
    }

    return ''
  },
  async submitTurf() {
    if (this.loading) return
    const validationError = this.validateTurfForm()
    if (validationError) {
      this.messageType = 'error'
      this.message = validationError
      return
    }

    this.loading = true
    this.messageType = 'info'
    this.message = 'Saving turf...'
```

```
    this.lastCreatedTurfId = null

    try {
      const response = await axios.post(ADD_TURF_ENDPOINT, this.buildPayload(), {
        timeout: 12000
      })

      if (response.data?.success) {
        this.messageType = 'success'
        this.message = response.data.status || 'Turf added successfully.'
        this.lastCreatedTurfId = response.data.turf_id || null
        await this.loadOwnerFinance()
        this.form = {
          ...createOwnerForm(),
          city: this.form.city,
          sport_type: this.form.sport_type
        }
        this.clearSelectedImage()
        return
      }

      this.messageType = 'error'
      this.message = response.data?.status || 'Failed to add turf.'
    } catch (error) {
      this.messageType = 'error'
      if (error.response?.data?.status) {
        this.message = error.response.data.status
      } else if (error.code === 'ECONNABORTED') {
        this.message = 'Server timeout. Please try again.'
      } else {
        this.message = 'Server connection failed.'
      }
    } finally {
      this.loading = false
    }
  }
}
```

==========
69. src/support/ownerPromoCodesSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const OWNER_PROMO_CODES_ENDPOINT = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend/owner_promo_codes.php`

export function createOwnerPromoForm() {
  return {
    code: '',
    discount_type: 'percent',
    discount_value: '',
    min_booking_amount: '',
    start_date: '',
    end_date: ''
  }
}

export function formatOwnerPromoMoney(value) {
  const amount = Number(value || 0)
  return Number.isFinite(amount) ? amount.toFixed(2) : '0.00'
}

export function formatOwnerPromoDate(value) {
  if (!value) return 'No limit'
  const date = new Date(`${value}T00:00:00`)
  if (Number.isNaN(date.getTime())) return value
```

```
  return date.toLocaleDateString('en-GB', {
    day: 'numeric',
    month: 'short',
    year: 'numeric'
  })
}

export function buildOwnerPromoSummary(summary) {
  return [
    { label: 'Total Codes', value: summary.total_codes || 0 },
    { label: 'Active Codes', value: summary.active_codes || 0 },
    { label: 'Inactive Codes', value: summary.inactive_codes || 0 }
  ]
}

export async function fetchOwnerPromoCodes(ownerId) {
  const response = await axios.get(OWNER_PROMO_CODES_ENDPOINT, {
    params: { owner_id: ownerId },
    timeout: 12000
  })
  return response.data
}

export async function createOwnerPromoCode(ownerId, form) {
  const response = await axios.post(OWNER_PROMO_CODES_ENDPOINT, {
    owner_id: ownerId,
    action: 'create',
    ...form
  }, {
    timeout: 12000
  })
  return response.data
}

export async function toggleOwnerPromoCode(ownerId, promoId, isActive) {
  const response = await axios.post(OWNER_PROMO_CODES_ENDPOINT, {
    owner_id: ownerId,
    action: 'toggle',
    promo_id: promoId,
    is_active: isActive ? 1 : 0
  }, {
    timeout: 12000
  })
  return response.data
}



==========================================================================================
70. src/support/ownerSlotControlSupport.js
==========================================================================================

import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const OWNER_SLOT_CONTROL_ENDPOINT = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend/owner_slot_control.php`

export function createOwnerSlotControlState() {
  return {
    turfId: '',
    slotDate: new Date().toISOString().slice(0, 10)
  }
}

export function formatOwnerSlotDate(value) {
  if (!value) return 'N/A'
  const date = new Date(`${value}T00:00:00`)
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleDateString('en-GB', {
```

```
    day: 'numeric',
    month: 'short',
    year: 'numeric'
  })
}

export function formatOwnerSlotTime(value) {
  if (!value) return 'N/A'
  const date = new Date('1970-01-01T${value}')
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleTimeString('en-US', {
    hour: 'numeric',
    minute: '2-digit'
  })
}

export function formatOwnerSlotMoney(value) {
  const amount = Number(value || 0)
  return Number.isFinite(amount) ? amount.toFixed(2) : '0.00'
}

export function getOwnerSlotStatusTone(slot) {
  if (slot.is_booked) return 'bg-rose-100 text-rose-700'
  if (slot.is_enabled) return 'bg-emerald-100 text-emerald-800'
  return 'bg-slate-200 text-slate-700'
}

export async function fetchOwnerSlotControl(ownerId, turfId, slotDate) {
  const response = await axios.get(OWNER_SLOT_CONTROL_ENDPOINT, {
    params: {
      owner_id: ownerId,
      turf_id: turfId || undefined,
      slot_date: slotDate
    },
    timeout: 12000
  })
  return response.data
}

export async function updateOwnerSlotAvailability(ownerId, turfId, slotDate, slotIds, isEnabled) {
  const response = await axios.post(OWNER_SLOT_CONTROL_ENDPOINT, {
    owner_id: ownerId,
    turf_id: turfId,
    slot_date: slotDate,
    slot_ids: slotIds,
    is_enabled: isEnabled
  }, {
    timeout: 12000
  })
  return response.data
}
```

==========
71. src/support/ownerTurfsSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const OWNER_TURFS_ENDPOINT = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend/owner_my_turfs.php'

export function formatOwnerTurfMoney(value) {
  const amount = Number(value || 0)
  return Number.isFinite(amount) ? amount.toFixed(2) : '0.00'
}

export function formatOwnerTurfDate(value) {
```

```
  if (!value) return 'Unknown'
  const date = new Date(value)
  if (Number.isNaN(date.getTime())) return String(value)
  return date.toLocaleDateString('en-GB', {
    day: 'numeric',
    month: 'short',
    year: 'numeric'
  })
}

export function getOwnerTurfStatusTone(status) {
  const value = String(status || '').toLowerCase()
  if (value === 'active') return 'bg-emerald-100 text-emerald-800'
  if (value === 'inactive') return 'bg-slate-200 text-slate-700'
  if (value === 'pending') return 'bg-amber-100 text-amber-800'
  if (value === 'rejected') return 'bg-rose-100 text-rose-800'
  return 'bg-slate-200 text-slate-700'
}

export function buildOwnerTurfSummary(summary) {
  return [
    { label: 'Total Turfs', value: summary.total || 0 },
    { label: 'Active', value: summary.active || 0 },
    { label: 'Inactive', value: summary.inactive || 0 },
    { label: 'Pending', value: summary.pending || 0 }
  ]
}

export async function fetchOwnerTurfs(ownerId) {
  const response = await axios.get(OWNER_TURFS_ENDPOINT, {
    params: { owner_id: ownerId },
    timeout: 12000
  })
  return response.data
}

export async function toggleOwnerTurfStatus(ownerId, turfId, status) {
  const response = await axios.post(OWNER_TURFS_ENDPOINT, {
    owner_id: ownerId,
    turf_id: turfId,
    status
  }, {
    timeout: 12000
  })
  return response.data
}
```

==========
72. src/support/userBrowseTurfsSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const API_BASE = `${API_PROTOCOL}//${API_HOST}/turfbooking/backend`
const VALIDATE_PROMO_ENDPOINT = `${API_BASE}/validate_promo_code.php`

function toMinutes(timeText) {
  const value = String(timeText || '')
  const parts = value.split(':')
  if (parts.length < 2) return 0
  const h = Number(parts[0] || 0)
  const m = Number(parts[1] || 0)
  return (h * 60) + m
}

function getDhakaDateParts(date = new Date()) {
```

```
  const parts = new Intl.DateTimeFormat('en-US', {
    timeZone: 'Asia/Dhaka',
    year: 'numeric',
    month: '2-digit',
    day: '2-digit'
  }).formatToParts(date)

  const map = {}
  parts.forEach((part) => {
    map[part.type] = part.value
  })

  return {
    year: map.year || '1970',
    month: map.month || '01',
    day: map.day || '01'
  }
}

export function getDhakaTodayDate() {
  const d = getDhakaDateParts()
  return `${d.year}-${d.month}-${d.day}`
}

function getDhakaCurrentTime() {
  const parts = new Intl.DateTimeFormat('en-US', {
    timeZone: 'Asia/Dhaka',
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hourCycle: 'h23'
  }).formatToParts(new Date())

  const map = {}
  parts.forEach((part) => {
    map[part.type] = part.value
  })

  return `${map.hour || '00'}:${map.minute || '00'}:${map.second || '00'}`
}

function buildImageUrl(imageUrl) {
  if (!imageUrl) return ''
  if (String(imageUrl).startsWith('http')) return imageUrl
  return `${API_BASE}/${String(imageUrl).replace(/^\/+/, '')}`
}

export function getAvailableSports(turfs) {
  const sports = new Set()
  turfs.forEach((turf) => {
    if (turf.sport_type) sports.add(String(turf.sport_type))
  })
  return Array.from(sports).sort((a, b) => a.localeCompare(b))
}

export function getFilteredTurfs(turfs, filters) {
  const search = String(filters.searchText || '').toLowerCase()
  const location = String(filters.locationFilter || '').toLowerCase()
  const minPrice = filters.minPrice === '' ? null : Number(filters.minPrice)
  const maxPrice = filters.maxPrice === '' ? null : Number(filters.maxPrice)

  let rows = turfs.filter((turf) => {
    const turfPrice = Number(turf.price_per_hour || 0)
    const matchSearch = !search || String(turf.turf_name || '').toLowerCase().includes(search)
    const matchSport = filters.sportFilter === 'all' || String(turf.sport_type || '') === filters.sportFilter
    const composedLocation = `${turf.city || ''} ${turf.area || ''} ${turf.location || ''} ${turf.address || 
''}`.toLowerCase()
    const matchLocation = !location || composedLocation.includes(location)
    const matchMin = minPrice === null || turfPrice >= minPrice
    const matchMax = maxPrice === null || turfPrice <= maxPrice
    return matchSearch && matchSport && matchLocation && matchMin && matchMax
  })
```

```
  rows = [...rows]
  if (filters.sortBy === 'price_low') {
    rows.sort((a, b) => Number(a.price_per_hour || 0) - Number(b.price_per_hour || 0))
  } else if (filters.sortBy === 'price_high') {
    rows.sort((a, b) => Number(b.price_per_hour || 0) - Number(a.price_per_hour || 0))
  } else if (filters.sortBy === 'name_az') {
    rows.sort((a, b) => String(a.turf_name || '').localeCompare(String(b.turf_name || '')))
  } else {
    rows.sort((a, b) => String(b.created_at || '').localeCompare(String(a.created_at || '')))
  }

  return rows
}

export const userBrowseTurfsMethods = {
  updateTurfField(turf, field, value) {
    if (!turf || typeof field !== 'string') return
    turf[field] = value
    if (field === 'promo_code') {
      turf.applied_promo = null
    }
  },
  goBack() {
    if (window.history.length > 1) {
      this.$router.back()
      return
    }
    this.$router.push('/dashboard')
  },
  statusTextClass(type) {
    if (type === 'success') return 'text-green-700'
    if (type === 'error') return 'text-red-700'
    return 'text-blue-700'
  },
  normalizeTurfRows(rawTurfs) {
    return rawTurfs.map((turf) => ({
      ...turf,
      image_full_url: buildImageUrl(turf.image_url),
      booking_date: '',
      available_slots: [],
      selected_slot_id: 0,
      promo_code: '',
      applied_promo: null,
      payment_method: 'cash',
      message: '',
      messageType: ''
    }))
  },
  setTurfMessage(turf, text, type = 'info') {
    turf.message = text
    turf.messageType = type
  },
  formatMoney(value) {
    return Number(value || 0).toFixed(2)
  },
  formatTime(timeText) {
    const value = String(timeText || '')
    if (!value) return '--:--'

    const parts = value.split(':')
    if (parts.length < 2) return value.slice(0, 5)

    let hour = Number(parts[0])
    const minute = String(parts[1] || '00').padStart(2, '0')
    if (Number.isNaN(hour)) return value.slice(0, 5)

    if (hour >= 24) hour %= 24
    const ampm = hour >= 12 ? 'PM' : 'AM'
    let displayHour = hour % 12
    if (displayHour === 0) displayHour = 12
```

```
    return ‘${displayHour}:${minute} ${ampm}‘
  },
  formatSlotLabel(slot) {
    return ‘${this.formatTime(slot.start_time)} - ${this.formatTime(slot.end_time)} (BDT)‘
  },
  formatDuration(hours) {
    const value = Number(hours || 0)
    if (value <= 0) return ’0h’
    return ‘${value.toFixed(2)}h‘
  },
  isValidDateInput(dateText) {
    return /^\d{4}-\d{2}-\d{2}$/.test(String(dateText || ’’))
  },
  isPastDate(dateText) {
    if (!this.isValidDateInput(dateText)) return false
    return String(dateText) < this.todayDate
  },
  getDateYear(dateText) {
    if (!this.isValidDateInput(dateText)) return 0
    return Number(String(dateText).slice(0, 4))
  },
  isPastSlotDateTime(slotDate, startTime) {
    if (!slotDate || !startTime) return false
    if (String(slotDate) < this.todayDate) return true
    if (String(slotDate) > this.todayDate) return false
    const nowTime = getDhakaCurrentTime()
    const slotStart = String(startTime).slice(0, 8)
    return slotStart <= nowTime
  },
  getCurrentUserId() {
    try {
      const direct = Number(localStorage.getItem(’user_id’) || 0)
      if (direct > 0) return direct
      const raw = localStorage.getItem(’user’)
      if (!raw) return 0
      const parsed = JSON.parse(raw)
      return Number(parsed?.user_id || 0)
    } catch (error) {
      return 0
    }
  },
  getSelectedSlot(turf) {
    if (!turf || !Array.isArray(turf.available_slots)) return null
    return turf.available_slots.find((slot) => Number(slot.slot_id) === Number(turf.selected_slot_id)) || null
  },
  onSlotChange(turf) {
    turf.applied_promo = null
    if (!Number(turf.selected_slot_id || 0)) {
      this.setTurfMessage(turf, ’Please select a slot.’, ’info’)
      return
    }
    this.setTurfMessage(turf, ’Slot selected. Review and confirm booking.’, ’info’)
  },
  calculateDurationHours(startTime, endTime) {
    const startMinutes = toMinutes(startTime)
    const endMinutes = toMinutes(endTime)
    if (endMinutes <= startMinutes) return 0
    return (endMinutes - startMinutes) / 60
  },
  getPriceBreakdown(turf) {
    const selected = this.getSelectedSlot(turf)
    if (!selected) return null

    const durationHours = this.calculateDurationHours(selected.start_time, selected.end_time)
    const rate = Number(turf.price_per_hour || 0)

    let subtotal = durationHours > 0 ? durationHours * rate : 0
    if (subtotal <= 0) {
      subtotal = Number(selected.base_price || 0)
    }

    return {
```

```
    durationHours,
    subtotal,
    promoCode: turf.applied_promo?.code || '',
    discountAmount: Number(turf.applied_promo?.discount_amount || 0),
    serviceFee: 0,
    total: Math.max(0, Number(turf.applied_promo?.final_total ?? subtotal))
  }
},
async loadTurfs() {
  this.loading = true
  this.errorText = ''
  this.statusText = 'Loading turfs...'

  try {
    const response = await axios.get('${API_BASE}/get_turfs.php', { timeout: 8000 })
    if (!response.data?.success) {
      this.errorText = response.data?.status || 'Failed to load turfs.'
      this.statusText = ''
      return
    }

    const rows = Array.isArray(response.data.turfs) ? response.data.turfs : []
    this.turfs = this.normalizeTurfRows(rows)
    this.statusText = this.turfs.length ? '' : 'No turfs found.'
  } catch (error) {
    this.errorText = error.response?.data?.status || 'Server connection failed while loading turfs.'
    this.statusText = ''
  } finally {
    this.loading = false
  }
},
onDateChange(turf) {
  turf.available_slots = []
  turf.selected_slot_id = 0
  turf.applied_promo = null

  const turfId = Number(turf.turf_id || 0)
  const existingTimer = this.dateInputDebounceMap[turfId]
  if (existingTimer) {
    clearTimeout(existingTimer)
  }

  if (!turf.booking_date) {
    this.setTurfMessage(turf, 'Please pick a date first.', 'info')
    return
  }
  if (!this.isValidDateInput(turf.booking_date)) {
    this.setTurfMessage(turf, '', 'info')
    return
  }

  const year = this.getDateYear(turf.booking_date)
  if (year > 0 && year < 2000) {
    this.setTurfMessage(turf, '', 'info')
    return
  }

  const timer = setTimeout(() => {
    if (this.isPastDate(turf.booking_date)) {
      this.setTurfMessage(turf, 'Select valid date', 'error')
      return
    }
    this.loadTurfSlots(turf)
  }, 350)

  this.dateInputDebounceMap = {
    ...this.dateInputDebounceMap,
    [turfId]: timer
  }
},
async loadTurfSlots(turf) {
  if (!turf.booking_date) {
```

```
      this.setTurfMessage(turf, 'Please select a date to check slots.', 'error')
      return
    }
    if (!this.isValidDateInput(turf.booking_date)) {
      this.setTurfMessage(turf, '', 'info')
      return
    }
    if (this.isPastDate(turf.booking_date)) {
      this.setTurfMessage(turf, 'Select valid date', 'error')
      return
    }

    this.slotLoadingMap = { ...this.slotLoadingMap, [turf.turf_id]: true }
    turf.available_slots = []
    turf.selected_slot_id = 0
    turf.applied_promo = null
    this.setTurfMessage(turf, 'Checking available slots...', 'info')

    try {
      const response = await axios.get('${API_BASE}/get_available_slots.php', {
        timeout: 8000,
        params: { turf_id: turf.turf_id, slot_date: turf.booking_date }
      })

      if (!response.data?.success) {
        this.setTurfMessage(turf, response.data?.status || 'Could not load slots.', 'error')
        return
      }

      const rows = Array.isArray(response.data.slots) ? response.data.slots : []
      turf.available_slots = rows.map((slot) => ({
        slot_id: Number(slot.slot_id || 0),
        slot_date: slot.slot_date,
        start_time: slot.start_time,
        end_time: slot.end_time,
        base_price: Number(slot.base_price || 0)
      }))

      if (!turf.available_slots.length) {
        this.setTurfMessage(turf, 'No available slots for this date.', 'info')
        return
      }

      this.setTurfMessage(turf, '', 'success')
    } catch (error) {
      const message = error.response?.data?.status || 'Server connection failed while checking slots.'
      this.setTurfMessage(turf, message, 'error')
    } finally {
      this.slotLoadingMap = { ...this.slotLoadingMap, [turf.turf_id]: false }
    }
  },
  async applyPromoCode(turf) {
    const selected = this.getSelectedSlot(turf)
    if (!selected) {
      this.setTurfMessage(turf, 'Please select a slot before applying a promo code.', 'error')
      return
    }

    const promoCode = String(turf.promo_code || '').trim().toUpperCase()
    if (!promoCode) {
      turf.applied_promo = null
      this.setTurfMessage(turf, 'Enter a promo code first.', 'error')
      return
    }

    this.promoLoadingMap = { ...this.promoLoadingMap, [turf.turf_id]: true }
    this.setTurfMessage(turf, 'Checking promo code...', 'info')

    try {
      const response = await axios.post(VALIDATE_PROMO_ENDPOINT, {
        slot_id: selected.slot_id,
        promo_code: promoCode
```

```
      }, {
        timeout: 8000,
        headers: { 'Content-Type': 'application/json' }
      })

      if (!response.data?.success) {
        turf.applied_promo = null
        this.setTurfMessage(turf, response.data?.message || 'Promo code is not valid.', 'error')
        return
      }

      turf.promo_code = String(response.data?.promo?.code || promoCode)
      turf.applied_promo = response.data?.promo || null
      this.setTurfMessage(turf, response.data?.message || 'Promo code applied successfully.', 'success')
    } catch (error) {
      turf.applied_promo = null
      const message = error.response?.data?.message || 'Server connection failed while applying promo code.'
      this.setTurfMessage(turf, message, 'error')
    } finally {
      this.promoLoadingMap = { ...this.promoLoadingMap, [turf.turf_id]: false }
    }
  },
  async createBooking(userId, slotId, bookedPrice, promoCode = '') {
    const payload = {
      user_id: userId,
      slot_id: slotId,
      booked_price: bookedPrice,
      promo_code: String(promoCode || '').trim().toUpperCase()
    }
    const response = await axios.post(`${API_BASE}/book_slot.php`, payload, {
      timeout: 8000,
      headers: { 'Content-Type': 'application/json' }
    })
    return response.data
  },
  async bookTurf(turf) {
    const userId = this.getCurrentUserId()
    if (!userId) {
      this.$router.push('/login')
      return
    }
    if (this.isPastDate(turf.booking_date)) {
      this.setTurfMessage(turf, 'Select valid date', 'error')
      return
    }

    const selected = this.getSelectedSlot(turf)
    if (!selected) {
      this.setTurfMessage(turf, 'Please select an available slot first.', 'error')
      return
    }
    if (this.isPastSlotDateTime(selected.slot_date || turf.booking_date, selected.start_time)) {
      this.setTurfMessage(turf, 'Past time slot cannot be booked (Bangladesh time).', 'error')
      return
    }

    const breakdown = this.getPriceBreakdown(turf)
    if (!breakdown || breakdown.total <= 0) {
      this.setTurfMessage(turf, 'Invalid booking amount. Please reselect slot.', 'error')
      return
    }

    this.bookingMap = { ...this.bookingMap, [turf.turf_id]: true }
    this.setTurfMessage(turf, 'Sending booking request...', 'info')

    try {
      const bookingResult = await this.createBooking(userId, selected.slot_id, breakdown.total,
turf.applied_promo?.code || '')
      if (bookingResult?.success) {
        this.setTurfMessage(turf, bookingResult.message || 'Booking request sent successfully.', 'success')
        turf.available_slots = turf.available_slots.filter((slot) => Number(slot.slot_id) !==
Number(selected.slot_id))
```

```
        turf.selected_slot_id = 0
        turf.promo_code = ''
        turf.applied_promo = null
        return
      }
      this.setTurfMessage(turf, bookingResult?.message || 'Booking failed.', 'error')
    } catch (error) {
      const message = error.response?.data?.message || error.response?.data?.status || 'Server connection
failed.'
      this.setTurfMessage(turf, message, 'error')
    } finally {
      this.bookingMap = { ...this.bookingMap, [turf.turf_id]: false }
    }
  }
}
```

==========
73. src/support/userDashboardSupport.js
==========

```
import axios from 'axios'

const API_PROTOCOL = typeof window !== 'undefined' && window.location.protocol.startsWith('http')
  ? window.location.protocol
  : 'http:'
const API_HOST = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1'
const API_BASE = '${API_PROTOCOL}//${API_HOST}/turfbooking/backend'

export function getUserSession() {
  try {
    const raw = localStorage.getItem('user')
    return raw ? JSON.parse(raw) : null
  } catch (error) {
    return null
  }
}

export function createUserDashboardStats() {
  return { total: 0, upcoming: 0, cancelled: 0 }
}

export function createUserPointsBreakdown() {
  return { booking: 0, review: 0, resell: 0, referral: 0 }
}

function getScrollableElement(refValue) {
  if (!refValue) return null
  if (typeof refValue.scrollIntoView === 'function') return refValue
  if (refValue.$el && typeof refValue.$el.scrollIntoView === 'function') return refValue.$el
  return null
}

export const userDashboardMethods = {
  safeNavigate(path) {
    const resolved = this.$router.resolve(path)
    if (resolved.matched && resolved.matched.length > 0) {
      this.$router.push(path)
    }
  },
  switchToOwnerDashboard() {
    if (!this.isOwner) return
    localStorage.setItem('mode', 'owner')
    this.$router.push('/owner-dashboard')
  },
  switchToAdminDashboard() {
    if (!this.isAdmin) return
    localStorage.setItem('mode', 'admin')
    this.$router.push('/admin-dashboard')
  },
  goBack() {
```

```
  if (window.history.length > 1) {
    this.$router.back()
    return
  }
  this.$router.push('/login')
},
goToBookings() {
  this.safeNavigate('/bookings')
},
goToTransactions() {
  this.safeNavigate('/transactions')
},
goToTurfs() {
  this.safeNavigate('/browse')
},
scrollToWallet() {
  getScrollableElement(this.$refs.walletCard)?.scrollIntoView({ behavior: 'smooth', block: 'center' })
},
scrollToUpcoming() {
  getScrollableElement(this.$refs.upcomingCard)?.scrollIntoView({ behavior: 'smooth', block: 'center' })
},
openReview() {
  if (!this.pendingReview) return
  this.showReviewModal = true
  this.reviewMsg = ''
},
closeReview() {
  this.showReviewModal = false
},
statusClass(status) {
  return String(status || 'pending').toLowerCase()
},
statusBadgeTone(status) {
  const value = this.statusClass(status)
  if (value === 'confirmed') return 'success'
  if (value === 'pending') return 'warning'
  if (value === 'cancelled') return 'danger'
  return 'info'
},
formatDate(value) {
  if (!value) return 'N/A'
  const date = new Date(value)
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleDateString('en-US', { year: 'numeric', month: 'short', day: 'numeric' })
},
formatTime(value) {
  if (!value) return 'N/A'
  const txt = String(value)
  const parts = txt.split(':')
  if (parts.length < 2) return txt
  let hour = Number(parts[0])
  const minute = parts[1]
  if (Number.isNaN(hour)) return txt
  const ampm = hour >= 12 ? 'PM' : 'AM'
  hour %= 12
  if (hour === 0) hour = 12
  return `${hour}:${minute} ${ampm}`
},
formatDateTime(value) {
  if (!value) return 'Unknown time'
  const date = new Date(String(value).replace(' ', 'T'))
  if (Number.isNaN(date.getTime())) return value
  return date.toLocaleString('en-US', {
    month: 'short',
    day: 'numeric',
    hour: '2-digit',
    minute: '2-digit'
  })
},
formatMoney(value) {
  const amount = Number(value || 0)
  return Number.isFinite(amount) ? amount.toFixed(2) : '0.00'
```

```
  },
  async loadDashboard() {
    const response = await axios.get(`${API_BASE}/user_dashboard.php?user_id=${this.userId}`)
    if (!response.data?.success) return
    this.stats = response.data.stats || this.stats
    this.points = Number(response.data.points || 0)
    this.pointsBreakdown = response.data.points_breakdown || this.pointsBreakdown
    this.nextBooking = response.data.next_booking || null
    this.activities = Array.isArray(response.data.activities) ? response.data.activities : []
  },
  async loadPendingReview() {
    const response = await axios.get(`${API_BASE}/review_pending.php?user_id=${this.userId}`)
    if (response.data?.success) {
      this.pendingReview = response.data.pending
    }
  },
  async loadAll() {
    try {
      await Promise.all([this.loadDashboard(), this.loadPendingReview()])
    } catch (error) {
      console.error(error)
    }
  },
  async cancelUpcomingBooking() {
    if (!this.nextBooking?.booking_id || this.cancelUpcomingLoading) return

    this.cancelUpcomingLoading = true
    this.upcomingActionMsg = ''

    try {
      const response = await axios.post(`${API_BASE}/cancel_booking.php`, {
        user_id: Number(this.userId || 0),
        booking_id: Number(this.nextBooking.booking_id)
      }, {
        headers: { 'Content-Type': 'application/json' },
        timeout: 10000
      })

      if (response.data?.success) {
        const refund = this.formatMoney(response.data.refund_amount)
        this.upcomingActionMsg = `Cancellation request sent. After admin approval: Refund Tk ${refund} (80%)`
        await this.loadAll()
      } else {
        this.upcomingActionMsg = response.data?.message || 'Could not cancel booking.'
      }
    } catch (error) {
      this.upcomingActionMsg = error.response?.data?.message || 'Cancellation failed.'
    } finally {
      this.cancelUpcomingLoading = false
    }
  },
  async submitReview() {
    if (!this.pendingReview) return
    if (this.rating < 1) {
      this.reviewMsg = 'Please select a rating (1-5).'
      return
    }

    this.reviewLoading = true
    this.reviewMsg = ''

    const payload = new URLSearchParams()
    payload.append('user_id', this.userId)
    payload.append('booking_id', this.pendingReview.booking_id)
    payload.append('turf_id', this.pendingReview.turf_id)
    payload.append('rating', this.rating)
    payload.append('comment', this.comment)

    try {
      const response = await axios.post(`${API_BASE}/review_submit.php`, payload, {
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
      })
```

```
      this.reviewMsg = response.data?.message || 'Done'
      if (response.data?.success) {
        this.pendingReview = null
        this.rating = 0
        this.comment = ''
        this.showReviewModal = false
        await this.loadAll()
      }
    } catch (error) {
      console.error(error)
      this.reviewMsg = 'Review submit failed.'
    }

    this.reviewLoading = false
  },
  logout() {
    localStorage.clear()
    this.$router.push('/login')
  }
}
```